

國立成功大學
資訊工程學系
碩士論文

建構及預測區域 OD 矩陣 - 以 YouBike 為例

Construct and predict the regional Origin-Destination Matrix
from rental data - using the YouBike system in Taipei

研究生：蕭詠麟

指導教授：李 強 博士

中華民國一百零九年七月

摘要

Origin-Destination Matrix (OD 矩陣)是空間資訊的專有名詞，其有效檢視道路的交通流量、城市的人流狀況、公共交通的需求程度等。OD 矩陣亦被廣泛應用在交通、旅遊、商業、運輸等方面，因此深受學者及企業研究與應用。然而，OD 矩陣的建構及預測是十分複雜與困難。首先在建構方面，OD 矩陣建構過程中如果沒有適當定義起點與終點，容易造成資源的浪費，而且過往的方法定義起終點並沒有考慮資料的特性，容易把屬性不相似的資料歸類為同一個起終點，可能對後續預測的效果造成影響。在 OD 矩陣預測方面，空間因素的資訊無疑是預測的重要一環，但資料的空間特性往往難以表示，過往方法對於資料的空間屬性表示也過於簡單，未必能有效提升模型預測的效果。再者，外在因素間接影響模型預測的結果，例如雨天減少戶外地區的人流、週末提升觀光景點的交通量、上下班時段公共交通需求量增加等，因此在建構及預測 OD 矩陣時我們都必需考慮完整因素。鑑於以上建構及預測 OD 矩陣的問題，本論文開發了一套完整的 OD 矩陣建構及預測框架。在建構方面，我們基於 DBSCAN 開發 HTS-DBSCAN 演算法，該方法有效針對資料的距離與時間特性進行分群，同時改善傳統 DBSCAN 雜訊過多及資料無法切割的問題。在預測方面，我們提出了空間資料表示的方法，有效表示資料的距離與影響範圍。另外，也加入多種的外在因素進行預測，藉此提升模型預測的效果。在本論文最後，我們利用台灣台北市的 YouBike 借用量資料進行了一系列的實驗，並得到了不錯的成果。

關鍵字：OD 矩陣；空間資訊；空間資訊分群；人流預測

目錄

摘要	2
目錄	3
圖片目錄	5
表格目錄	7
I. Introduction	8
II. Related Work	18
2.1 DBSCAN 相關研究	18
2.1.1 DBSCAN	18
2.1.2 ST-DBSCAN	18
2.1.3 HDBSCAN	18
2.2 深度學習	19
2.2.1 Artificial Neural Network	19
2.2.2 Deep Neural Network	19
2.2.3 Convolutional Neural Network	20
2.2.4 Long Short Term Memory	20
2.3 人流與目的地預測相關研究	21
2.3.1 ST-RNN	21
2.3.2 POI2Vec	21
2.3.3 Deep Spatio-temporal Residual Network	22
2.3.4 Deep Multi-View Spatial-Temporal Network	22
2.4 OD 矩陣相關研究	23
2.4.1 Grid-Embedding based Multi-task Learning	23
2.4.2 Efficient OD Trip Matrix Prediction Based on Tensor Decomposition	23
2.4.3 MUlti-task Representation learning model for Arrival Time estimation	24
III. Data Set	25
3.1 台北市 YouBike 資料	25
3.2 台北市 2016 年天氣資料	26
IV. Algorithm	28
4.1 站點分群	28
4.1.1 DBSCAN	29
4.1.2 Time Series DBSCAN(TS-DBSCAN)	35
4.1.3 Hierarchical Time Series DBSCAN(HTS-DBSCAN)	42
4.2 歷史區域 OD 矩陣建構	46

4.2.1 HTS-DBSCAN 雜訊點處理	47
4.2.2 區域 OD 矩陣起點與終點定義	49
4.2.3 歷史區域 OD 矩陣建構	50
4.3 未來 OD 矩陣預測	52
4.3.1 歷史因素建模	52
4.3.2 空間因素建模	53
4.3.3 外在因素建模	57
4.3.4 綜合因素預測	58
V. Experiment	59
5.1 站點分群	59
5.1.1 DBSCAN、TS-DBSCAN 與 HTS-DBSCAN 結果比較分析	59
5.1.2 HTS-DBSCAN 與 K-means 結果比較分析	61
5.2 區域 OD 矩陣預測	65
5.2.1 其他模型比較	66
5.2.2 不同時間單位預測	67
5.2.3 單一群體及所有群體預測	76
5.2.4 模型因素分析	78
5.2.5 輸入歷史時間點數量分析	81
VI. Conclusion	83
6.1 優點與貢獻	83
6.2 結論	83
References	85

圖片目錄

Figure 1.1 台北市地圖	8
Figure 1.2 台北市 YouBike 站點分佈地圖	10
Figure 1.3 東湖國中與東湖國小位置地圖	11
Figure 1.4 (a)東湖國中一天的 YouBike 借用量	11
Figure 1.4 (b)東湖國小一天的 YouBike 借用量	11
Figure 1.5 不同站點一天借用量	14
Figure 1.6 人流量與空間因素示意圖	15
Figure 1.7 空間因素示意圖	16
Figure 1.8 群體空間因素示意圖	17
Figure 2.1 Deep Neural Network	19
Figure 2.2 VGG16	20
Figure 2.3 Long Short Term Memory(LSTM)	20
Figure 2.4 ST-RNN 架構圖	21
Figure 2.5 ST-ResNet	22
Figure 2.6 GEML	23
Figure 2.7 MURAT	24
Figure 4.1 建構與預測 OD 矩陣的系統架構	28
Figure 4.2 DBSCAN 基本概念和定義：(a)從 q 可達到 p 密度；(b) p 和 q 通過 o 彼此 密度連接；(c) border object, core object 及 noise	30
Figure 4.3 DBSCAN 演算法圖解	32
Figure 4.4 DBSCAN 的分群結果	34
Figure 4.5 站點借用量折線圖	36
Figure 4.6 TS-DBSCAN 演算法圖解	38
Figure 4.7 TS-DBSCAN 分群結果	42
Figure 4.8 HTS-DBSCAN 演算法圖解	44
Figure 4.8 HTS-DBSCAN 雜訊點處理流程圖解	48
Figure 4.10 經過雜訊點處理後的分群結果	50

Figure 4.11 群體標示示意圖.....	50
Figure 4.12 OD 矩陣預測深度學習架構.....	52
Figure 4.13 LSTM 擷取歷史因素的特徵.....	53
Figure 4.14 CNN 擷取空間因素的特徵.....	54
Figure 4.15 群體 A 對於其他群體的空間因素圖.....	55
Figure 4.16 群體空間因素圖片	55
Figure 4.17 以群體中心計 Decay Function.....	56
Figure 4.18 DNN 擷取空間因素的特徵.....	57
Figure 4.19 綜合因素預測	58
Figure 5.1 DBSCAN 的分群結果	60
Figure 5.2 TS-DBSCAN 的分群結果	61
Figure 5.3 HTS-DBSCAN 的分群結果	61
Figure 5.4 K-means 分群結果.....	62
Figure 5.5 起點與終點為群體 6 的真實結果與預測結果比對折線圖	68
Figure 5.6 群體 6 始點到達群體 10 的真實結果與預測結果比對折線圖	69
Figure 5.7 群體 6 始點到達群體 12 的真實結果與預測結果比對折線圖	69
Figure 5.8 群體 6 始點到達群體 2 的真實結果與預測結果比對折線圖	70
Figure 5.9 以時段為時間單位之真實結果與預測結果比對折線圖	73
Figure 5.10 以天為時間單位之真實結果與預測結果比對折線圖	75
Figure 5.11 各種模型因素真實與預測結果比對圖 - 小時.....	79
Figure 5.12 各種模型因素真實與預測結果比對圖- 時段.....	81

表格目錄

Table 1.1 台北市早上七點至八點 YouBike 的 OD 矩陣	9
Table 3.1 (a)台北市 YouBike 站點資料.....	25
Table 3.1 (b)台北市 2016 年 YouBike 借用量資料	25
Table 3.2 (a)台北市 2016 年天氣資料.....	26
Table 3.2 (b) 30 種天氣 Condition	27
Table 3.2 (c)台北市 2016 年天氣 Condition 資料.....	27
Table 4.1 歷史區域 OD 矩陣起點與終點的定義	50
Table 4.2 區域歷史 OD 矩陣示意表格	51
Table 5.1 開發環境與設備	59
Table 5.2 HTS-DBSCAN 與 K-means 各群體數據	63
Table 5.3 HTS-DBSCAN 與 K-means 分群結果綜合數據	64
Table 5.4 一天八個時段的时间分佈與描述	66
Table 5.5 以三小時時段進行比較	67
Table 5.6 以小時為時間單位預測	68
Table 5.7 以時段為時間單位預測	71
Table 5.8 以天為時間單位預測	74
Table 5.9 以小時為單位預測及學習單一群體的 OD 矩陣	77
Table 5.10 以時段為單位預測及學習所有群體的 OD 矩陣	77
Table 5.11 各種模型因素比對分析 - 小時.....	78
Table 5.12 各種模型因素比對分析 - 時段.....	80
Table 5.13 輸入歷史時間點數量分析	82

I. Introduction

Origin-Destination Matrix (OD 矩陣) 是空間資訊中的一個專有名詞，其目的是表示從起點到終點的交通流量或人口移動量，就以下 Figure 1.1 台北市地圖為例，我們把台北市地圖分為 11 個區域，分別是北投、士林、大同、信義等地區，並把 11 個地區標示為表一中的起點與終點，表一中的縱軸標題是起點，而橫軸標題則是終點。假設表一為台北市早上七點至八點 YouBike 的 OD 矩陣，而我們則可以在表中得知台北市七點至八點時，從北投區內使用 YouBike 移動的人有 1022 人，而從北投區作為起點使用 YouBike 出發到達士林區的人分別有 247 人，從士林區到達中正區的人 1384 人，如此類推。經過查詢 Table1.1 的 OD 矩陣，我們可以清楚了解台北市早上七點至八點使用 YouBike 的人口移動量，該人口移動量間接反映了台北市所有地區的交通情況，以及各地區市民的生活方式，而台北市的交通管理工程師可以跟據人流量的變動改善台北市公共交通工具的班次，以及交通道路的管制，減少路段塞車與擁擠的情況出現。



Figure 1.1 台北市地圖

Table 1.1 台北市早上七點至八點 YouBike 的 OD 矩陣

起點 \ 終點	北投	士林	大同	中山	...	中正	大安	信義	南港	文山
北投	1022	247	582	1382	...	1481	994	1537	1932	236
士林	125	943	894	1823	...	1384	1239	1742	2112	312
					...					
文山	154	213	934	919	...	1047	1758	1689	2405	672

OD 矩陣可以清楚了解各個地區在不同時段的人流量，該人流量間接反映市民的生活情況與地區的交通流量等資訊，因此大量的企業及學者都致力使用 OD 矩陣來進行城市的各項研究，如 Lingbo Liu 等人 [1]利用 OD 矩陣預測計程車需求，令計程車公司可跟據需求量預先分配計程車到不同區域。另外，Shudong Liu 等人 [2]針對使用者出發時間和出發點的坐標預測用戶的旅行目的地。Y.Englezou 等人 [3]利用 OD 矩陣推估交通路段的流量及需求。Adrien Ickowicz 等人 [4]利用渡輪運輸數據進行 OD 矩陣的評估。Juha-Matti Kuusinen 等人 [5]利用電梯進出資料推估 OD 矩陣，目的是去估計樓層之間的乘客流量。由上述例子可見，OD 矩陣對於公共交通的需求、道路流量的評估、城市人流的管制等方面都非常有幫助，因此本論文也希望針對 OD 矩陣進行研究，提出一套有效於建與預測 OD 矩陣的框架

然而，OD 矩陣的建構與人流的預測是十分複雜與困難的。當中例如 Yuandong Wang 等人 [6]利用出租車資料建構及預測 OD 矩陣，該方法是先針對地圖以網格的方式進行切割，並把資料跟據網格建構 OD 矩陣，再利用深度學習學習每個網格的語義表示，最後再以 LSTM 進行預測 OD 矩陣。另外，Huaxiu Yao [7]利用計程車資料針對空間、時間與語義的深度學習建模，預測每個地區的計程車需求量。再者，Anahid Nabavi Larijani [8]利用手機數據來調查巴黎及其郊區內始發目的地流的行為，該方法先針對手機的軌跡定義起點與終點，而且需要處理每個手機訊號發射站的影響範圍，

最後再分析巴黎及其郊區的人流量變化。綜觀上述，建構及預測 OD 需要繁多複雜的步驟，如資料轉換、地區處理、預測建模等流程。為了清楚表示 OD 建構及預測上的複雜，我們把當中的困難點羅列如下：

問題一：OD 容易浪費資源空間

以 2016 年台北市 YouBike 為例，台北市佔地面積 271.8 平方公里，並劃分為 12 個區，全市的 YouBike 站點約 200 多個站點，如 Figure 1.2 YouBike 站點分佈地圖，其中站點較多的區域分別為大安區(31 個)、信義區(26 個)、中山區(25 個)。假設我們直接利用所有 YouBike 站點定義 OD 矩陣的起點與終點，會使該 OD 矩陣成為 200×200 的矩陣，如果我們要紀錄 2016 年一整年，並以小時作為紀錄的單位，該 OD 矩陣所佔用的空間為 $200 \times 200 \times 366 \text{ 天} \times 24 \text{ 小時} = 351,360,000$ 筆資料，這會造成大量的資源浪費，而且過大的 OD 矩陣也會影響 OD 矩陣後續預測的效果。

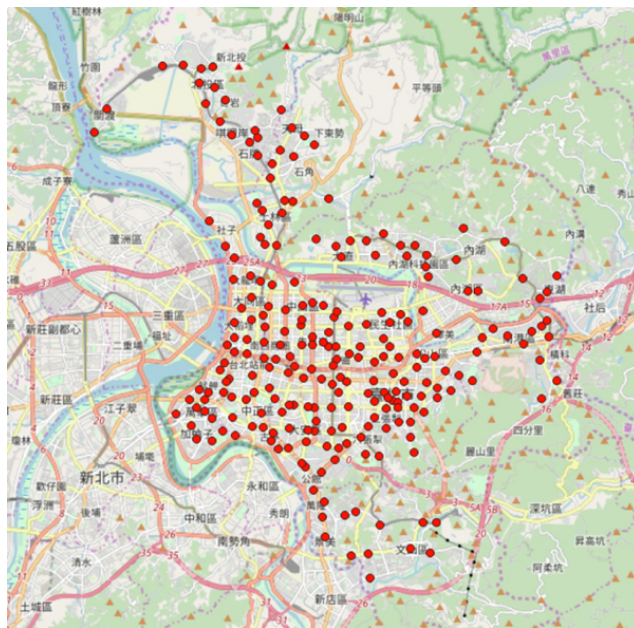


Figure 1.2 台北市 YouBike 站點分佈地圖

另外，由於台北市佔地面積少，部份站點距離與屬性相近，若同時把兩個屬性相近的站點定義 OD 矩陣起終點容易加大資源的浪費。如 Figure 1.3 東湖國中與東湖國小為例，兩個站點距離約 650 米，而且從 Figure 1.4 顯示兩個站點借用量相似，借用量的高峰時段範圍都是在早上 6 點至 10 點，其他時段的借用量相對較少，因此我們

可以推斷該兩個站點的屬性為住宅區，因為大多數的乘客都是在早上時段借用腳踏車上班或上課，假若把多組屬性相近或重覆站點同時定義 OD 矩陣的起終點，容易造成不必資源浪費。



Figure 1.3 東湖國中與東湖國小位置地圖

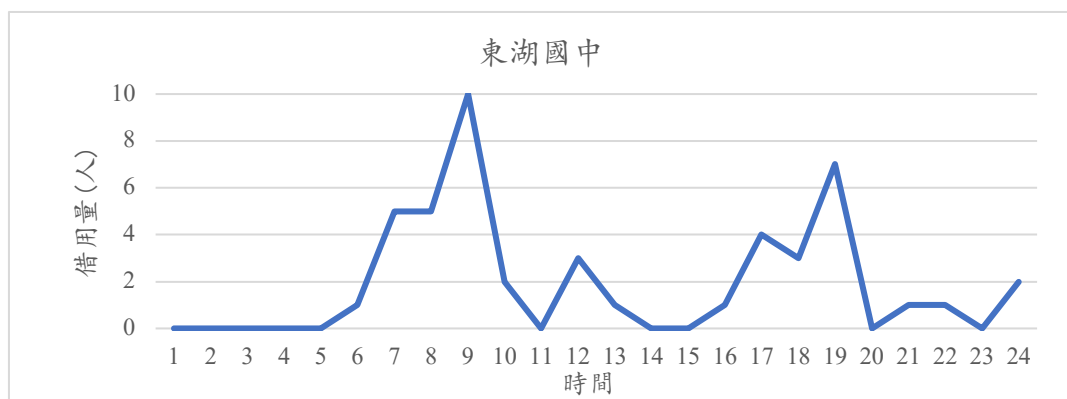


Figure 1.4 (a)東湖國中一天的 YouBike 借用量

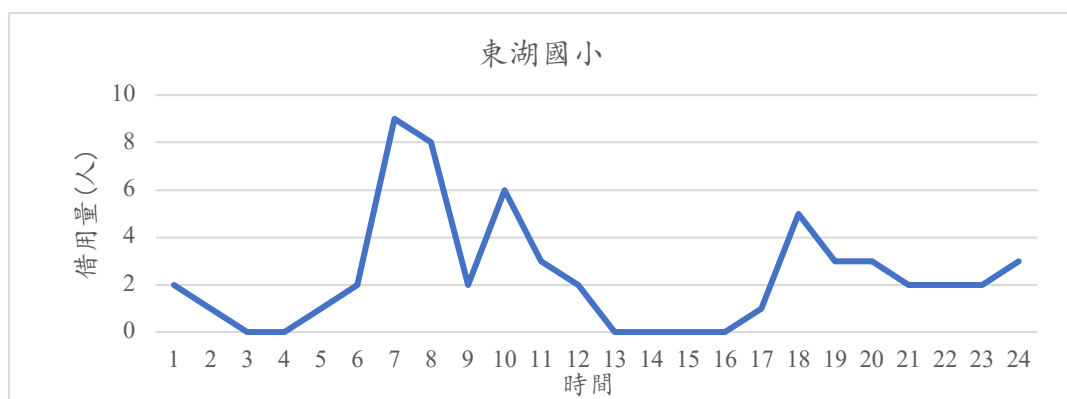


Figure 1.4 (b)東湖國小一天的 YouBike 借用量

問題二：地圖中的空間資訊難以討論

空間因素對預測 OD 矩陣無疑是重要的一環，然而空間的資料是難以獲得，在缺乏或稀少的空間資源下，我們難以完整表示各個區域的空間因素。就本論文 YouBike 站點為例，每個站點所包含的資訊為經緯度與不同時段的借用量，當中只有前者與空間因素有關，後者則與時間因素有關。假設我們單純利用經緯度或站點之間的距離作為空間因素表示進行預測，這無疑並不足夠，因為每個站點除了經緯度與距離外，站點的影響範圍也非常重要，站點影響範圍間接表示站點所服務的區域，服務的區域越大則代表借用量越大，因此我們可以透過站點影響範圍提升預測 OD 矩陣準確性，不能單純考量經緯度與距離，而忽略站點影響範圍。

然而要表示站點影響範圍是十分困難的，而且當同時針對站點的經緯度、距離與影響範圍進行建模時，輸入格式的定義並不容易。當中距離所包含的可能是 $1 \times n$ 個維度， n 為某一個站點與所有站點的距離，而面對過多維度的輸入也會加大模型學習的難度。另外，以單一數字表示站點的影響範圍未必能夠完善表達真實的意義，因為對於距離站點 100 公尺與 500 公尺兩個相同屬性的建築物而言，前者 100 公尺的建築物對於站點的影響程度一定是較 500 公尺建築物大，因此如果以單一數字表示影響範圍容易造成資訊失真的問題。

再者，當站點經過分群後，以群體的方式定義距離與影響範圍時，定義難度則大大提升。例如以群體為中心計算距離，則對群體外圍的站點或範圍過大的群體造成空間資訊的失真。另外分群的結果形狀與大小不一，難以表示定義其真正的服務區域，由此可見，空間因素有限的狀況下，不論是針對站點或群體都難以定義其表示的方式。

問題三：外在因素影響預測

OD 矩陣預測與人流預測容易受大量的外在因素影響，大量的外在因素都間接反映站點借用與歸還量的多寡，例如週末可能提升觀光地區站點的借用量與歸還量，平日則對企業區或住宅區的使用情況有影響。另外時間也直接影響企業區或住宅區的借

用率，在早上的時候住宅區周遭的站點會較多人使用，大多數人在早上借用腳踏車上班及上課，下午或晚上則是相反，大多數人在企業區或學校區附近的站點借用腳踏車回家。天氣更是直接影響腳踏車的使用狀況，下雨、颱風、打雷等惡劣天氣直接大大降低使市民腳踏車的意願。由以上可見外在因素對於 OD 矩陣與人流預測的重要性，每個因素都直接影響站點的使用狀況，對於嚴重影響預測的效果。

由以上的問題可見，建構及預測區域 OD 矩陣是相當困難，以下我們會針對以上的問題描述過往研究的解決方法：

問題一過往解決方法：站點分群

面對以上的問題，過往大多數都是針對雜亂的站點進行分群，希望透過不同的分群方法解決以上站點過多的問題。現時分群方法中被廣泛應用在空間資訊中大多數是利用 K-means 與 DBSCAN。首先，K-means 優點在於他只需要給定一個群體數量的參數，該方法便會跟據使用者所給定的參數進行分群。然而，面對台北市複雜與凌亂的站點，如 Figure 1.2，我們無法清楚明確定義所有站點需要成群的數量，給定的參數過大或過少都會影響分群的結果，因此針對我們的研究，我們無法利用 K-means 進行分群。另外，DBSCAN 的優點最主要分為三個：1. 不需要事先給定分群的數量，只需要輸入距離與最少成群站點數，方法便會自動跟據兩個參數進行分群，解決 K-means 需要給定群體數量參數的問題；2. DBSCAN 可以在分群的過程中發現任意形狀的群體，面對規劃複雜的城市與不規則分佈的站點，DBSCAN 有助於我們找出不符合參數及任意形狀的群體；3. DBSCAN 可以在分群的過程中找出不符合參數的站點成為雜訊點，因此我們可以針對雜訊點進行過濾與分析。由以上的兩個分群的方法可見，DBSCAN 較為適合使用在我們研究上。

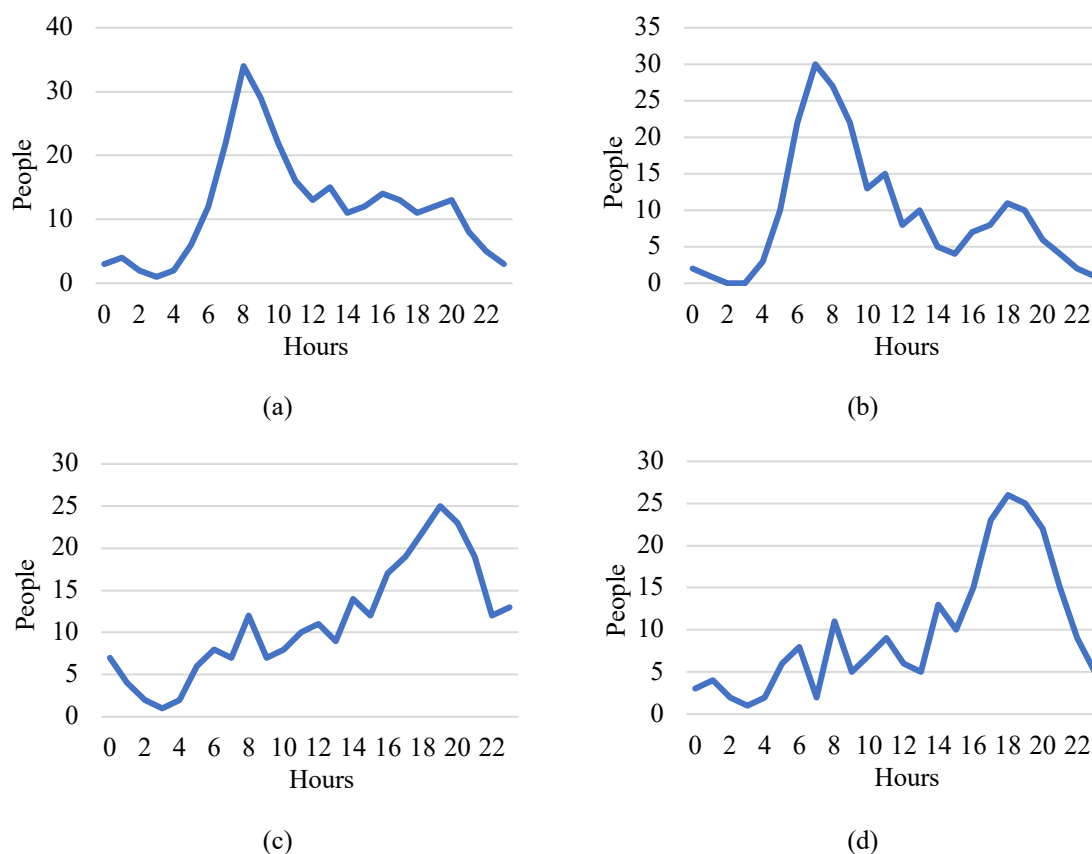


Figure 1.5 不同站點一天借用量

DBSCAN 最主要考慮距離因素進行分群，然而每個站點不單只包含距離一個因素，每個站點會牽涉到時間與借用量等因素，時間與借用量可以反映出站點的屬性，在 Figure 1.5(a)與(b)一天的借用量都十分相似，最主要都是早上時段借用量較多，而(c)與(d)則下午及晚上時段較多借用量，因此我們可以藉此推斷(a)與(b)為住宅區，因為大多數市民在早上從住宅區借用腳踏車上班與上課，而(c)與(d)為企業區，較多人下午及晚上借用腳踏車下班回家。如果我們只單純考慮距離進行分群，我們可能會忽略每個站點潛在的重要因素，容易把不同屬性的站點歸類為同一群。

過往與時間有關的 DBSCAN 的方法大多數針對單一個時間點或考慮是否在時間範圍內進行分群，然而以上過往與時間有關的 DBSCAN 都無法應用本論文的研究上，本論文希望是考慮站點一天的借用量進行分群，而站點一天借用量的格式為時間序列，因此無法直接套用在過往的 DBSCAN 上。

問題二過往解決方法：圖片表示空間因素



Figure 1.6 人流量與空間因素示意圖

過程針對空間因素大多數以圖片方式作為表示，他們把與人流量與空間因素進行結合，先把站點或區域標示在圖片中，再以顏色的深度定義人流量的多寡，如 Figure 1.6，此方法被大眾擴範應用 [6] [7] [9]。然而以上的方式有兩個缺點：(1) 人流量為時間因素，距離與影響範圍屬於空間因素，把兩個不同因素的資料結合進行深度學習的建模及預測，可能加大學習的難度，甚至影響預測的效果。(2) Figure 1.6 中主要以人流量與距離等因素作為考量，然而圖中缺乏考慮該區域的影響範圍，以致空間因素考量不完整。

問題三過往解決方法：加入大量外在因素

外在因素間接影響人流或 OD 矩陣的預測，因此學者們 [9] [7] 希望加入大量的外在因素提升預測的準確性，例如降雨量、風向、濕度等。然而過多的因素可能加深學習的難度，造成反效果。再者，某些因素對於預測當地的敏感度不高，未必能有效提升預測的效果，以本論文為例，台北市是經常下雨的城市，然而當地大多數都是短暫微雨為主，而短暫微雨的降雨量其實并不高，與晴天的降雨量差異並不大，這對於模型學習而言可能難以分別晴天與短暫微雨的天氣，形成模型對資料敏感度的問題。可是短暫的微雨也會影響 YouBike 的使用量，間接影響區域 OD 矩陣預測的問題。

鑑於以上建構及預測區域 OD 矩陣的困難及過往方法的問題，本論文就以上的困難及問題開發一套完整建構及預測區域 OD 矩陣的架構，以下為本架構針對以上各個問題所提出的解決方法。

本論文所提出的問題一解決方法：時間序列分群

在本論文中我們考慮到傳統與過往 DBSCAN 無法考慮時間序列的問題，因此我們希望提出一個新的 DBSCAN，以針對站點距離與一天借用量的時間序列進行分群。在分群的過程中我們會計算每個站點的距離與時間序列的相似度，當距離與時間序列的相似度均合乎使用者所給定的參數時，DBSCAN 便會自動把合乎的站點組成群體，藉此把所有屬性相似的站點歸類為同一個族體，並利用分群後的結果定義我們 OD 矩陣中的起終點，以解決站點過多難以定義 OD 矩陣起終點的問題。

本論文所提出的問題二解決方法：建立空間圖片

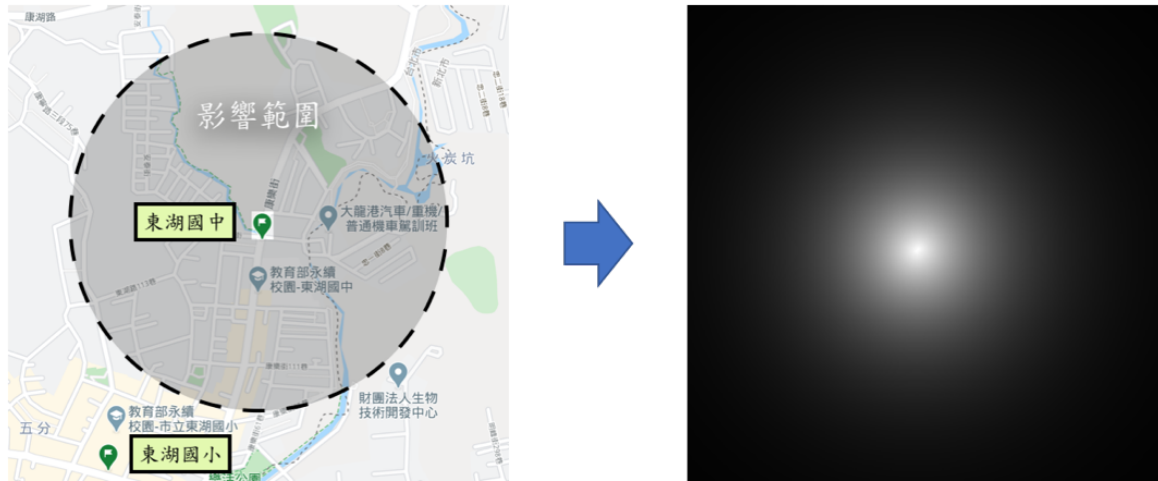


Figure 1.7 空間因素示意圖

針對過往圖片表示空間因素的問題，我們利用 Decay Function [29]計算每個站點所影響的範圍，並把計算所得的結果以圖片進行表示，形成空間因素示意圖，如 Figure 1.7。然後，我們會跟據分群的結果，把各個群體中所有站點的空間因素示意圖組成群體空間因素示意圖，如 Figure 1.8，並利用 Convolutional Neural Network 擷取群體之間的距離，以及群體中每個站點的影響範圍，藉此學習每張空間因素示意圖的潛在特徵，提升 OD 矩陣預測的精準度。

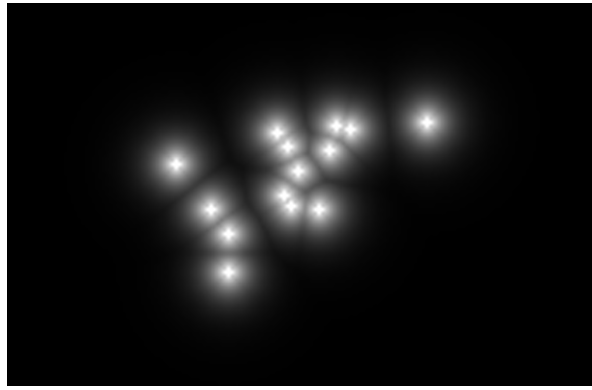


Figure 1.8 群體空間因素示意圖

本論文所提出的問題三解決方法：加入多種外在因素預測

鑑於以上外在因素對於站點影響的重要性，在本論文中我們會加入不同的外在因素解決以上的問題，並利用深度學習針對所有外在因素進行學習，避免與其他因素混淆影響學習效果，並藉此提取外在因素的特徵。再者我們採用天氣條件表示預測目標當天的狀況，如晴天、多雲、颱風，大雨等條件直接函蓋完整的天氣狀況，避免加入大量額外天氣的因素如降雨量、溫度、風向等加深模型學習的難度影響預測效果。另外我們以 one-hot 的方式作為輸入，利用 Embedding 的方式快速學習所有外在因素的潛在特徵表示。

以上是本論文考慮到建構與預測 OD 矩陣所潛在的種種問題，因此我們以台北市 YouBike 自行車的使用資料作為實驗的對象，並提出有效及完整建構與預測 OD 矩陣的流程方法解以上所提及的問題，最後亦設計多種實驗方式證明方法的可行性。

以下是本論文所包含的所有貢獻：

- (1) 開發一套利用歷史交通資料預測未來 OD 矩陣的流程框架，減低 OD 矩陣浪費資源的問題及有效預測各個群體起點到終點的人流量。
- (2) 開發 Hierarchical Time Series DBSCAN (HTS-DBSCAN)，能考慮時間序列列與距離進行行行分群。
- (3) 同時考慮站點經緯度、距離與影響範圍建立分群後的群體空間示意圖。
- (4) 開發有效跟據歷史、空間與外在因素，預測未來區域 OD 矩陣深度學習框架。

II. Related Work

本章節中最主要分為四個子章節，分別為 OD 矩陣相關研究、DBSCAN 相關研究、深度學習及人流與目的地預測相關研究。在 2.1 會先講解 DBSCAN 相關研究，由於本論文會開發以時間序列進行分群的 DBSCAN 演算法，因此我們會探討過往與時間有關或較為著名的 DBSCAN 演算法。在 2.2 章節中我們會探討深度學習的歷史與相關研究。在 2.3 中我們介紹人流與目的地預測相關研究，由於本論文所研究的 OD 矩陣的目的是了解各個區域的人口移動量，因此人流與目的地預測有莫大的相似性。最後，在 2.4 中會介紹過往建構及預測 OD 矩陣的相關研究，從中探討過往建構及預測 OD 矩陣所提出的方法。

2.1 DBSCAN 相關研究

2.1.1 DBSCAN

DBSCAN 是 [10]1996 年由 Martin Ester 等人所提出的分群演算法，該演算法優點在於不需要事先定義群體數量，另外在分群時可區分雜訊，再者可以發現任意形狀的族群。DBSCAN 只需要給定兩個參數：(1)族群形成所需的最少點數；(2)點與點之間的最大距離，即可分群。

2.1.2 ST-DBSCAN

ST-DBSCAN [11]是 2007 年由 Derya Birant 等人所開發的，該演算法主要的優點是可以以時間與空間的因素進行分群，而該演算法具備了根據資料的非空間、空間和時間值發現不同的族群。

2.1.3 HDBSCAN

HDBSCAN [12]是由 Leland McInnes 等人所開發的，其概念是先根據資料的密度與稀疏度對空間進行轉換，轉換後建立距離加權的最小生成樹，然後利用最小生成樹建構連接組件的群體層次結構，並根據最小的群體大小壓縮群體層次的結構，最後從

壓縮樹中提取穩定的族群。然而該方法對於對於資料中較為邊界的點在分群時未能有理想的效果。

2.2 深度學習

2.2.1 Artificial Neural Network

Artificial Neural Network(ANN)的概念基本上是從生物學中引入的，ANN 中的神經元與人體中的神經元十分相似，ANN 透過所有神經元相互連接的幫助下傳遞信息，接收的神經元會把信息進行調整，調整後會得出預測的答案，預測答案與真實答案計算差異後，把每一個神經元進行倒傳遞，讓預測的結果更加接近真實狀況。

2.2.2 Deep Neural Network

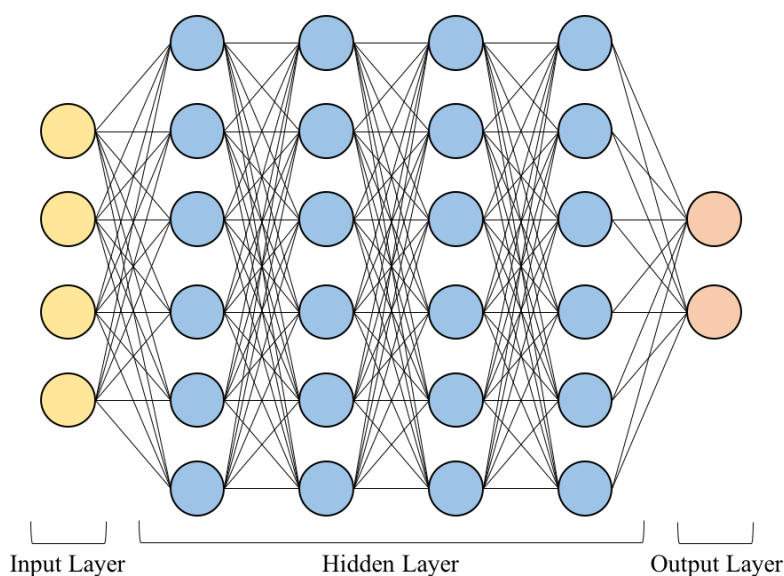


Figure 2.1 Deep Neural Network

Deep Neural Network(DNN)最主要是由 Input Layer、Hidden Layer 與 Output Layer 所組成，如 Figure 2.1。其概念與 ANN 相同，通過倒傳遞更新不同神經元的權重，DNN 由於擁有多層的隱藏層，因此相對 ANN 而言能學習更複雜的資訊。由於 DNN 效果不錯，因此被廣泛應用到不同的層面上，例如交通管理、醫療系統、商業分析等，解決以往難以利用演算法處理的問題。

2.2.3 Convolutional Neural Network

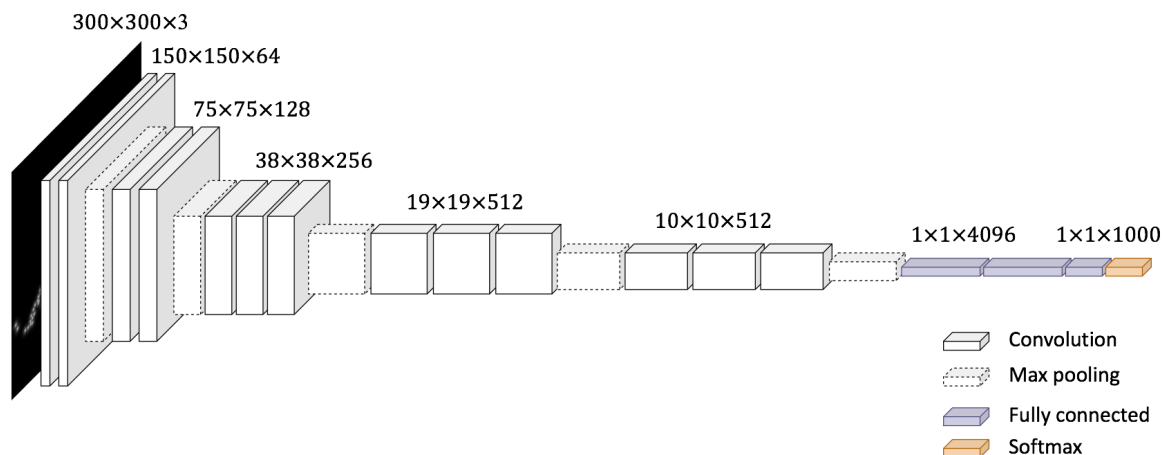


Figure 2.2 VGG16

Convolutional Neural Network(CNN)是由多個卷積層、池化層與全連接層所組成的，其應用層面大多數為圖像辨識、語音處理、特徵特取等，CNN 當中 VGG16[13] 被公認為處理圖像辨識與特徵特取等工作有著良好的效果，VGG16 是由 16 組卷積層、池化層與全連接層所組成的，如 Figure 2.2。CNN 衍生不同的深度學習模型如 ResNet [14]、DenseNet [15]、ShuffleNet [16]等，均有著優異的表現。

2.2.4 Long Short Term Memory

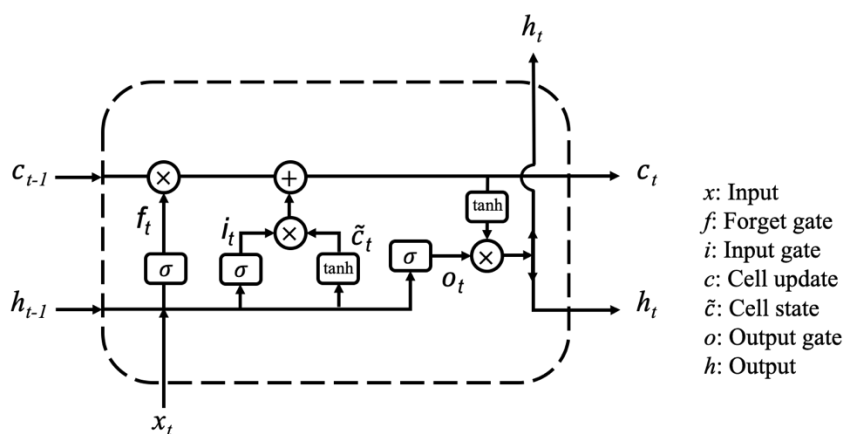


Figure 2.3 Long Short Term Memory(LSTM)

Long Short Term Memory(LSTM)是 Recurrent Neural Network(RNN)的改良，其改良了 RNN 本身梯度消失(Gradient Vanish)問題，LSTM 最主要由 Input Gate、Forget

Gate 與 Output Gate 三個 Gate 所組成的。如 Figure 2.3 所示， x_t 分別傳到 Input Gate、Forget Gate 與 Output Gate，Input Gate 負責決定本次 Input 是否進入 LSTM 中，Forget Gate 則負責決定是否忘記 LSTM 所保留的值，而 Output Gate 決定本次的 Input 是否可以進行輸出。LSTM 藉著三個 Gate 改善 RNN 誤差梯度隨著事件間的時間長度成指數般的消失的問題，而 LSTM 亦衍生不同的時間序列模型如 GRU [17]、Convolutional LSTM [18]、LSTNet [19] 等。

2.3 人流與目的地預測相關研究

2.3.1 ST-RNN

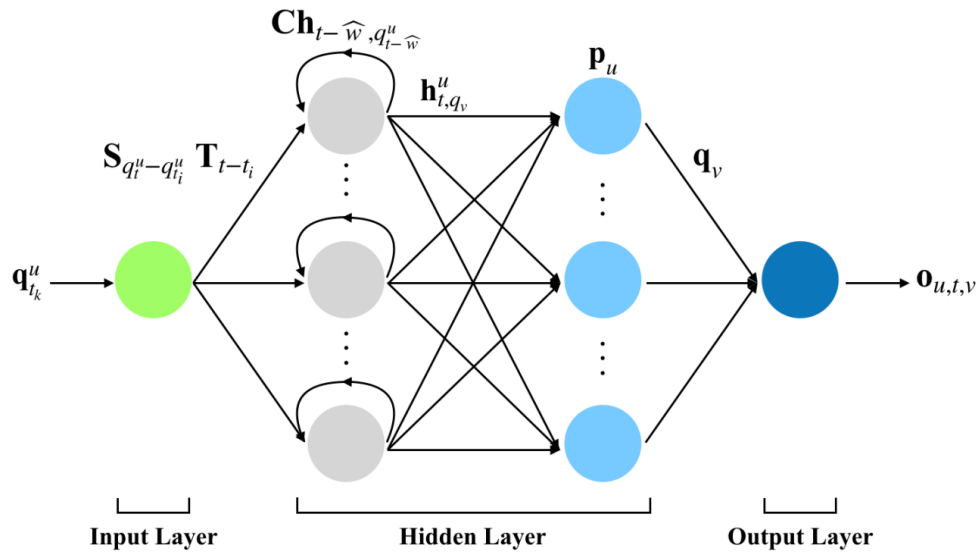


Figure 2.4 ST-RNN 架構圖

ST-RNN [20] 是由 Qiang Liu 等人開發的模型，該模型考慮了過往方法沒有考慮使用者行為之間的時間間隔、位置之間的連續地理距離與使用者個人特徵或偏好預測目的地。因此 ST-RNN 結合時間、空間與使用者行為，該模型會輸入使用者目前的位置，並考慮所有目的地的時間間隔與距離特徵，最後加入使用者行為因素，預測使用者下一個到達的目的地。

2.3.2 POI2Vec

POI2Vec [21] 該方法是給定一個目的地，並預測該目的地的潛在使用者，而該方

法是改良 Word2Vec [22]中的 CBOW 模型，CBOW 是把所有的景點轉為向量，學習各個景點之間的相關性。該模型是先給定一個目的地 A，模型會跟據使用者在某時間點內所到達過的目的地行為，預測使用者在未來會否到達目的地 A。

2.3.3 Deep Spatio-temporal Residual Network

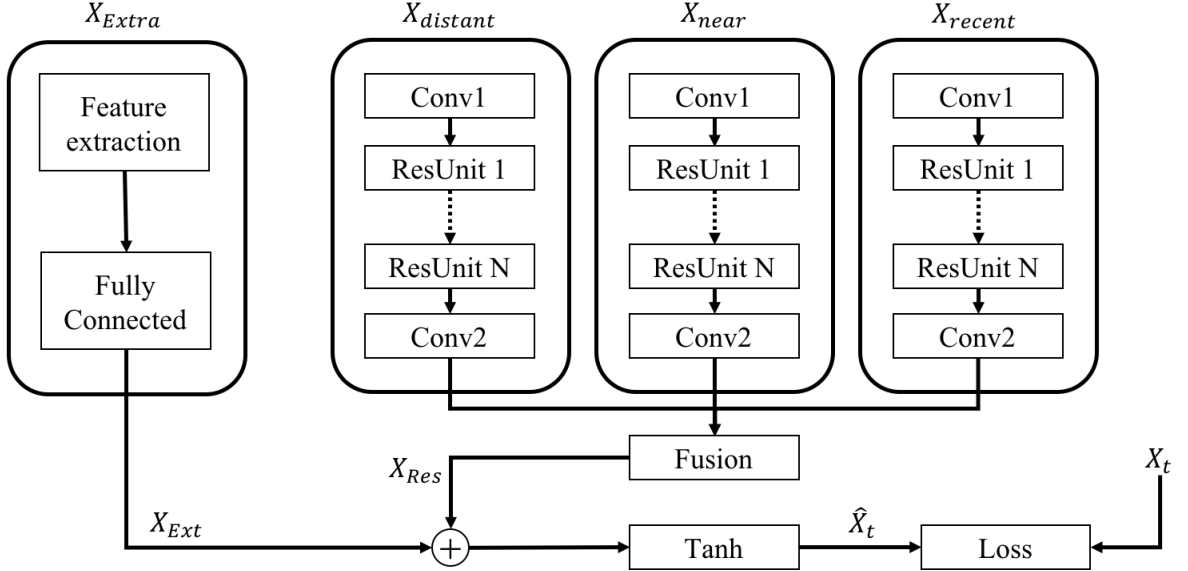


Figure 2.5 ST-ResNet

Deep Spatio-temporal Residual Network [9](ST-ResNet)是由 Junbo Zhang 等人所開發的，該方法的目的是利用預測人群流量對交通管理和公共安全，他們把歷史的人流資料轉成人流空間圖片，再把圖片分成遠、中、近三個時段，各自輸入至 ResNet 模型進行建模，另外利用 Fully connected 擷取外在因素的特徵，最後預測每個地區中人群的最終流量，如 Figure 2.5。該方法利用北京和紐約市進行的兩種人群流動的實驗。

2.3.4 Deep Multi-View Spatial-Temporal Network

Deep Multi-View Spatial-Temporal Network [7] (DMVST-Net)目的是針對計程車的需求量進行預測，該方法考慮了空間、時間、天氣與時間相關性四種因素，他們先利用 CNN 針對需求量的空間圖進行建模，然後把 CNN 輸出的結果加入天氣的因素輸入 LSTM 學習時間的因素，另外利用 Embedding 的方式學習每個地點的語義表示，最後針對地點的語義表示與時間因素輸入至 Fully connected 進行預測。

2.4 OD 矩陣相關研究

2.4.1 Grid-Embedding based Multi-task Learning

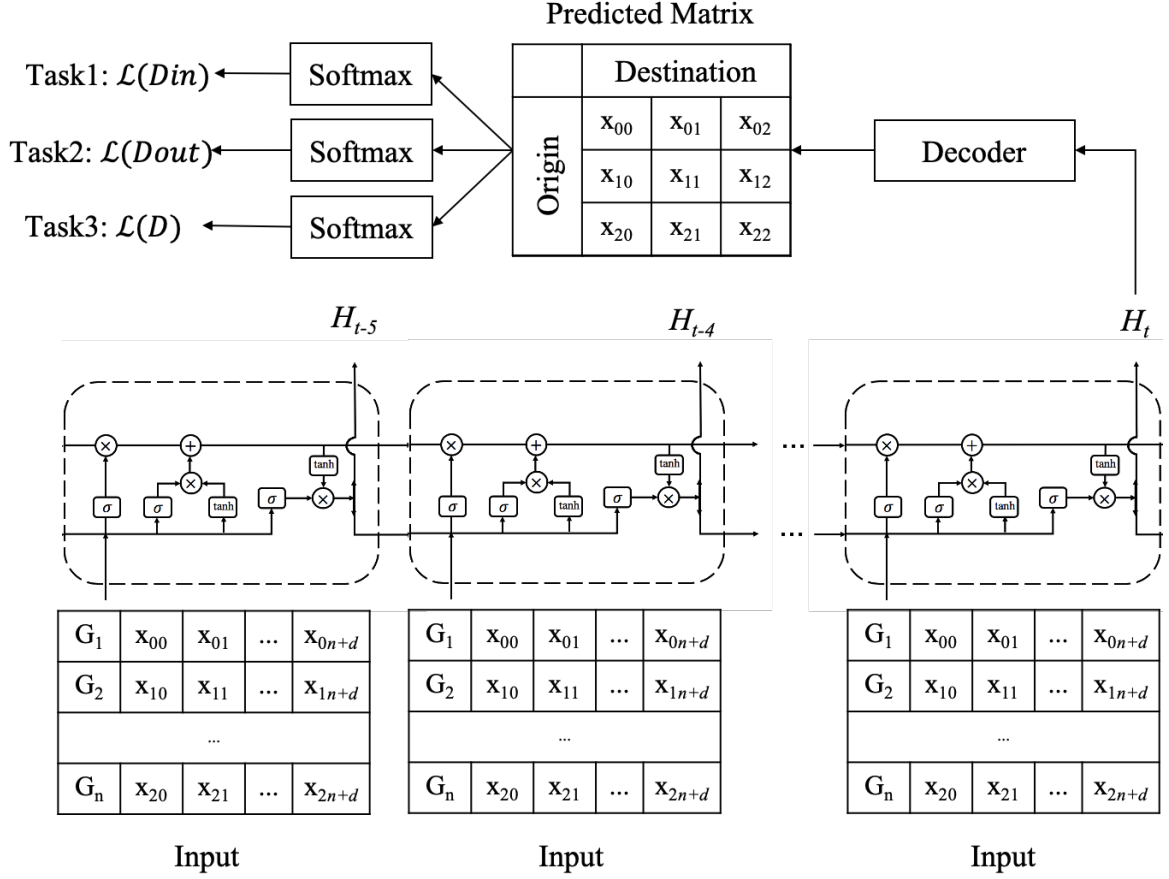


Figure 2.6 GEML

Grid-Embedding based Multi-task Learning [6] (GEML)是由 Yuandong Wang 等人所開發的，他們是把地圖以網格的方式進行切割，接著把網格嵌入擷取乘客的空間移動性模式和不同區域的鄰域關係利用 Graph Convolutional Networks 進行建模，然後再把 Graph Convolutional Networks 建模完成的網格語義表示與歷史流量輸入 LSTM 中，並針對時間屬性建模，如 Figure 2.6，從而學習 OD 矩陣特性並進行預測，在該研究中他們以兩個叫車應用軟體 UCAR 和 Didi 所生成的兩個真實世界數據集進行實驗。

2.4.2 Efficient OD Trip Matrix Prediction Based on Tensor Decomposition

Jiangtao Ren [23]考慮到傳統 ARIMA，SVR 和神經網絡等方法，在處理多維 OD

需求數據時會導致效率下降，因此他們基於時間的車輛 OD 行程矩陣建模為由四個屬性組成的四階向量：起點，目的地，車輛類型和時間，通過採用 CANDECOMP / PARAFAC(CP)向量分解進行預測，該方法展示如何使用向量分解來預測時間因子矩陣中的未來流量需求。他們利用廣東高速公路收費數據集進行實驗，該方法證明有效地減少預測時間。

2.4.3 Multi-task Representation learning model for Arrival Time estimation

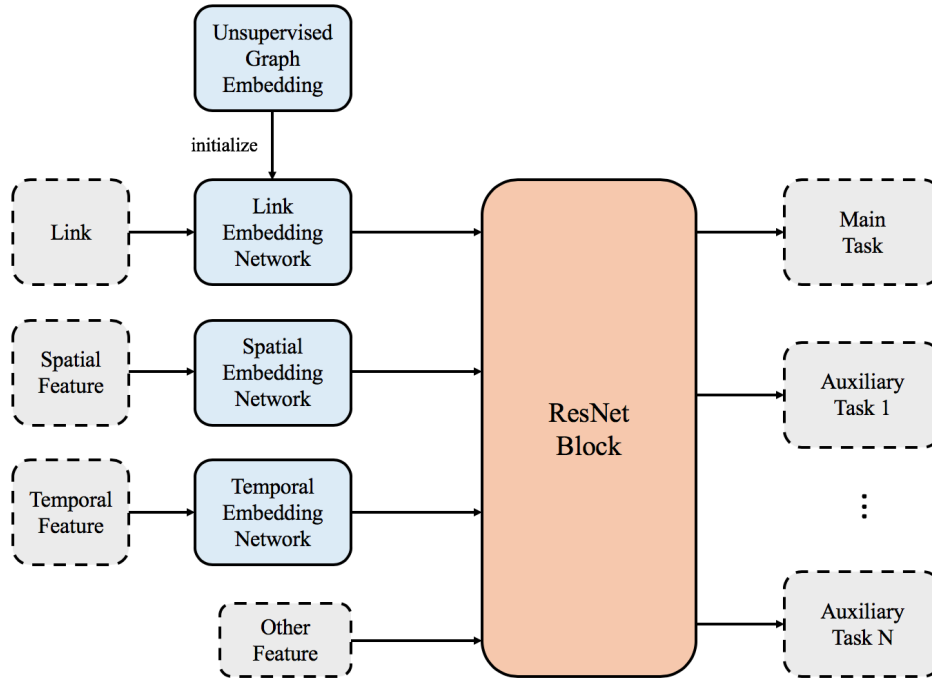


Figure 2.7 MURAT

MURAT (Multi-task Representation learning model for Arrival Time estimation) [24] is developed by Yaguang Li et al. This method considers that the travel route given by the user may change dynamically due to traffic conditions, user preferences, etc. Different destinations are easy to cause errors in estimation, so their purpose is to predict travel time without online route information. This method produces a meaningful representation form, which retains various travel attributes in the real world, and also utilizes the basic road network and spatiotemporal features for estimation. In addition, in the training stage, historical travel path information is used to improve performance.

III. Datasets

在本章節中我們將介紹在本論文所使用到的資料集，資料集總共分為兩個種類：

(1) 台灣 YouBike 資料；(2) 台北市天氣資料，以下我們會跟據兩種資料集進行詳細的介紹。

3.1 台北市 YouBike 資料

台北市 YouBike 資料來源為政府資料開放平臺網站 [25]。該資料可分為(1) 台北市 YouBike 站點資料及(2) 台北市 YouBike 借用量資料兩項，如 Table 3.1 所示。

Table 3.1 (a) 台北市 YouBike 站點資料

Column Name	Explanation
sno	YouBike 站點編號
sna	YouBike 站點名稱
sarea	站點所位於的地域
address	站點所位於地址
latitude	站點所位於的緯度
longitude	站點所位於的經度

Table 3.1 (b) 台北市 2016 年 YouBike 借用量資料

Column Name	Explanation
ChargeTime	乘客歸還腳踏車時，所收費的時間
BorrowTime	乘客借用腳踏車的時間
BorrowStation	乘客借用腳踏車的站點
ReturnTime	乘客歸還腳踏車的時間
ReturnStation	乘客歸還腳踏車的站點
UsingTime	乘客借用腳踏車的時長

Table 3.1(a) 其資料來源是由 YouBike 公共自行車 [26] 所提供，資料集包含了 2015 到 2017 年的台北市 YouBike 站點，而我們會對應 Table 3.1(b) 借用量資料把不存在 2016 年的站點進行清洗，例如原本屬於 2015 年或以前的站點，經過 2016 年被移除或更改，導致無法對應 Table 3.1(b)，針對以上的站點我們會把它進行過濾。此外，也會刪除借用量過低或異常的站點。經過處理後，Table 3.1(a) 總共包含 241 個站

點。

Table 3.1(b)是台北市 2016 年 YouBike 借用量資料，該資料的日期範圍為 2016 年 1 月 1 日至 12 月 31 日，共 366 天借用量資料，每份資料紀錄當天所有使用者借用腳踏車的起點與終點。另外，由於台北市與新北市是相連的，因此大量的市民會從台北市借用腳踏車到達新北市或從新北市到達台北市的站點，所以當某筆借用資料其中的借用與歸還站點不屬於台北市的站點，我們會把該筆資料刪除，最後 Table 3.1(b)只保留借用站點與歸還站點都位於台北市的資料，

3.2 台北市 2016 年天氣資料

由於天氣的狀況間接影響腳踏車的借用狀況，因此本論文亦加入天氣資料提升區域 OD 矩陣預測的效果，Table 3.2 為本論文所使用到的天氣資料。

Table 3.2 (a)台北市 2016 年天氣資料

Column Name	Explanation
Time	時間
Temperature	溫度，單位為攝氏
Dew Point	露點溫度
Humidity	濕度
Wind	風向
Wind Speend	風向速度
Wind Gust	陣風
Pressure	氣壓
Precip.	降水量
Condition	天氣的狀態，總共 30 種狀態

Table 3.2(a)是台北市 2016 年的天氣資料，資料來源為 Weather Underground [27] 網站，該網站是 IBM Cloud 的一項商業天氣服務，可透過互聯網提供實時的天氣訊息。該資料的日期範圍為 2016 年 1 月 1 日至 12 月 31 日共 366 天，並紀錄每小時的天氣狀況。在本研究中我們只會利用 Table 3.2(a)中 Time 與 Condition 兩個欄位，由於 Condition 能直接表示該小時的天氣狀況，所以我們利用 Condition 取代其他天氣狀

況的欄位。Condition 共包括 30 個天氣狀態，如 Table 3.2(b)。刪除其他天氣欄位後，Table 3.2(a) 格式則表示為 Table 3.2(c)。

Table 3.2 (b) 30 種天氣 Condition

Condition		Condition	
1	Cloudy	16	Light Rain with Thunder
2	Fair	17	Mostly Cloudy
3	Fair / Windy	18	Mostly Cloudy / Windy
4	Fog	19	Partly Cloudy
5	Haze	20	Partly Cloudy / Windy
6	Heavy Rain	21	Rain
7	Heavy Rain Shower	22	Rain Shower
8	Heavy Rain Shower / Windy	23	Rain Shower / Windy
9	Heavy T-Storm	24	Showers in the Vicinity
10	Light Drizzle	25	Smoke
11	Light Drizzle / Windy	26	T-Storm
12	Light Rain	27	T-Storm / Windy
13	Light Rain / Windy	28	Thunder
14	Light Rain Shower	29	Thunder / Windy
15	Light Rain Shower / Windy	30	Thunder in the Vicinity

Table 3.2 (c) 台北市 2016 年天氣 Condition 資料

Column Name	Explanation
Time	時間
Condition	天氣的狀態，總共 30 種狀態

IV. Algorithm

本章節會介紹本論文所提出建構與預測 OD 矩陣的方法，Figure 4.1 為本論文所提出的完整系統架構，系統架構最主要分為三個部份，最左邊 HTS-DBSCAN grouping 是我們利用本論文所提出的 Hierarchical Time Series DBSCAN 針對 Table 3.1 台北市 YouBike 站點資料進行分群，該方法會考慮站點的距離與借用量進行分群，分群後的結果我們會同時利用 Table 3.1(b)台北市 2016 年 YouBike 借用量資料建立歷史 OD 矩陣，則架構圖中間 Build Origin-Destination Matrix。最後我們會把建構完成的歷史 OD 矩陣與外在因素資料作為訓練模型的資料，並利用架構圖右邊我們所開發的模型，預測未來 OD 矩陣。在本章節共有三個子章節，我們首先會在 4.1 章節介紹站點分群；在 4.2 章節介紹歷史 OD 矩陣建構；在 4.3 章節介紹未來 OD 矩陣預測。

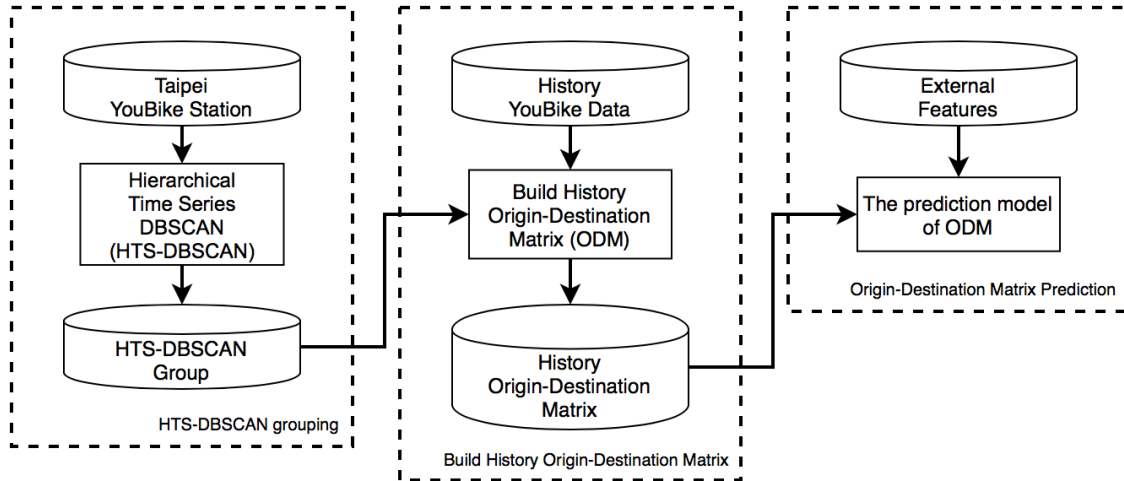


Figure 4.1 建構與預測 OD 矩陣的系統架構

4.1 站點分群

為了避免站點過多影響 OD 矩陣浪費大量資源與預測的效果，因此我們會先針對站點進行分群，而我們利用 DBSCAN 進行分群，原因最主要分為三個：(1)不需要事先給定分群的數量；(2)可以發現任意形狀的族群；(3)能夠識出資料中的雜訊點。以下我們會先用傳統的 DBSCAN 進行分群，另外再介紹本論文基於 DBSCAN 所改良的兩種針對時間序列分群的方法：(1)Time Series DBSCAN(TS-DBSCAN)以站點距離

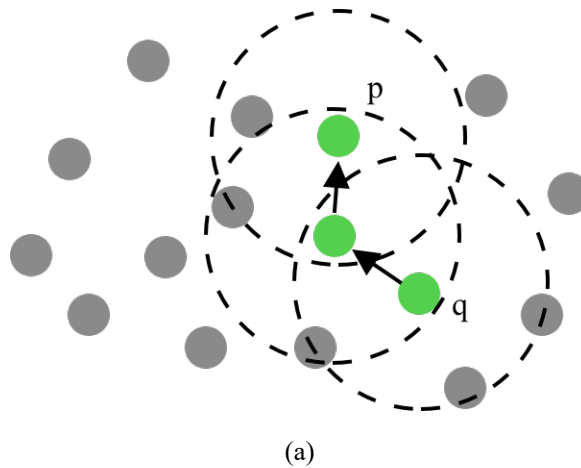
與借用量作為時間與空間的考慮因素進行分群，解決 DBSCAN 以單一距離因素進行分群的不完整性；(2) Hierarchical Time Series DBSCAN (HTS-DBSCAN) 利用階層式的方式分群，先以距離進行第一次分群，再以距離與借用量二個因素進行第二次的分群，解決 TS-DBSCAN 雜訊點過多的問題。

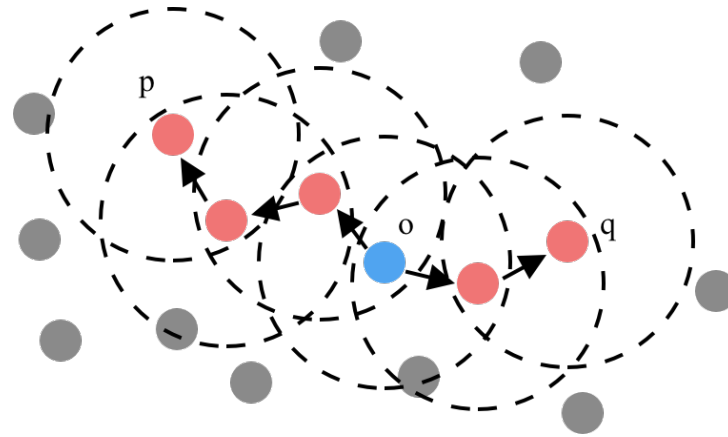
在本章節中的 4.1.1 中，我們會先以介紹 DBSCAN 的基本定義與演算法流程，並探討 DBSCAN 分群上的缺失。在 4.1.2 中我們會跟據 DBSCAN 分群上的缺失開發 TS-DBSCAN，並解說 TS-DBSCAN 的定義與演算法流程步驟，而且也會檢視 TS-DBSCAN 的分群問題。最後在 4.1.3 中提出 HTS-DBSCAN 解決 DBSCAN 與 TS-DBSCAN 分群上的問題，並針對 HTS-DBSCAN 進行詳細的解說及其方法流程。

4.1.1 DBSCAN

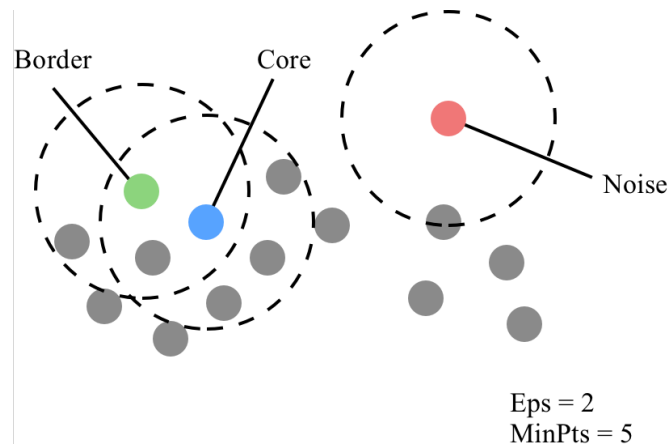
4.1.1.1 DBSCAN 基本概念與定義

DBSCAN 主要是發現任何數據集中任意形狀的簇群，同時可以區分雜訊點。更具體地說，DBSCAN 是基於用戶定義的距離半徑 Eps ，在 Eps 半徑內出現的點數是否超過 $MinPts$ ，藉此找出資料集中的群體。以下為 DBSCAN 演算法介紹基本概念和定義：





(b)



(c)

Figure 4.2 DBSCAN 基本概念和定義：(a)從 q 可達到 p 密度；(b) p 和 q 通過 o 彼此密度連接；(c) border object, core object 及 noise

定義 1 給定一個資料集 $S = \{o_1, o_2, \dots, o_n\}$ ，演算法會把 S 基於相似的屬性測量分割為 $C = \{C_1, C_2, \dots, C_k\}$ ，稱為分群。當 $C_i \subseteq S, (i = 1, 2, \dots, k), \cap_{i=1}^k C_i = \emptyset$ 及 $\cup_{i=1}^k C_i = S$ ， C_i 稱為族群，

定義 2 (Neighborhood) 它由兩個物件 p 和 q 的距離函數（例如 Manhattan Distance, Euclidean Distance）判斷，由 $\text{dist}(p, q)$ 表示。

定義 3 (Eps-neighborhood) 物件 p 的 *Eps-neighborhood* 是由 $\{q \in S | \text{dist}(p, q) \leq \text{Eps}\}$ 定義。

定義 4 (Core object) Core object 是指某個點其給定半徑 (*Eps*) 的鄰域必須至少包含最小數量 (*MinPts*) 的其他點。

定義 5 (Directly density-reachable) 如果 p 在 q 的 Eps 範圍內，並且 q 是核心物件，則物件 p 可以直接從物件 q 到達密度。

定義 6 (Density-reachable) 如果有一連串的物件 $p_1, \dots, p_n$ ， $p_1 = q$ 和 $p_n = p$ ，並且其密度都是由 p_i 所到達的，並符合 Eps 和 $MinPts$ ，且 $1 \leq i \leq n, p_i \in S$ ， p 的密度則是由 q 所直達。

定義 7 (Density-connected) 如果存在對象 $o \in S$ ，則對象 p 相對於 Eps 和 $MinPts$ 密度連接到對象 q ，使得 p 和 q 相對於 Eps 和 $MinPts$ 密度都可以從 o 到達密度。

定義 8 (Density-based cluster) 集群 C 是 S 的一個非空子集，滿足以下「Maximality」和「Connectivity」要求：

(1) $\forall p, q$: 如果 $q \in C$ 和 p 的密度可由 q 所到達，並符合 Eps 和 $MinPts$ ，則 $p \in C$ 。

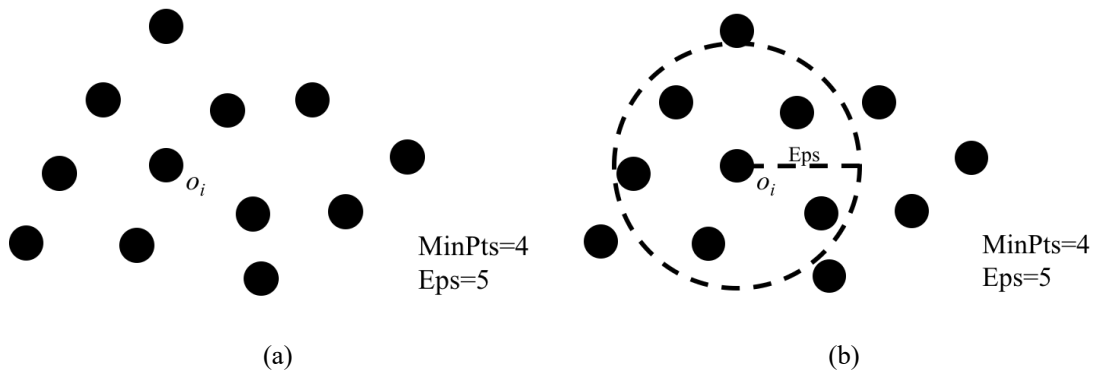
(2) $\forall p, q \in C$: p 的密度與 q 連接，並符合 Eps 和 $MinPts$ 。

定義 9 (Border object) 如果物件 p 不是核心物件，但密度可以從另一個核心對象到達，物件 p 則是邊界物件。

定義 10 (Noise object) *Noise object* 是指 o 其給定半徑 (Eps) 的鄰域中，所包含的其他點數量不符合最小數量 ($MinPts$) 時， o 則被定義為 *Noise object*。

4.1.1.2 DBSCAN 演算法流程

DBSCAN 最主要分為以下四個步驟：



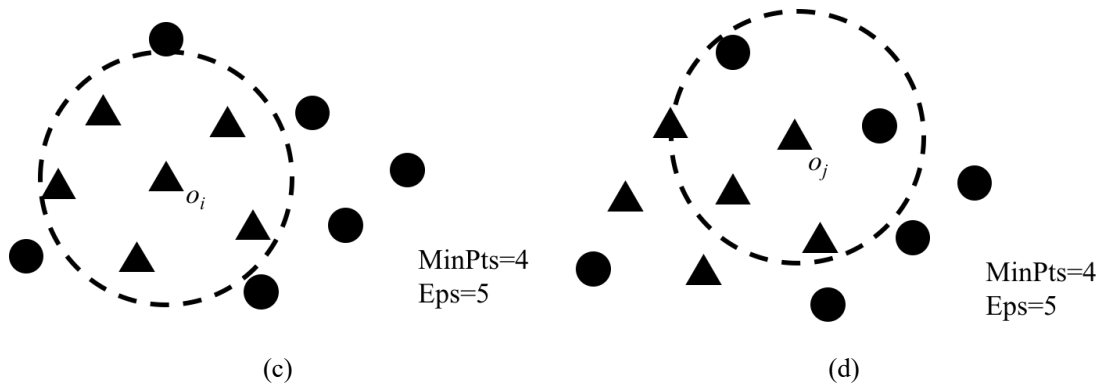


Figure 4.3 DBSCAN 演算法圖解

- Step1.** 從資料集 S 中取出一個站點，稱作為 o_i ，如 Figure 4.3(a)。
- Step2.** 以 Eps 作為半徑，擴張 o_i 的範圍，利用 Euclidean Distance 計算 o_i 範圍中所包含的站點數，如 Figure 4.3(b)。
- Step3.** 如果 o_i 半徑範圍內的站點數目多於或等於 $MinPts$ ，則標記 o_i 與 o_i 半徑範圍內的站點為新的群體，否則標為雜訊點，如 Figure 4.3(c)。
- Step4.** 從 o_i 半徑範圍內取出一個點，稱作為 o_j ， o_j 重覆 Step2，直到所有站點都擴張完成，如 Figure 4.3(d)。

Euclidean Distance

在 4.1.1.2 章節的 Step2 中，針對兩個站點之間的距離我們利用 Euclidean Distance，該函數需要給定兩個點即公式 4.1.1 中的 q 與 p ，並計算 q_i 與 p_i 兩個點之間坐標的距離， n 為的 q 與 p 所包含坐標數目，其數學公式如下：

$$d_{L2}(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \quad 4.1.1$$

4.1.1.3 DBSCAN 虛擬程式碼

Algorithm1: DBSCAN 是 DBSCAN 虛擬程式碼，並假設 S 作為所有站點的資料集，一開始我們會先從 S 中取出一個點(即 p_i)，檢查 p_i 是否已進行分群，如果已分群則從 S 中取下一點，如果 p_i 還沒有分群則跟據 Eps 作為半徑，計算 p_i 周圍所包含

的點，並取出存放在 X 中(line 3 - line 5)，檢查 X 的 *Neighborhood* 數目是否少於 $MinPts$ ，如果少於 $MinPts$ ， p_i 會被標記為噪音點(line 6 - line 7)，如果 X 的鄰居數目大於或等於 $MinPts$ ，我們會把 $cluster_label$ 加 1 存放到 $cluster_label$ 形成一個新的族群，並把 p_i 標記為 $cluster_label$ 族群(line 9 - line 10)，然後把 X 中所有的點也標記為 $cluster_label$ 族群(line 11 - line 13)，接著再把 X 的所有點加入至 R 集合中(line 14)。我們再把 R 中的點取出其中一個點 r ，計算 r 周圍所包含的點取放在 Y 中(line 13 - line 17)，如果 Y 所包含點的數目大過等於 $MinPts$ 時，取出 Y 中第一個點(即 Y_i)，如果 Y_i 不是雜訊點以及還沒有分群，則把 Y_i 加入到 R 中，直到 Y 中所有的點都檢查完成(line 18 - line 22)，該方法直到 S 中所有的點都已經檢查過一次後便會終止。

Algorithm1: DBSACN

Input : **S:** $\{c_1, c_2, \dots, c_k\}$ Set of objects
 Eps: Maximum geographic coordinates distance value
 MinPts: Minimum number of points within Eps
Output : **C:** $\{c_1, c_2, \dots, c_k\}$ Set of clusters

```

1      Initialize return integer  $cluster\_label = 0$ ;
2      Initialize return set  $R = \emptyset$ ;
3      For  $i=1$  to  $|S|$  do
4          If  $p_i$  is not cluster Then
5               $X = \text{Get\_Neighbors}(S, p_i, Eps)$ ;
6              If  $|X|$  less than  $MinPts$  Then
7                  Mark  $p_i$  is noise;
8              Else
9                   $cluster\_label = cluster\_label + 1$ ;
10                 Mark  $p_i$  be  $cluster\_label$ ;
11                 For  $j = 1$  to  $|X|$  do

```

```

12           Mark All point in  $X$  be cluster_label;
13       End
14        $R.push(X)$ ;
15       While  $R$  is not  $\emptyset$  do
16            $r \leftarrow R.dequeue()$ ;
17            $Y = \text{Get\_Neighbors}(S, r, Eps)$ ;
18           If  $|Y|$  greater than equal to  $MinPts$  then
19               For  $j = 1$  to  $|Y|$  do
20                   If  $Y_i$  is not noise and  $Y_i$  is not cluster do
21                       Mark  $Y_i$  be cluster_label;
22                        $R.push(Y_i)$ ;
23                   End
24               End
25           End
26       End
27   End
28 End
29 End

```

4.1.1.4 DBSCAN 分群問題

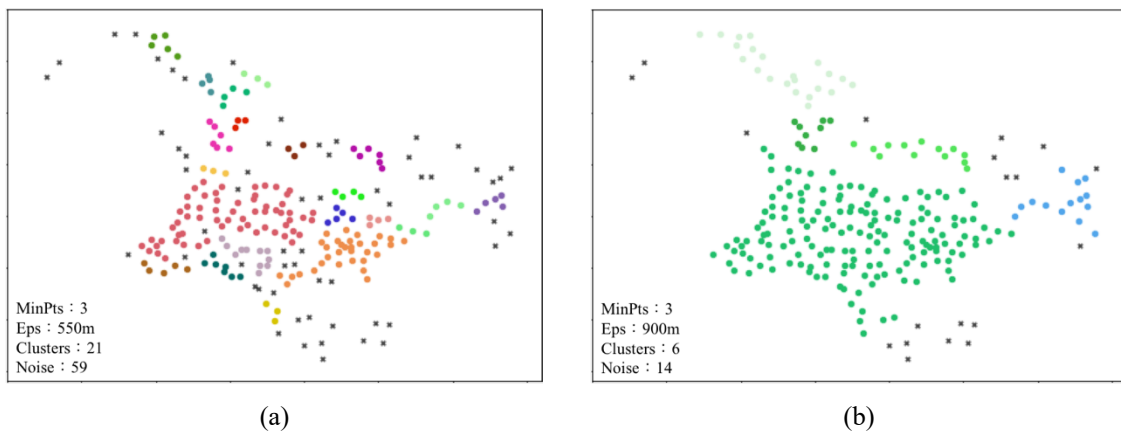


Figure 4.4 DBSCAN 的分群結果

Figure 4.4 是利用 DBSCAN 以兩個不同半徑距離針對站點進行分群的結果，Figure 4.4(a)是以 *MinPts* 為 3 及半徑 550m 進行分群的結果，結果包含 21 個群體與 59 個雜訊點。Figure 4.4(b)是以 *MinPts* 為 3 及半徑 900m 進行分群的結果，結果包含 6 個群體與 14 個雜訊點。我們也可從兩個分群結果中發現不同的問題，以 Figure 4.4(a) 為例，當我們把半徑距離設定太小，面對偏遠的站點 DBSCAN 無法進行分群，最後造成 59 個雜訊點，對只有 241 個站點的資料集而言雜訊點的數量是頗高的。Figure 4.4(b)中，當我們把半徑距離設定太大時，241 個站點只有分割為 6 群，部分站點過於密集，導致大量站點無法分割，如圖中深綠色的群體，嚴重影響分群的效果。

然而從上述我們可以明顯看出，傳統的 DBSCAN 僅考慮距離進行分群，很明顯具有巨大之的缺失，即為沒有考慮時間因素資訊。此外，過往改良的 DBSCAN 也都僅單純考慮單一時間點進行分群，如果我們只考慮單一時間點的使用量進行分類，可能會導致不同屬於的空間點位分為同一個族群，影響最終預測或查詢的效果，因此我們需要開發新的 DBSCAN，跟據站點的距離與時間因素進行分群，解決 DBSCAN 分群上的缺失。

從上述我們可以明顯看出，DBSCAN 以距離進行分群，很明顯具有巨大之的缺失，半徑距離太小造成大量雜訊點，半徑距離太大則因站點過於密集導致無法分割。再者，DBSCAN 僅考慮距離進行分群，忽略了原本站點持有的時間因素如借用量，未能完整考慮站點原有的特性，容易導致屬性不相容的站點歸類為同一群體，影響最終預測或查詢 OD 矩陣的效果。因此我們要開發新的 DBSCAN，跟據距離與時間序列進行分類，改善 DBSCAN 在分群上所出現的問題。

4.1.2 Time Series DBSCAN(TS-DBSCAN)

在 4.1.1 章節中的我們利用 DBSCAN 進行站點分群，然而分群的結果未如理想，當半徑距離太小造成大量雜訊點，半徑距離太大則因站點過於密集導致進行無法分割。針對以上的結果，證明在本研究中我們無法單純利用距離因素進行分群。在考慮使用

其他因素針對站點分群前，我們要先對每個站點進行了解，分析每個站點何種因素最能表示站點的意義和特性。

在 Table 3.1(b)台北市 2016 年 YouBike 借用量資料中，該資料紀錄了每個站點一天的借用量，而借用量則反映該站點周遭民眾的生活習慣，而我們可以把每個站點的借量各別擷取進行分析，從中了解每個站點的特性與意義。Figure4.5(a)與(b)分別是站點 A 與站點 B 的借用量，從兩個圖中我們可以發現站點 A 在早上六點至十點時借用量較多，站點 B 則是相反，大約概在下午五點至晚上十點時借用量較多。我們可以從中推斷站點 A 早上較多人借用的原因是大多民眾在該時段上班與上課，屬性與住宅區十分相似。站點 B 則較多民眾在下午至晚上時段下班或下課，屬性與企業區較為相似。

由 Figure4.5(a)與(b)可見，借用量可間接反映站點的特性，因此我們希望開發一個新的 DBSCAN，以借用量作站點時間上的因素，並同時考量距離進行分群，從而相同屬性的站點進行歸類，亦希望解決 4.1.1 章節中 DBSCAN 分群上的問題。

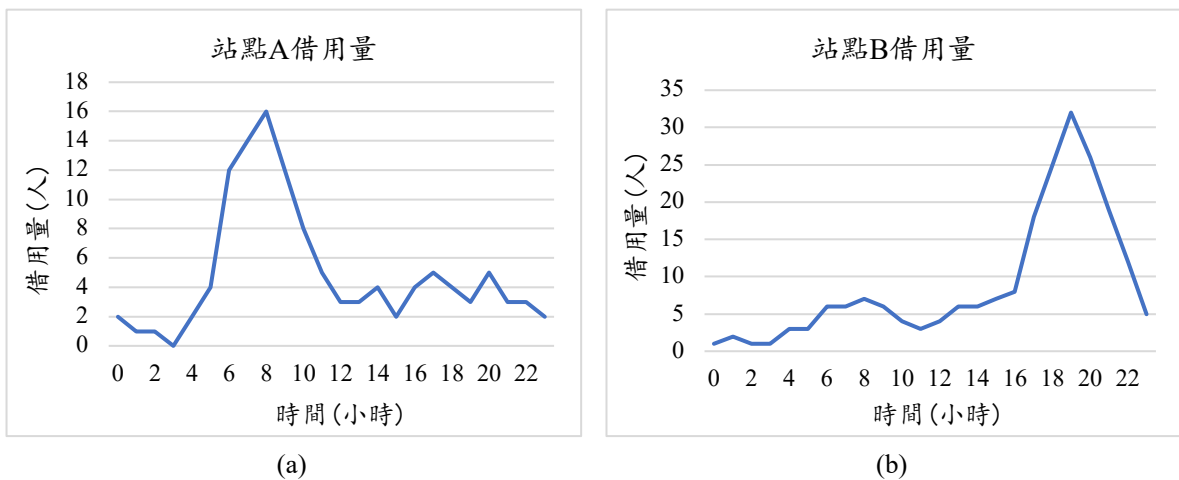


Figure 4.5 站點借用量折線圖

過往考慮時間因素的 DBSCAN 分別有 ST-DBSCAN [11]等，然而以上經過改良 DBSCAN 方法都是針對單一時間點或以時間範圍進行考量，而借用量的格式則是時間序列，因此無法直接應用在本研究上，所以我們需要開發新的 DBSCAN 以針對時間序列與距離作為考量進行分群，並解決 4.1.1 章節中 DBSCAN 在分群上的問題。

我們所提出的方法命名為 Time Series DBSCAN(TS-DBSCAN)，以下是針對 TS-DBSCAN 演算法進行介紹，在 4.1.2.1 中我們會介紹 TS-DBSCAN 基本概念與定義。接著在 4.1.2.2 中我們會介紹 TS-DBSCAN 演算法流程與圖解。在 4.1.2.3 章節中我們會解釋 TS-DBSCAN 的虛擬程式碼。最後 4.1.2.4 我們會探討 TS-DBSCAN 分群上的問題。

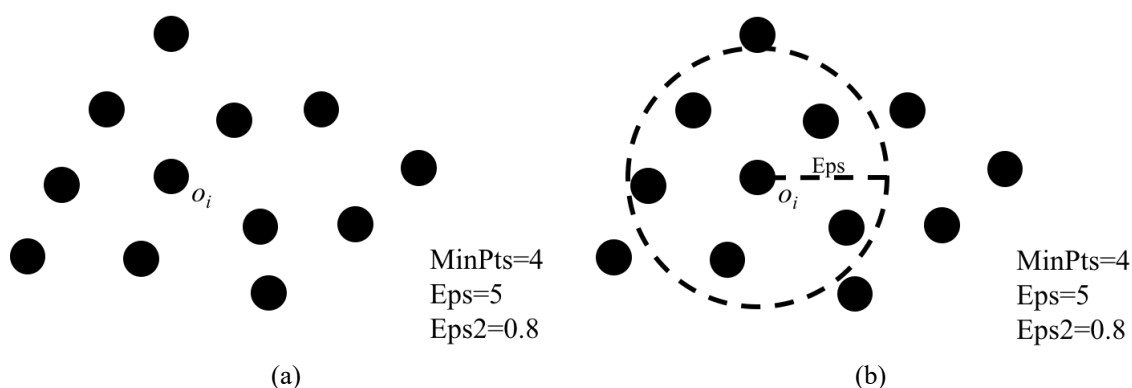
4.1.2.1 TS-DBSCAN 基本概念與定義

TS-DBSCAN 最主要有三個參數：(1) Eps (2) $Eps2$ (3) $MinPts$ 。 Eps 代表半徑，TS-DBSCAN 在分群時會以 Eps 進行站點的擴張，尋找半徑範圍內的站點。 $Eps2$ 是代表時間序列相似度的分數，在本論文中是用作搜尋半徑範圍內借用量相似度高的站點。 $MinPts$ 則與 DBSCAN 相同，當站點半徑範圍內符合時間序列相似度的分數時，計算站點符合的數量是否多於或等於 $MinPts$ ，假設符合則組成群體，否則標式為雜訊點。

在每個 TS-DBSCAN 中每個點必須包含經緯度與時間序列，在本論文中時間序列是代表每個站點一天的借用量，該借用量維度為 1×24 ，24 是指小時，如 Figure 4.5。TS-DBSCAN 會跟據每個站點的距離與時間序列進行分群。

4.1.2.2 TS-DBSCAN 演算法流程

TS-DBSCAN 最主要分為以下五個步驟：



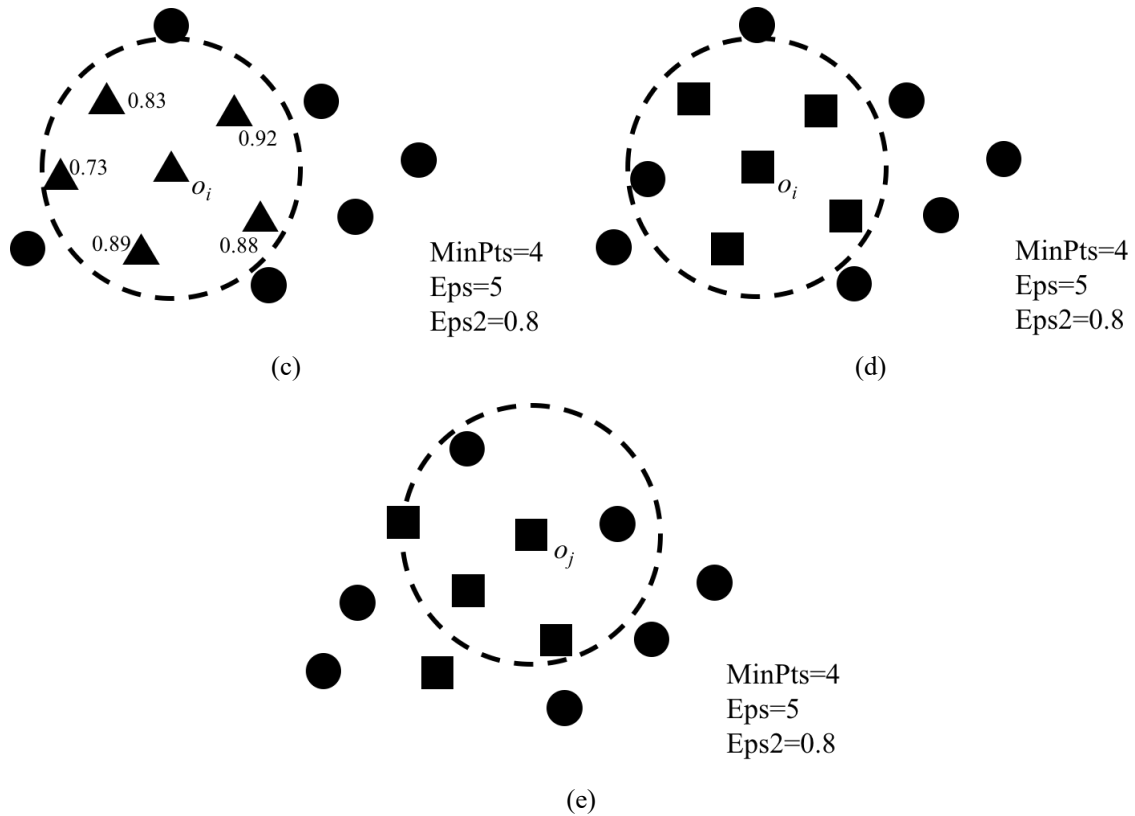


Figure 4.6 TS-DBSCAN 演算法圖解

- Step1.** 從資料集 S 中取出一個站點，稱作為 o_i ，如 Figure4.6(a)。
- Step2.** 以 Eps 作為半徑，擴張 o_i 的範圍，利用 Euclidean Distance 計算 o_i 範圍中所包含的站點數，如 Figure 4.6(b)。
- Step3.** 對 o_i 半徑範圍內的站點利用 Pearson Correlation 計算借用量的相似度，如果借用量相似度大於或等於 $Eps2$ ，則把乎合的站點進行標示，如 Figure 4.6(c)。
- Step4.** 計算 o_i 半徑範圍內的站點，並且借用量相似度乎合 $Eps2$ 的站點數量，如果站點數量多於或等於 $MinPts$ ，則標記 o_i 與 o_i 半徑範圍內及乎合 $Eps2$ 的站點為新的群體，否則標為雜訊點，如 Figure4.6(d)正方形為新的群體。
- Step5.** 從 o_i 半徑範圍內取出一個點，稱作為 o_j ， o_j 重覆 Step2，直到所有站點都擴張完成，如 Figure4.6(e)。

Pearson Correlation

在 4.1.2.1 章節的 Step3 中，利用 Pearson Correlation 計算兩個站點借用量的相似度，該函數需要給定兩個點 p 與 q ，並計算 p 與 q 兩個點借用量的相似性，其公式如下：

$$\text{PearsonCorrelation}(p, q) = \frac{\sum_{i=1}^n (p_i - \bar{p})(q_i - \bar{q})}{\sqrt{\sum_{i=1}^n (p_i - \bar{p})^2} \sqrt{\sum_{i=1}^n (q_i - \bar{q})^2}} \quad 4.1.2$$

公式 4.1.2 所輸入的借用量大小為 1×24 維度，24 代表一天 24 小時每小時站點的借用量，而 n 則是 p 與 q 的維度大小即 24， i 則代表借用量序列中第幾位置，而 $\bar{p}$ 是 p 的平均值，則表示公式 4.1.3：

$$\bar{p} = \frac{1}{n} \sum_{i=1}^n x_i \quad 4.1.3$$

$\bar{q}$ 則是 q 的平均值與 4.1.3 同理。

我們利用 Pearson Correlation 計算兩個站點借用量相似度最大的原因是該函式是跟據時間序列成長趨勢計算兩者相似度，假設兩個站點借用量在第 i 個時間點同時上升或同時下降時，我們稱之為正相關。如果兩個站點借用量在第 i 個時間點成長趨勢形成相反的狀況，即一個站點借用量上升，另一個站點借用量下降，我們稱之為負相關。我們透過以上方式找出站點借用量成長趨勢相似的站點進行分群，找出某些站點在早上時段借用量較高，或某些站點在下午及晚上時段借用量較高，藉此計算所有站點的借用量的相似度進行分群，解決因站點借用量差異過大而使屬性相同的站點過濾等問題(如 Euclidean Distance)。

4.1.2.3 TS-DBSCAN 演算法虛擬程式碼

Algorithm2: Time Series DBSCAN(TS-DBSCAN) 是 TS-DBSCAN 的虛擬程式碼，Algorithm2: Time Series DBSCAN(TS-DBSCAN)與 Algorithm1: DBSCAN 十分相似，但 TS-DBSCAN 多一個參數則是 $Eps2$ ， $Eps2$ 的範圍是由 0 到 1 之間， $Eps2$ 越小相像

度越低， $Eps2$ 是協助過濾借用 量相似度過低的站點，並把借用 量相似度乎合 $Eps2$ 與距離乎合 Eps 的站點歸類為同一個族群，如 line 5 與 line 17 中 `Get_Neighbors` 透過半徑距離 Eps 與借用 量相似度 $Eps2$ 搜尋 p_i 所包含的 *Neighbors*，藉此同時考量距離空間因素與借用 量時間因素進行分群，解決 DBSCAN 分群上的缺失。

Algorithm2 : Time Series DBSCAN(TS-DBSCAN)

Input : **S:** $\{c_1, c_2, \dots, c_k\}$ Set of objects
 Eps: Maximum geographic coordinates distance value
 Eps2: Minimum time similarity value
 MinPts: Minimum number of points within Eps and Eps2

Output : **C :** $\{c_1, c_2, \dots, c_k\}$ Set of clusters

```

1      Initialize return integer cluster_label = 0;
2      Initialize return set  $R = \emptyset$ ;
3      For  $i=1$  to  $|S|$  do
4          If  $p_i$  is not cluster Then
5               $X = \text{Get\_Neighbors}(S, p_i, Eps, Eps2)$ ;
6              If  $|X|$  less than MinPts Then
7                  Mark  $p_i$  is noise;
8              Else
9                   $cluster\_label = cluster\_label + 1$ ;
10                 Mark  $p_i$  be cluster_label;
11                 For  $j = 1$  to  $|X|$  do
12                     Mark All point in  $X$  be cluster_label;
13                 End
14                  $R.\text{push}(X)$ ;
15                 While  $R$  is not  $\emptyset$  do
16                      $r \leftarrow R.\text{dequeue}()$ ;
17                      $Y = \text{Get\_Neighbors}(S, r, Eps, Eps2)$ ;
18                     If  $|Y|$  greater than equal to MinPts then
19                         For  $j = 1$  to  $|Y|$  do
20                             If  $Y_i$  is not noise and  $Y_i$  is not cluster do
21                                 Mark  $Y_i$  be cluster_label;
```



```

22                                     R.push( $Y_i$ );
23                                     End
24                                     End
25                                     End
26                                     End
27                                     End
28                                     End
29                                     End

```

Algorithm3: Get_Neighbors 是 Algorithm2 中 line 5 與 line 17 搜尋乎合 Eps 與 $Eps2$ 兩個參數的函式，函式一開始會先從 D 中取出一個點 p_i ， p_i 與 q 利用 Euclidean Distance 計算兩個站點之間的直線距離，並存放在 d 中。如果 d 少於等於半徑距離 Eps ，則把 p_i 放進 X 中。如果 d 大於半徑距離 Eps ，則在 D 中存取下一點進行檢查 (line 3 – line 8)。然後從 X 中取出其中一個點作為 x ，並與 q 利用 Pearson Correlation 計算兩個站點借用量的相似度存放在 t 中，如果 t 大於等於借用量相似度 $Eps2$ ，則把 x 存放在 N 中，否則捨棄，並檢查下一個站點，直到 X 中所有的站點都檢查完成 (line 10 – line 16)，最後回傳 N 。

Algorithm3 : Get_Neighbors

Input : **S:** $\{c_1, c_2, \dots, c_k\}$ Set of objects
 Eps: Maximum geographic coordinates distance value
 Eps2: Minimum time similarity value

Output : **N:** $\{n_1, n_2, \dots, n_k\}$ Set of clusters

```

1      Initialize return set  $X = \emptyset$ ;
2      Initialize return set  $N = \emptyset$ ;
3      For  $i=1$  to  $|S|$  do
4           $d = \text{EuclideanDistance}(p_i, q)$ ;
5          If  $d$  less than equal to  $Eps$  Then
6               $X.\text{push}(p_i)$ 
7          End
8      End

```

```

9
10     While  $X$  is not  $\emptyset$  do
11          $x \leftarrow X.dequeue()$ ;
12          $t = \text{PearsonCorrelation}(x, q)$ 
13         If  $t$  greater than equal to  $Eps2$  Then
14              $N.push(x)$ 
15         End
16     End

```

4.1.2.4 TS-DBSCAN 分群問題

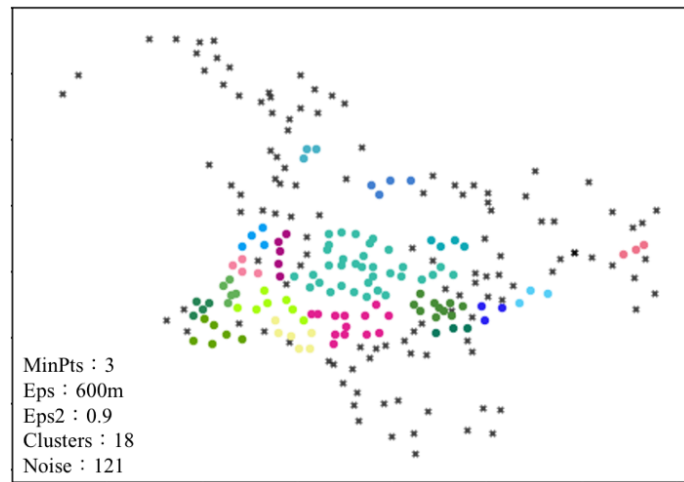


Figure 4.7 TS-DBSCAN 分群結果

Figure 4.7 是 TS-DBSCAN 的分群結果，分群是跟據 MinPts 為 3，半徑 Eps 為 600m，借用量相似度為 0.9，最後得出的結果為群體數為 18 與雜訊點 121。從結果中我們可以發現 TS-DBSCAN 的確有跟據借用量相似度進行分群，然而在分群的過程中大量的站點無法形成群體，造成 121 個雜訊點，導致以上的問題最大原因是 TS-DBSCAN 在分群的過程中，站點需要同時乎合距離 Eps 與時間 $Eps2$ 的參數，才會成為同一個群體，這無疑是提高站點成群的難度，最後造成大量的點產生。

4.1.3 Hierarchical Time Series DBSCAN(HTS-DBSCAN)

在 4.1.2 章節中我們提出利用距離與時間序列進行分群的 TS-DBSCAN，然而分群的結果也是未如理想，TS-DBSCAN 需要站點同時乎合距離 Eps 、時間序列相似度

分數 $Eps2$ 兩個條件才能成為同一個群體，這無疑加大分群的難度，造成 4.1.2.3 章節中描述的問題，而我們針對以上的問題提出兩種解決方法：

第一種方法是把時間參數 $Eps2$ 調低，降低站點成群的難度。如果我們把時間參數 $Eps2$ 調低，的確能夠改善雜訊點過多的問題，然而時間參數過低則與 DBSCAN 無異，失去原本利用以借用量作為時間因素進行分群的意義，因此我們無法以降低 $Eps2$ 參數作為以並的解決方法。

第二種方法是把距離與時間的參數兩個條件分開處理，先以距離分群，再以借用量分群，由於 TS-DBSCAN 需要同時符合距離 Eps 與時間 $Eps2$ 兩個參數才能形成群體，提高成群難度，造成大量雜訊點的產生。假設我們先以距離針對資料集進行分群，先對偏遠的站點進行分群，然後再把密集無法切割群體以時間進行第二次分群，解決 TS-DBSCAN 成群的難度，並且保留以借用量分群的因素，解決 4.1.2.3 章節中所提到的問題。

以上的兩種方法可見，把距離與時間的參數兩個條件分開處理較其他解決方法可行，因此我們在針對上述的解決方法二開發 Hierarchical Time Series DBSCAN (HTS-DBSCAN)，該方法最主要的概是先針對資料集以距離進行分群，分群後會檢查每一個群體的站點數量，如果群體數量過大我們會針對該群體進行第二次的分群，第二次的分群則會以距離及時間作為考量因素。以下 4.1.3.1 章節中我們先介紹 HTS-DBSCAN 的基本概念與定義。第 4.1.3.2 章節中我們會先介紹 HTS-DBSCAN 演算法的流程與圖解，在 4.1.3.3 章節中我們會介紹 HTS-DBSCAN 的虛擬程式碼。

4.1.3.1 HTS-DBSCAN 基本概念與定義

HTS-DBSCAN 需要輸入六個參數，(1) $MinPts$ (2) Eps (3) $MaxClusterSize$ (4) $TSMInPts$ (5) $TSEps$ (6) $TSEps2$ ， $MinPts$ 與 Eps 是用作 HTS-DBSCAN 第一次分群時，單純以距離進行分群，即利用 DBSCAN 進行分群。而 $MaxClusterSize$ 是用作找出 HTS-DBSCAN 第一次分群時群體站點數超過 $MaxClusterSize$ 數的群體。最後

$TSMinPts$ 、 $TSEps$ 及 $TSEps2$ 則是用作 HTS-DBSCAN 第二次分群時，即利用 TS-DBSCAN 進行分群， $TSMinPts$ 、 $TSEps$ 及 $TSEps2$ 分別對應 TS-DBSCAN 的 $MinPts$ 、 Eps 及 $Eps2$ 三個參數。

HTS-DBSCAN 與 TS-DBSCAN 相同，每個站點必須包含經緯度與時間序列的資料，經緯度是用作 HTS-DBSCAN 第一次分群的資料，第二次分群時則會用到站點經緯度與時間序列兩個資訊。HTS-DBSCAN 時間序列與 TS-DBSCAN 相同，都是利用站點的一天的會用量進行分群，該維度為 1×24 。

4.1.3.2 HTS-DBSCAN 演算法流程

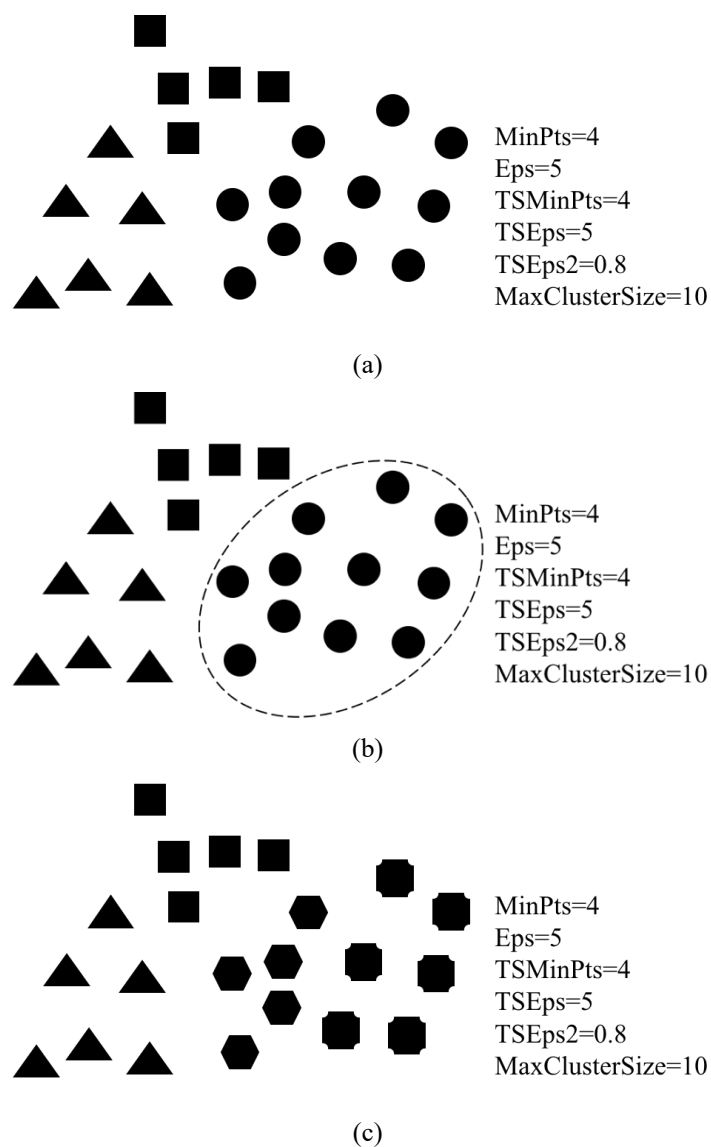


Figure 4.8 HTS-DBSCAN 演算法圖解

- Step1.** 先以 $MinPts$ 及 Eps 兩個參數利用 DBSCAN 針對資料集 S 進行分群，如 Figure 4.8(a) 不同形狀為不同群體，共有 3 個體群。
- Step2.** 在 DBSCAN 分群結果中，找出群體站點數量大於或等於 $MaxClusterSize$ 的群體，如 Figure 4.8(b) 被虛線包含的體群。
- Step3.** 把群體站點數量大於 $MaxClusterSize$ 的群體，以 $TSMinPts$ 、 $TSEps$ 及 $TSEps2$ 三個參數利用 TS-DBSCAN 進行分群，如 Figure 4.8(c) TS-DBSCAN 分群後切成兩個群體。
- Step4.** 當群體站點數量大於 $MaxClusterSize$ 的群體都經過 TS-DBSCAN 進行分群後，HTS-DBSCAN 即分群結束。

4.1.3.3 HTS-DBSCAN 演算法虛擬程式碼

Algorithm4 : Hierarchical Time Series DBSCAN (HTS-DBSCAN) 是 HTS-DBSCAN 的虛擬程式碼，HTS-DBSCAN 共有 5 個參數， Eps 為第一次分群的半徑的距離範圍， TS_Eps 為第二次分群的半徑的距離範圍， TS_Eps2 則是第二次分群的借用量相似度分數， $MinPts$ 為群體所包含的最小點數， $MaxClusterSize$ 為群體最多包含點數。HTS-DBSCAN 演算法一開始會先以距離進行，即先利用 Algorithm1 : DBSCAN 對資料集 S 進行分群，分群後的結果會存放在 N 中 (line 3)。然後取出 N 中每個群體進行檢查，檢查每個群體所包含站點的數量是否大於 $MaxClusterSize$ ，如果數量大於等於 $MaxClusterSize$ ，則利用距離與借用量兩個因素進行第二次分群，即利用 Algorithm2 : TS-DBSCAN 進行第二次分群 (line 4 – line 6)。TS-DBSCAN 分群後的結果存放在 X 中，再把 X 中每個群體加入至 C 中 (line 8 – line 9)。如果 N 中的群體站點數量少於 $MaxClusterSize$ 則把該群體加入 C 中 (line 12)。當所有群體都檢查完成後，即可回傳 C 作為 HTS-DBSCAN 最後的分群結果。

Algorithm4 : Hierarchical Time Series DBSCAN(HTS-DBSCAN)

Input : **S:** $\{c_1, c_2, \dots, c_k\}$ Set of objects
 Eps: The maximum geographic coordinates distance value for DBSCAN
 TS_Eps: The maximum geographic coordinates distance value for TS-DBSCAN
 TS_Eps2: The minimum time similarity value for TS-DBSCAN
 MinPts: Minimum number of points within Eps and Eps2
 MaxClusterSize: Maximum number of point in cluster

Output : **C :** $\{c_1, c_2, \dots, c_k\}$ Set of clusters

```
1      Initialize return set  $N = \emptyset$ ;  
2      Initialize return set  $C = \emptyset$ ;  
3  
4       $N = \text{DBSCAN}(S, Eps, MinPts)$   
5      For  $i=1$  to  $|N|$  do  
6          If  $|n_i|$  greater than equal to MaxClusterSize Then  
7               $X = \text{TS-DBSCAN}(n_i, TS\_Eps, TS\_Eps2, MinPts)$ ;  
8  
9              While  $X$  is not  $\emptyset$  do  
10                  $x \leftarrow X.\text{dequeue}()$ ;  
11                  $C.\text{Push}(x)$   
12             End  
13         Else  
14              $C.\text{Push}(n_i)$   
15         End  
16     End
```

4.2 歷史區域 OD 矩陣建構

在本章節中我們會介紹如何利用分群後的站點與歷史 YouBike 借用量資料建立區域 OD 矩陣。首先，在 4.2.1 中會介紹如何處理 HTS-DBSCAN 中的雜訊點。接著在 4.2.2 中我們會介紹如何定義區域 OD 矩陣的起點與終點。最後在 4.2.3 中會利用歷

史 YouBike 借用量資料建立歷史區域 OD 矩陣。

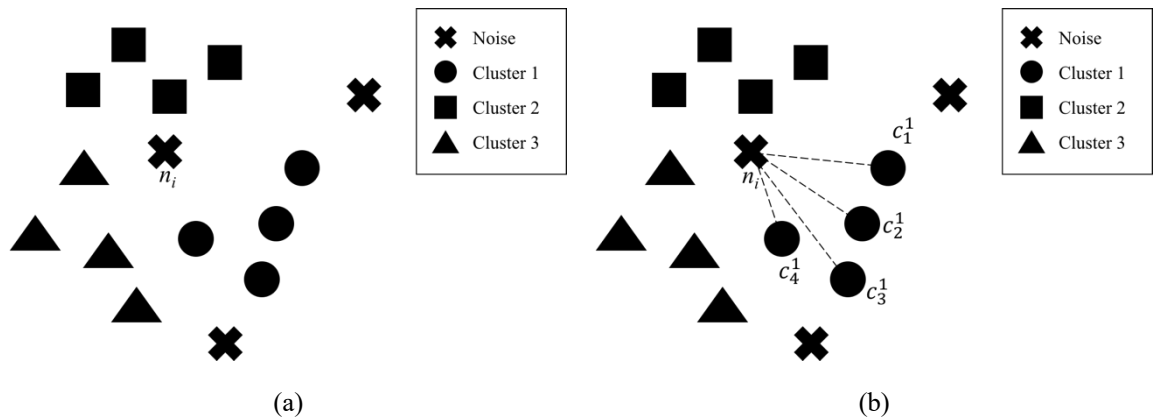
4.2.1 HTS-DBSCAN 雜訊點處理

HTS-DBSCAN 的確大大減小了 TS-DBSCAN 雜訊點的數目，但是考量到每個站點間接表示當地民眾的生活習慣與方式，另外雜訊點可能包含某些熱門或存有大量借用量的站點，如果我們直接把熱門的雜訊點刪除，可能導致該 OD 矩陣的資訊失真，再者可能未能反映該地區的真实人流狀況，因此為了避免忽略每個雜訊點存在的意義，因此我們針對 HTS-DBSCAN 雜訊點的處理會利用 Average linkage agglomerative algorithm，以下我們會介紹 HTS-DBSCAN 雜訊點處理流程與 Average linkage agglomerative algorithm 的解說。

4.2.1.1 HTS-DBSCAN 雜訊點處理流程

在本論文中，Average Linkage Agglomerative Algorithm 是用作計算某個雜訊點與群體的距離，並從中找出與該雜訊點最靠近的群體，並把該雜訊點加入到最靠近的群體中，藉此把所有的雜訊點歸類到不同相對靠近的群體中，以下為 Average Linkage Agglomerative Algorithm 的流程解說。

HTS-DBSCAN 雜訊點處理流程共有以下四個步驟：



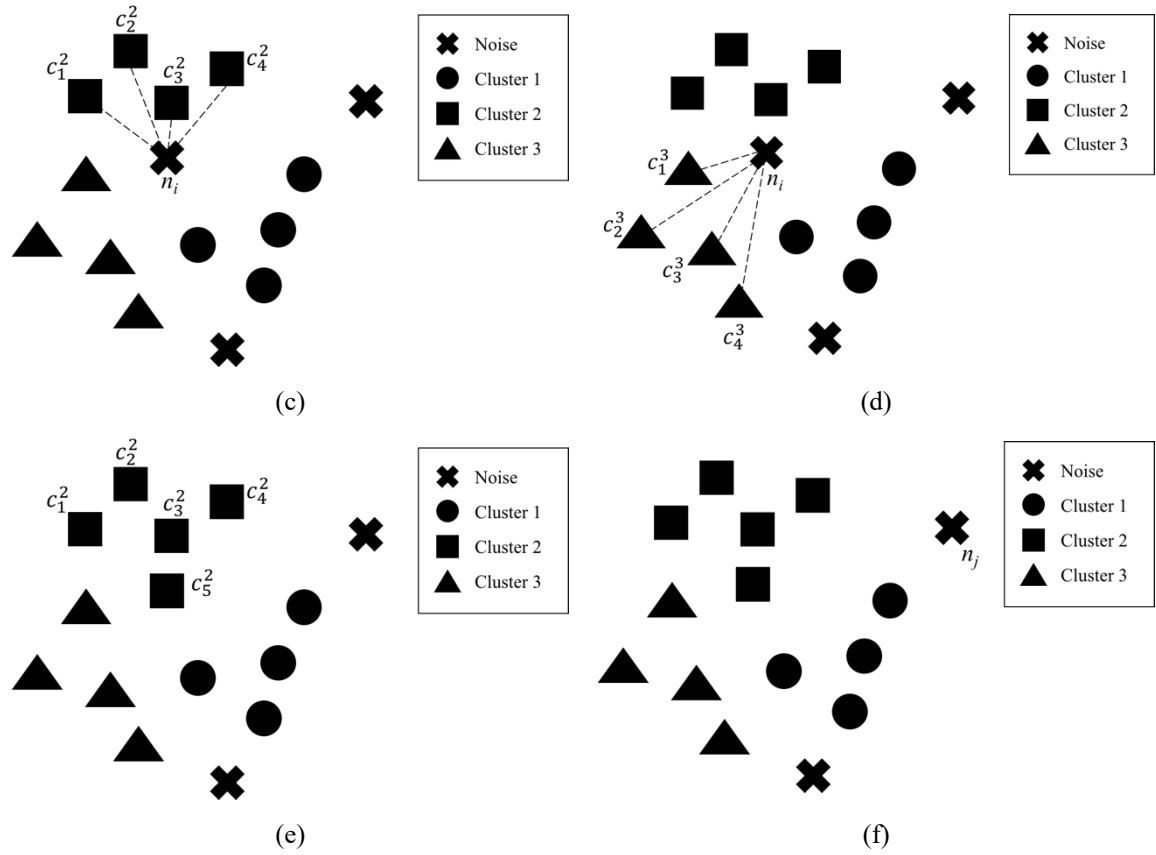


Figure 4.8 HTS-DBSCAN 雜訊點處理流程圖解

- Step1.** 先從所有的雜訊點中取出其中一個點，稱作為 n_i ，如 Figure 4.8(a)。
- Step2.** 利用 Average Linkage Agglomerative Algorithm 計算 n_i 與其他群體的距離，如 Figure 4.8(b)、(c)與(d)。
- Step3.** 把 n_i 歸類到距離最近的群體，如 Figure 4.8(e) n_i 被歸類到 Cluster 2 中。
- Step4.** n_i 被歸類完成後取下一個雜訊點 n_j 重覆 Step2 到 Step3 的步驟，直至所有雜訊點被歸類完成，即可結束，如 Figure 4.8(f)。

Average Linkage Agglomerative Algorithm

Step2 中的 Average Linkage Agglomerative Algorithm [28]是用作計算雜訊點與群體的距離，其公式如下：

$$\text{AverageLinkage}(n, c) = \frac{1}{N} \sum_{i=1}^N D(n, c_i) \quad 4.1.4$$

上面 4.1.4 公式中 n 為雜訊點， c 為一個群體， c_i 是 c 中的一個站點， $D(n, c_i)$ 則是利

用 Euclidean Distance 計算 n 與 c_i 之間的直線距離，然後把所有距離加總後平均，最後得出雜訊點 n 與群體 c 的距離。

4.2.1.2 HTS-DBSCAN 雜訊點處理虛擬程式碼

Algorithm5 : HTS-DBSCAN Noise Processing 一開始是先取出雜訊點集合 N 中其中一個點作為 n_i ，然後 n_i 與每一個群體利用 Average linkage agglomerative algorithm 計算站點與群體之間的距離(line 4)，若 n_i 與當前群體 c_j 計算的距離為最短，則把當前的距離與群體索引 j 存放在 $min_distance$ 與 min_index 中(line 5 – line 8)，直到所有群體計算完成，最後把 n_i 加入到 c_{min_index} 中。接著取下一個雜訊點重覆 line 2 – line 11 的動作，直到所有雜訊點檢查完成。

Algorithm5 : HTS-DBSCAN Noise Processing

Input : $N: \{n_1, n_2, \dots, n_k\}$ Set of noises
 $C: \{c_1, c_2, \dots, c_k\}$ Set of clusters

Output : $C: \{c_1, c_2, \dots, c_k\}$ Set of clusters include noises

```

1      For  $i=1$  to  $|N|$  do
2           $min\_distance = 99999, min\_index = -1$ 
3          For  $j=1$  to  $|C|$  do
4               $d = \text{AverageLinkage}(n_i, c_j)$ 
5              If  $d$  less than  $min\_distance$  Then
6                   $min\_distance = d$ 
7                   $min\_index = j$ 
8              End
9          End
10
11       $c_{min\_index}.Push(n_i)$ 
12      End

```

4.2.2 區域 OD 矩陣起點與終點定義

本章節我們會介紹把分群後及經過雜訊點處理的群體定義 OD 矩陣的起點與終

點，假設 Figure 4.10 是我們在 4.2.1 中雜訊點處理後的結果，當中每個不同顏色表示為不同的群體，而我們會把每個群體進行標示如 Figure 4.11。

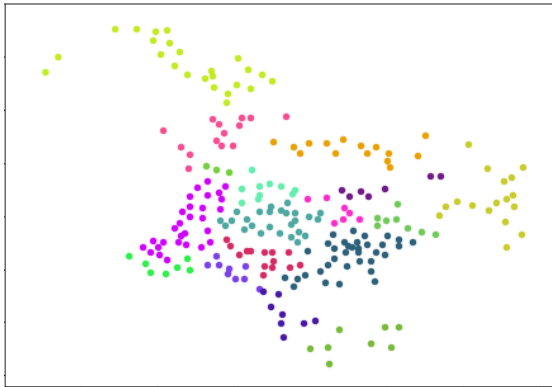


Figure 4.10 經過雜訊點處理後的分群結果

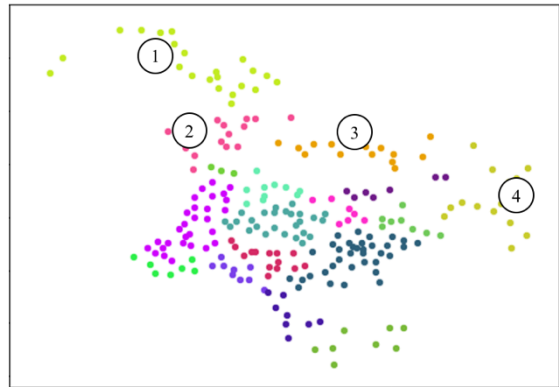


Figure 4.11 群體標示示意圖

每個群體標示完成後，我們會把已標示的每一個群體定義為區域 OD 矩陣的起點與終點，起點與終點的群體排序與數目都是相同的如，Table 4.1。Table 4.1 中 n 是代表群體的總數，因此 Table 4.1 會形成一個 $n \times n$ 的二維矩陣。

Table 4.1 歷史區域 OD 矩陣起點與終點的定義

終點 起點	群體 1	群體 2	群體 3	群體 4	...	群體 n
群體 1						
群體 2						
群體 3						
群體 4						
...						
群體 n						

4.2.3 歷史區域 OD 矩陣建構

當我們把區域 OD 矩陣的起點與終點定義完成後，我們會利用 Table 3.1(b) 台北市 2016 年 YouBike 借用量資料，把借用的站點與歸還的站點對應屬於自己的群體，並以不同的時間單位統計 Table 3.1(b) 每筆資料從起點到終點的人數，藉此建立歷史區域 OD 矩陣，如 Table 4.2 區域歷史 OD 矩陣示意表格。本論文分別以天、小時與三個小時作為不同的時間單位進行區域歷史 OD 矩陣的統計。統計完成的歷史 OD 矩陣則

會用作在 4.3 章節中利用深度學習進行學習及預測的資料集。

Table 4.2 區域歷史 OD 矩陣示意表格

起點 \ 終點	群體 1	群體 2	群體 3	群體 4	...	群體 n
群體 1	465	372	264	213	...	234
群體 2	491	685	547	128	...	594
群體 3	251	423	645	358	...	632
群體 4	193	278	331	475	...	485
...	...	...	...	...	...	...
群體 n	412	372	843	234	...	782

4.3 未來 OD 矩陣預測

在本章節中我們會介紹本論文所提出的深度學習架構以預測未來的 OD 矩陣，Figure 4.12 為本論文所提出的深度學習架構，該架構最主要分為四個部分：(1)歷史因素建模 (History Model)；(2)空間因素建模(Spatial Model)；(3)外在因素建模(Extra Model)；(4)綜合因素建模(Fusion Model)；以下我們分別會在 4.3.1 至 4.3.4 章節中進行詳細介紹與說明。

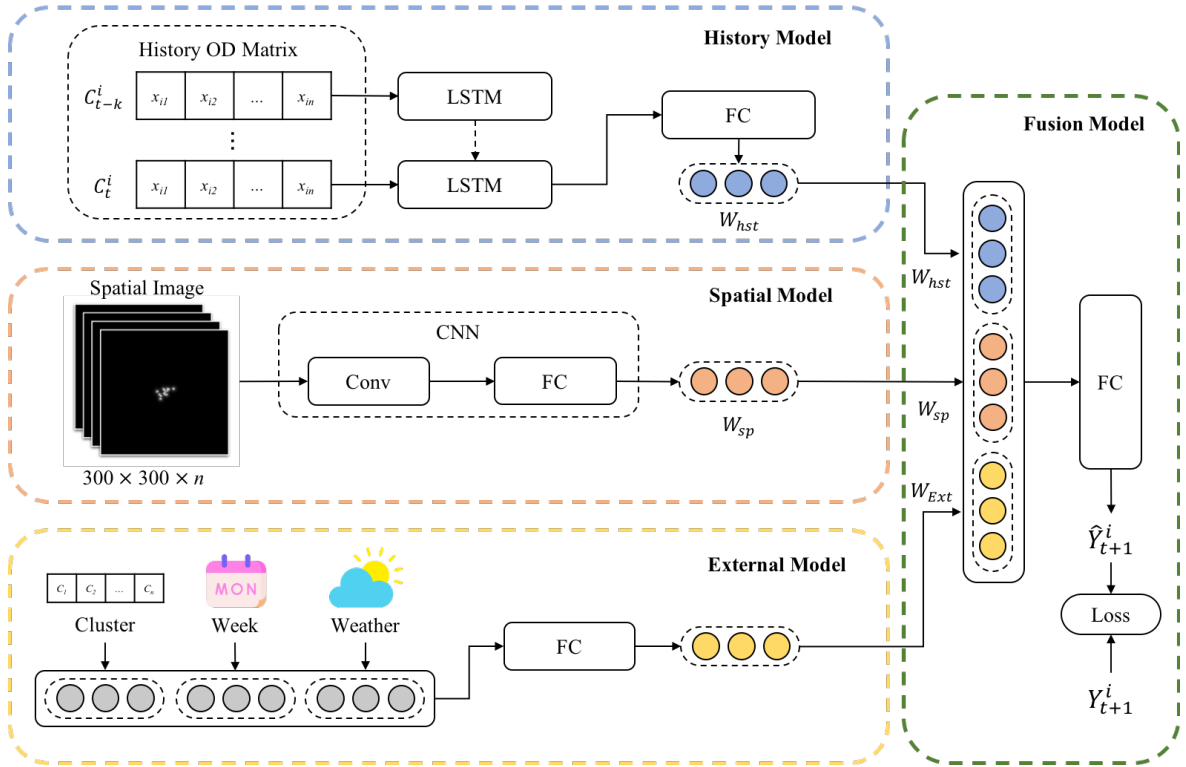


Figure 4.12 OD 矩陣預測深度學習架構

4.3.1 歷史因素建模

歷史因素建模最主要的目的是學習前幾個時間點始點群體到達其他群體的變化，藉此預測下一個時間點的區域 OD 矩陣。在歷史因素建模層中，我們會輸入始點群體前幾個時間點的區域 OD 矩陣至 LSTM 中，LSTM 學習歷史區域 OD 矩陣的變化進行預測，如 Figure 4.13。

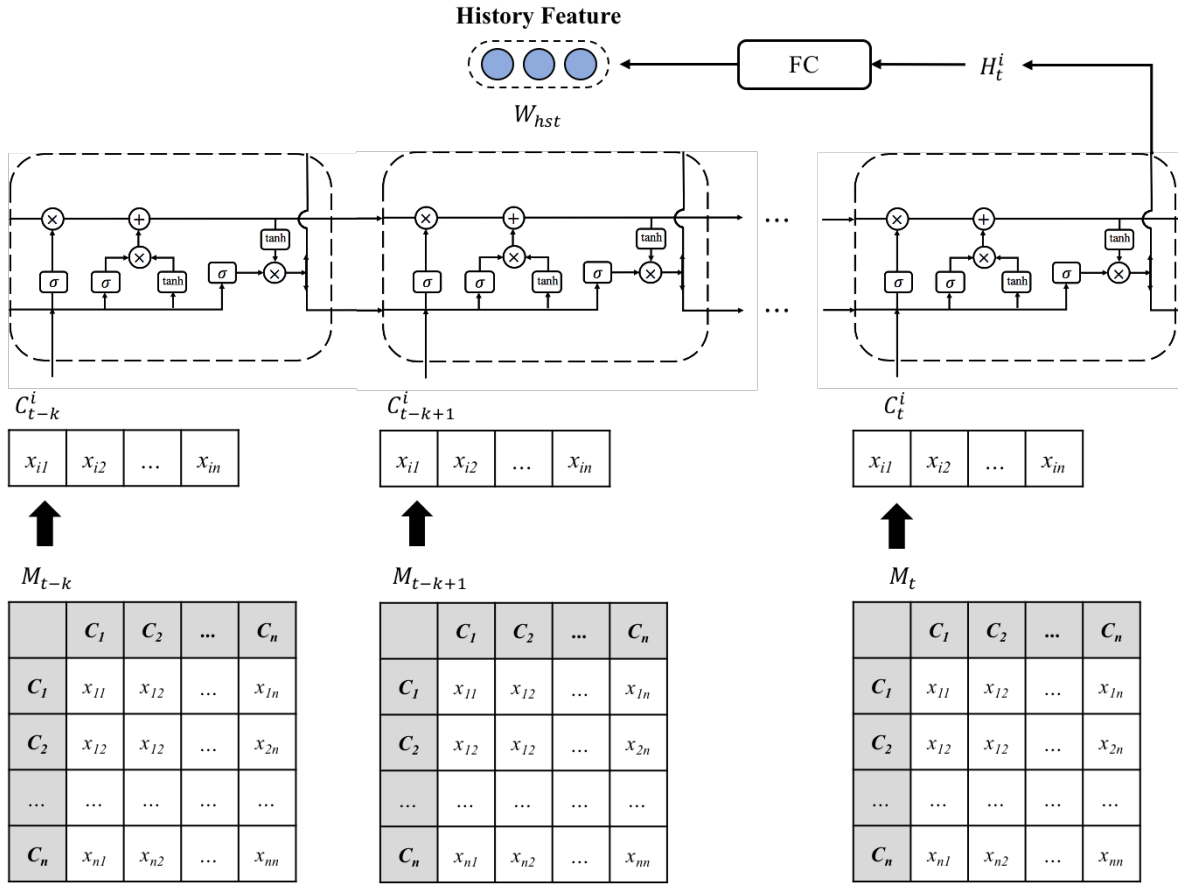


Figure 4.13 LSTM 擷取歷史因素的特徵

我們以 Figure 4.13 為例， M 代表歷史的 OD 矩陣， M 維度是 $n \times n$ ， n 代表群體的數量，我們會先從 t 到 $t-k$ 個時間點的 M 中取出需要預測群體的歷史 OD 流量即 C^i ，分別把 t 到 $t-k$ 個時間點群體 C^i 的歷史 OD 流量輸入至 LSTM 中，LSTM 會輸出歷史 OD 流量的重要資訊如 H_t^i ，最後 H_t^i 輸入利用 FC 中，FC 則代表 Fully Connected Layer，並預測下一個時間點群體 C^i 到達其他群體的 YouBike 流量，即 W_{hst} 。

4.3.2 空間因素建模

空間因素的目的是以圖片的方式表示每個群體的空間因素，並利用 CNN 擷取空間圖片的重要資訊。Figure 4.14 空間因素建模之架構，首先我們先產生始點群體的空間因素圖片，該圖片用作表示始點群體與其他群體的距離及站點的影響範圍，而圖片的深度是取決於群體的總數量，即 Figure 4.14 中的 n 。假設群體的總數量 n 為 17，該始點群體的空間因素圖片大小為 $300 \times 300 \times 17$ 。圖片產生完成後，會輸入至 CNN

擷取始點群體與其他群體的空間資訊，本研究中所應用的 CNN 架構為 VGG16，因為 VGG16 在影像處理與特徵提取有著出色的效果，因此我們直接利用 VGG16 進行空間因素上的建模，藉此學習空間圖片中重要資訊。以下章節我們會主要針對空間因素圖片進行描述與建構流程方法，並探討建構空間因素圖片所面對的問題。

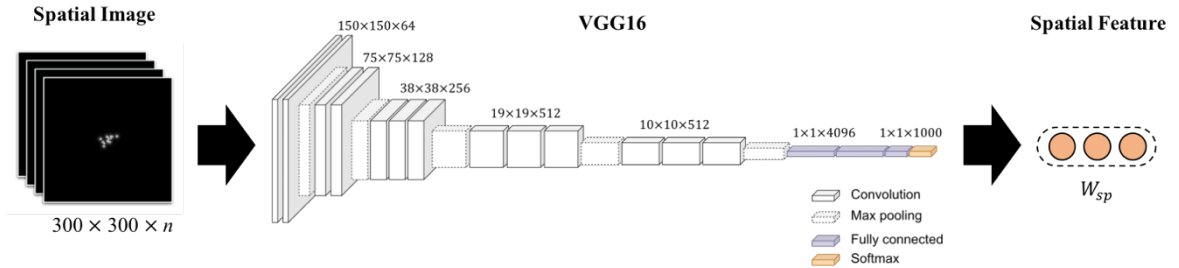


Figure 4.14 CNN 擷取空間因素的特徵

4.3.2.1 空間因素圖片建構

建構空間因素圖片的目的是為了表示群體之間的距離與每個站點的影響範圍，並透過學習從圖片中獲得重要空間的資訊，空間因素圖片的建構是跟據分群的結果而產生的。假設分群的結果有 17 個群體，空間因素圖片則是由 17 張 300×300 圖片組成的，每張圖片是以起點群體中心作為圖片中心，畫出每個群體所包含的站點，群體中每個站點是利用 Decay Function [29] 定義其影響的範圍，並畫在圖中。最後把以 17 個群體所產生的 17 張圖片以深度的方式合成一張圖片，形成空間因素圖片，即 $300 \times 300 \times 17$ 的空間因素圖片。

以 Figure 4.15 為例，群體的總數為 3，分別群體 A、B 與 C。假設以群體 A 作為始點群體，即模型的輸入，並建構群體 A 的空間因素圖片，而圖片則以 300×300 為大小。首先，我們會先以群體 A 的中心點作為圖片的中心，畫出群體 A 所包含的站點，每個站點以 Decay Function 定義其影響範圍，建構完成後會得到 300×300 的圖片，如 Figure 4.15(a)。接著，以同樣的方式把始點群體 A 中心點作為圖片的中心畫出群體 B 所包含的站點及其對應的位置，如 Figure 4.15(b)。最後以上述兩個方式畫出群體 C，如 Figure 4.15(c)。當所有群體都產生完成後，我們會得到 3 張 300×300 的圖片，把 3 張圖片跟據順序以深度的方式進行合成，最後會形成 $300 \times 300 \times 17$ 的圖

片，該圖片則是始點群體 A 的空間圖片，即空間因素模型的輸入。

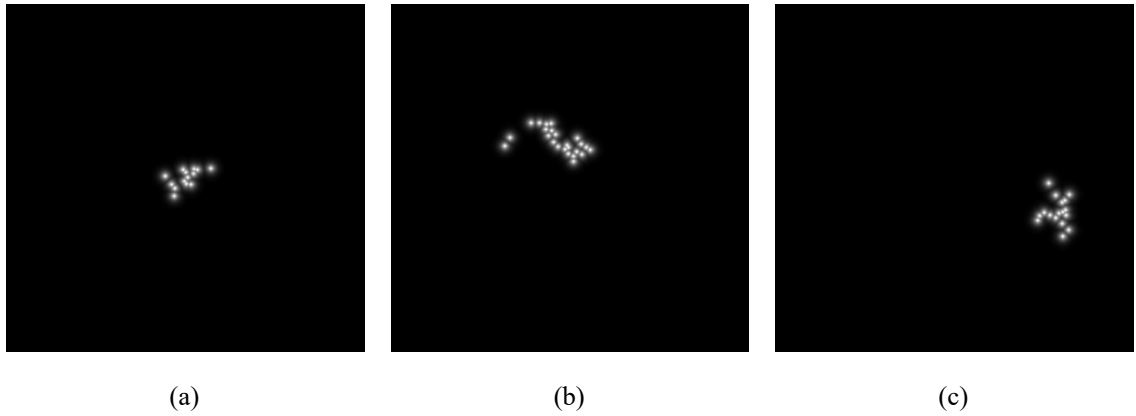


Figure 4.15 群體 A 對於其他群體的空間因素圖

4.3.2.2 Decay Function

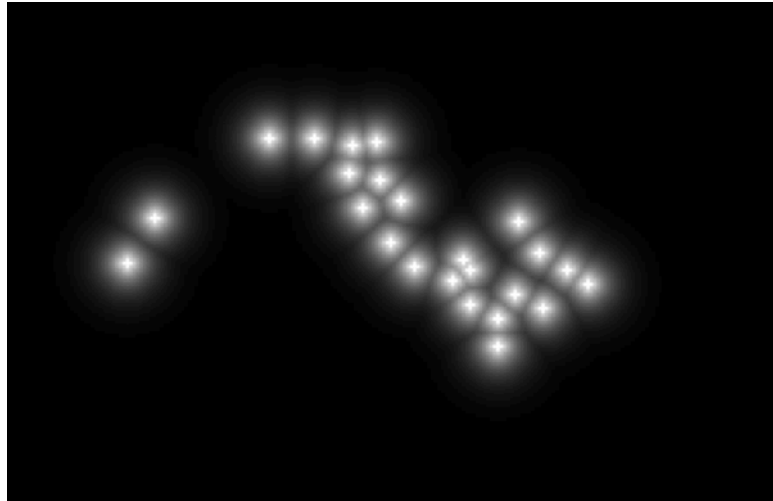


Figure 4.16 群體空間因素圖片

Decay Function 是 4.3.2.1 中用作定義群體中站點影響範圍，其效果如 Figure 4.16 所示，Figure 4.16 中每個白色的點代表一個站點與其影響的範圍，每個站點我們都以 Decay Function 計算影響的程度，當距離站點越遠，白色越淡，其影響程度也相對較低。當距離站點越近，白色越重，影響程度則較高，藉此定義群體中每個站點的影響範圍。

$$f(v, q) = ce^{-\alpha d(v, q)} \quad 4.2.1$$

公式 4.2.1 為 Decay Function 的公式表示， v 與 q 是兩個座標，在 Figure 4.16 中 v 為 YouBike 的站點， q 為圖中的每一個的 pixel， $d(v, q)$ 則是利用 Euclidean distance 計

算的距離， α 為衰減的速度， c 為衰減的最大值。公式其意義是透過兩點的距離，並跟據 α 衰減的速度，計算兩點的衰減程度。如果 v 與 q 的距離越近，則 Decay Function 計算的結果越大，表示 pixel q 對於站點 v 影響程度越大。如果 v 與 q 的距離越遠，Decay Function 計算的結果越小， q 對於站點 v 影響也隨之減少。在本研究中，我們設定每個站點的影響範圍約 500m，當距離站點超過 500m，影響的程度則為 0。

4.3.2.3 空間因素圖片問題探討

在建構空間因素圖片中，有兩個問題值得探討：(1)為什麼利用圖片，不利用距離數字表示空間的因素；(2)為什麼不以群體中心計算 Decay Function。

第一個問題最主要有兩大缺點：首先，假設利用距離數字表示空間因素，距離數字只能單純表示站點或群體之間的距離，但無法表示站點或群體影響的範圍，影響範圍間接代表該站點所服務的對象與市民，反映該地區的生活習慣，對於預測 OD 矩陣十分重要。另外，由於我們先對站點進行分群，如果我們以群體為中心，計算群體之間的距離，容易造成空間特徵失真，因為群體中心與群體外圍的站點距離可能相差甚大。若群體面積越大，群體中心與外圍站點的距離誤差則更大。

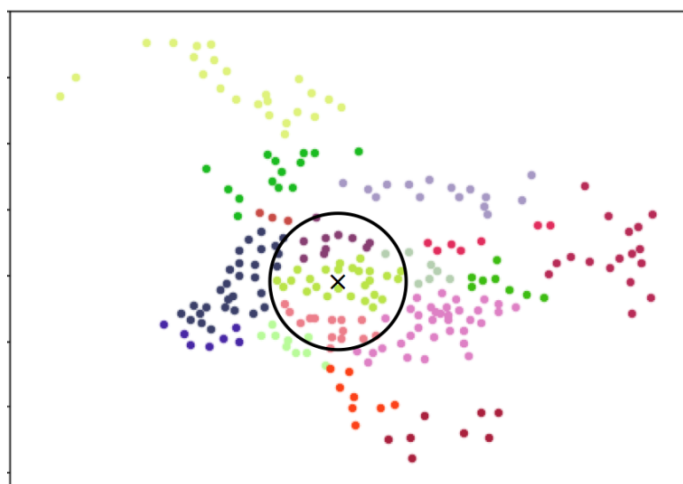


Figure 4.17 以群體中心計 Decay Function

第二個問題是以群體中心計算 Decay Function 所帶來的問題，由於 DBSCAN 分群的結果形狀不規則，如果以群體中心利用圓形畫出群體面積的範圍，可能畫出的範圍與真實群體的面積不乎，容易造成空間資訊失真的問題，如 Figure 4.17，因此我們

針對群體中的站點計算 Decay Function，畫出每個站點的影響範圍，避免空間資訊失真的問題。

4.3.3 外在因素建模

外在因素建模的目的為了模型面對突發狀況發法仍能有效預測區域 OD 矩陣，因為外在因素是間接影響區域 OD 矩陣預測的重要因素，例如早上 8 點到 9 點時，某些企業區的站點 YouBike 使用量會大幅增加。下雨天可能會影響民眾使用 YouBike 的意願。以上都對預測區域 OD 矩陣的影響十分重要，因此我們需要考慮外在因素避免突發狀況發生，影響預測區域 OD 矩陣的效果。

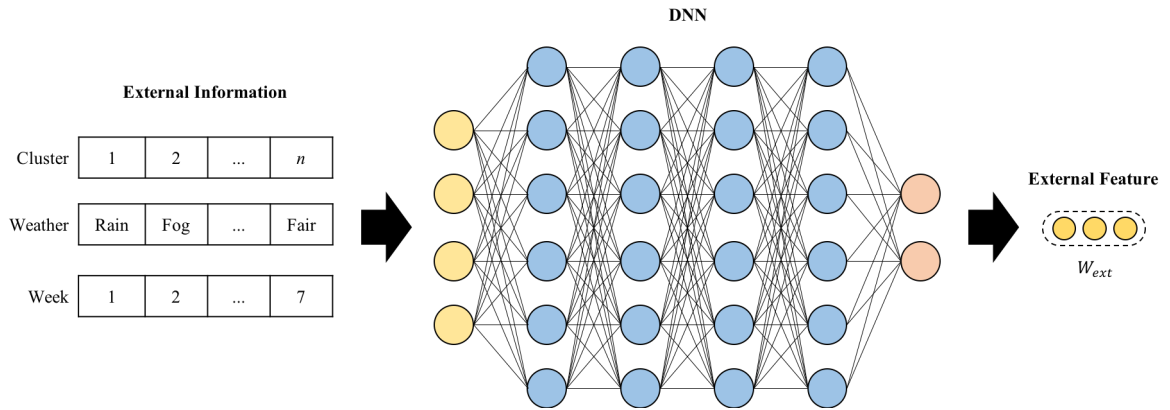


Figure 4.18 DNN 擷取空間因素的特徵

外在因素建模中，我們最主要考慮起點群體、天氣、星期與時間等因素，我們會跟據當時的狀況把以上的各個外在因素以 one-hot 的方式進行表示，並把群體、天氣、星期與時間的 one-hot 表示進行 Concatenate 處理，並輸入至 DNN 進行學習，從中提取起點群體的外在因素重要資訊，如 Figure 4.18。

4.3.3.1 天氣因素

天氣因素是影響區域 OD 矩陣預測的重要一環，下雨天則降低區域 OD 矩陣的流量，晴天則是相反。因此我們需要考量天氣因素從而預測區域 OD 矩陣。本論文所利用的天氣資料為 Table 3.2(a)，該資料包含每個小的天氣狀況，我們會跟據該時段的天氣狀況，取出該筆資料的 Condition，並 Condition 把以 one-hot 的方式表示輸入輸入

DNN 擷取其特徵向量。但預測不同的時間單位，天氣因素的處理方式則有所不同。

首先以小時為例，假設我們預測下午 5 點的區域 OD 矩陣，我們則會取出下午 5 點 Table 3.2(a)中的 Condition，把該 Condition 以 one-hot 表示，形成 1×30 的矩陣，輸入至 DNN 中進行預測。

另外是以天為例，假設目標是預測明天的區域 OD 矩陣，我們首先會形成 1×30 的矩陣，並統計明天每個小時的天氣 Condition，並在對應的 Condition 索引進行累加。每個小時累加完成後，該 1×30 的矩陣加總會等於 24，24 即代表 24 小時的天氣 Condition，然後再把 1×30 的矩陣輸入 DNN 擷取天氣的特徵向量。

4.3.4 綜合因素預測

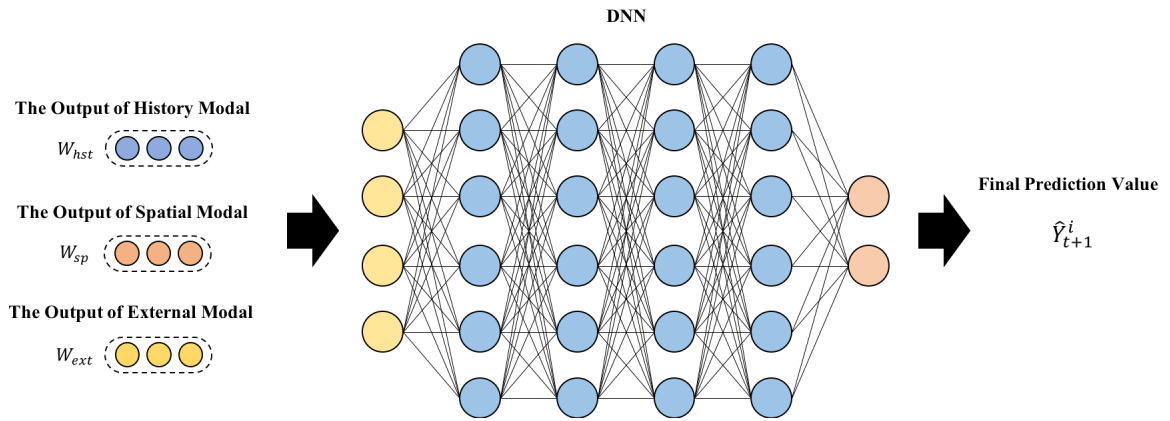


Figure 4.19 綜合因素預測

綜合因素預測是把 4.3.1 中歷史因素建模、4.3.2 中空間因素建模與 4.3.3 中外在因素建模三個輸出 W_{hst} 、 W_{sp} 、 W_{ext} 進行 Concatenate，並輸入至 DNN 預測下一個時間點起點群體到達其他群體的區域 OD 矩陣。最後，我們利用 Mean square error(MSE) 作為 Loss Function，並以真實的答案調整整個模型所有的參數，從而學習預測區域 OD 矩陣，如 Figure 4.19。

V. Experiment

本章節會針對站點分群與區域 OD 矩陣預測，當中我們會與不同的方法進行比較，以及以多角度進行分析，從而證明本論文所提出的方法是有效於建構及預測區域的 OD 矩陣。以下 Table 5.1 為本論文實作平台的環境與設備。

Table 5.1 開發環境與設備

CPU	Intel i5-9600 3.70GHz
Memory	64G
OS	Windows 10 Enterprise
GPU	NVIDIA RTX 2070 8G
Program	Python 3.6.2 64bit

5.1 站點分群

站點分群中我們最主要針對 TS-DBSCAN 與 HTS-DBSCAN 進行分析，證明本論文所提出的方法是有於進行時間序列與距離的分群。首在 5.1.1 中我們會比較 DBSCAN、TS-DBSCAN 與 HTS-DBSCAN 三個分群方法的結果，比較的方式會針對群體與雜訊點的數量進行分析。接著，在 5.1.2 中我們會利用 K-means 與 HTS-DBSCAN 進行比較，檢視兩種不同分群方法所得出的結果差異。

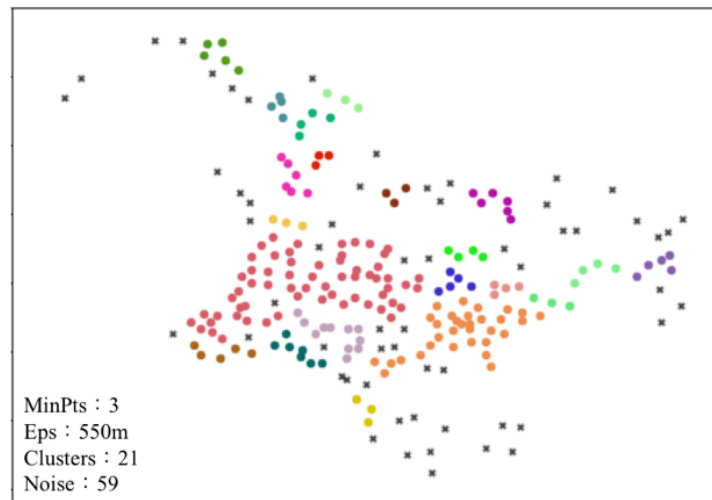
5.1.1 DBSCAN、TS-DBSCAN 與 HTS-DBSCAN 結果比較分析

本章節中我們會比 DBSCAN、TS-DBSCAN 與 HTS-DBSCAN 的結果差異，目的是為了檢視 HTS-DBSCAN 是否有改善 DBSCAN 與 TS-DBSCAN 分群上的問題。

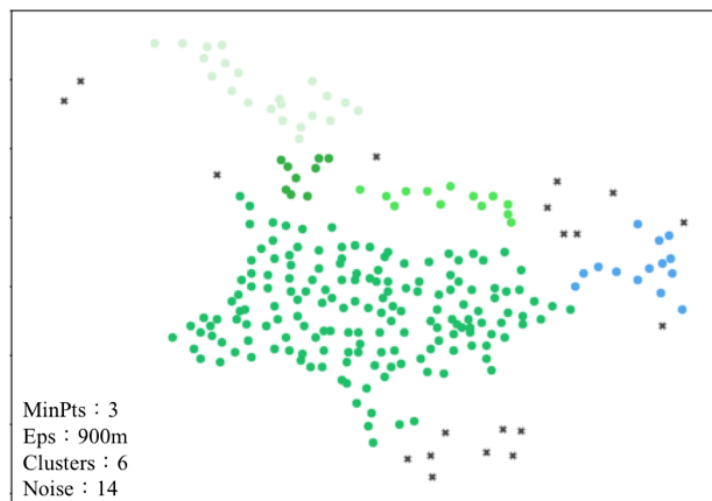
首先，我們先檢視 Figure 5.1 DBSCAN 分群的結果，Figure 5.1(a)與 Figure 5.1 (b) 兩者的差異是前者以 550m 作為半徑進行分群，後者則是以 900m 進行分群，從圖中我們發現以 550m 分群由於距離太小，導致大量的雜訊點。而以 900m 進行分群則因距離過大，導致群體無法切割，分群的結果只有六個群體。

接著，我們檢視 Figure 5.2 TS-DBSCAN 的結果，TS-DBSCAN 是以 540m 作為半徑的距離，時間序列相似度為 0.7。我們可以發現最大的問題是雜訊點過多，導致此

問題的原因是分群時需要同時考慮距離與時間序列相似度兩個因素，且兩個因素需要同時乎合才能成為群體，這讓大量的站點成為雜訊點，令分群的結果未如理想。



(a)



(b)

Figure 5.1 DBSCAN 的分群結果

最後，檢視 Figure 5.3 HTS-DBSCAN 的結果，HTS-DBSCAN 是在第一次分群時是以半徑 900m 進行分群，第二次分群則是以 540m 與時間序列相似度 0.7 進行分群。從 Figure 5.3 分群的結果我們可以清楚發現，HTS-DBSCAN 分群的雜訊點相對前面兩個群體而言是較少的。再者，分群時並沒有 DBSCAN 群體過大無法切割的問題，因此 HTS-DBSCAN 相比 DBSCAN 與 TS-DBSCAN 而言，的確解決了後者兩個方法的問題，並對於資料有著良好的分群效果。

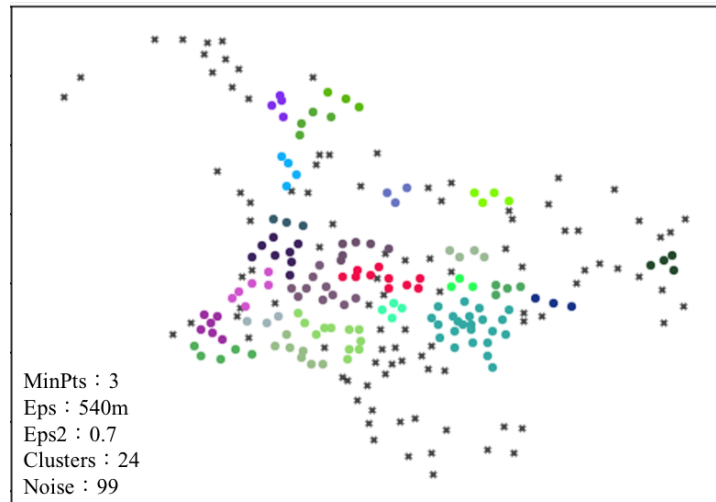


Figure 5.2 TS-DBSCAN 的分群結果

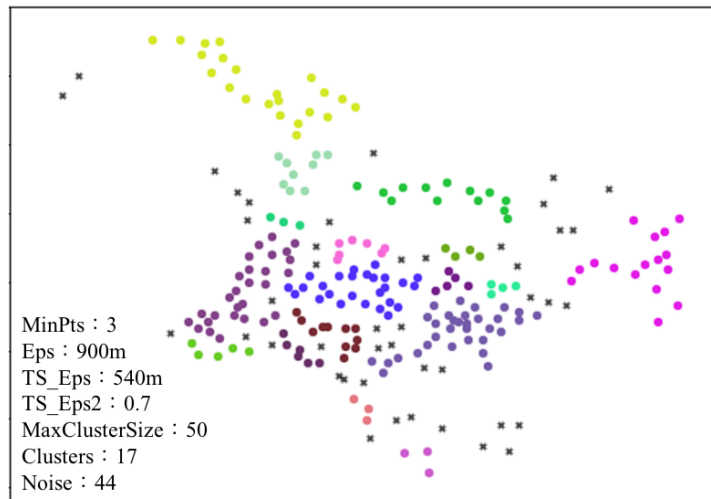


Figure 5.3 HTS-DBSCAN 的分群結果

5.1.2 HTS-DBSCAN 與 K-means 結果比較分析

在 HTS-DBSCAN 與 K-means 結果比較分析中，我們最主要針對群體的數量及兩個分群結果站點 YouBike 借用量相似度進行分析。首先，HTS-DBSCAN 分群的參數與 Figure 5.3 相同，而 K-means 的群體數量參數剛跟據 HTS-DBSCAN 分群的數量而進行設定，分群的結果如 Figure 5.4。

在分析前我們要先了解 K-means 與 HTS-DBSCAN 兩個分群的不同之處，K-means 的概念主要是針對資料與群體中心的距離進行分群，直到群體中心慢慢收斂才算完成。而 HTS-DBSCAN 是先利用距離進行分群，然後面對資料數量過大的群體再透過時間

序列(YouBike 借用量)與距離進行分群，因此就方法而言，HTS-DBSCAN 在針對站點借用量相似度分群上，比較能夠把相似屬性的站點分群為同一族群，K-means 則相對較約。以下我們會先針對 Table5.2 與 Table5.3 進行介紹及詳細分析。

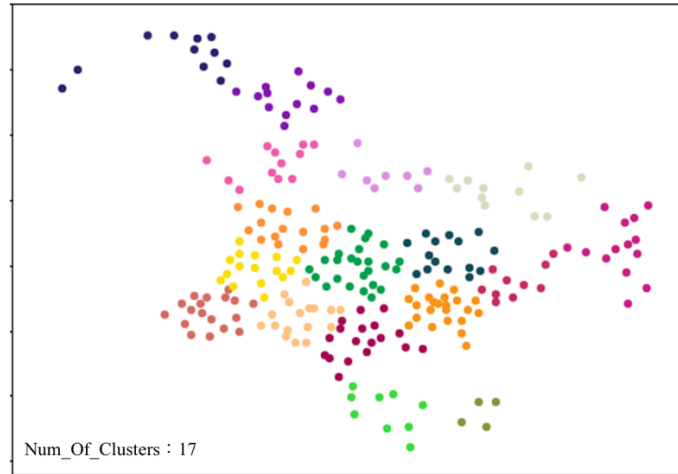


Figure 5.4 K-means 分群結果

Table 5.2 是 HTS-DBSCAN 與 K-means 分群結果中各每個群體各自的數據，數據所包含的是每個群體所包含的資料數目及其資料之間的相似度，以本論文而言，群體中的資料代表 YouBike 站點，而資料之間的相似度即 YouBike 站點之間的借用量相似度。值得注意的是，HTS-DBSCAN 方法是方過兩次分群所得(第一次利用 DBSCAN、第二次利用 TS-DBSCAN)，而 Table 5.2 中 HTS-DBSCAN 的群體 1 至 5 是經過第一次分群時所得的(DBSCAN)，群體 6 至 17 是經過第二次分群時所得的(TS-DBSCAN)，我們後續會以 HTS-DBSCAN 完整的分群與第二次分群(TS-DBSCAN)兩個項目與 K-means 進行比對。另外 Table 5.3 是 HTS-DBSCAN 與 K-means 分群結果綜合數據，我們比對兩個方法的每項數據，以及分析當中 HTS-DBSCAN 分群時對該數據所帶來的影響，以下我們會先分析 HTS-DBSCAN 與 K-means 的群體的大小，接著在針對群體中資料的相似度進行分析。

首先，我們先以群體的大小進行分析，從 Table 5.3 中，HTS-DBSCAN 最大與最小群體的資料數量差距較 K-means 大。其原因在於 HTS-DBSCAN 是以擴展的方式進行分群，被擴展的資料會再以半徑進行自身的擴展，因此群體在分群時群體大小及範

圍會越來越大，而且是以不規則的形狀進行發展。相反，K-means 是找出該資料最靠近的群心而組成群體的，經過多次收斂，群體所擴展的範圍則愈來愈慢或愈來愈小，因此 HTS-DBSCAN 最大與最小群體數量相對 K-means 較大。另外，在群體的平均數量中，HTS-DBSCAN 數量較少的原因是分群時會把不乎合參數的資料標示為雜訊點 (44 個)，因此 HTS-DBSCAN 平均數量較 K-means 少。

接著，我們針對站點的借用量相似度進行分析，而我們會分成兩個部份，第一個部分是分析 HTS-DBSCAN 與 K-means 分群結果中群體站點的相似度。第二部分是針對 HTS-DBSCAN 中第二次分群的結果進行分析，HTS-DBSCAN 第二次分群則是考慮時間序列因素(YouBike 借用量)，即利用 TS-DBSCAN 進行分群，並與 K-means 比對群體中站點的相似度。

Table 5.2 HTS-DBSCAN 與 K-means 各群體數據

HTS-DBSCAN			K-means		
Cluster No.	Cluster Size	Similarity	Cluster No.	Cluster Size	Similarity
1	9	0.68	1	15	0.72
2	22	0.75	2	18	0.68
3	16	0.64	3	12	0.66
4	13	0.67	4	4	0.76
5	3	0.74	5	13	0.77
6	32	0.77	6	14	0.63
7	4	0.86	7	25	0.76
8	7	0.82	8	9	0.65
9	5	0.71	9	17	0.86
10	28	0.69	10	11	0.62
11	4	0.89	11	17	0.83
12	26	0.77	12	23	0.79
13	11	0.88	13	16	0.73
14	6	0.87	14	10	0.78
15	3	0.81	15	11	0.65
16	5	0.86	16	18	0.83
17	5	0.87	17	8	0.64

首先第一個部份，我們可以從 Table 5.3 中了解 HTS-DBSCAN 不論是 Maximum Similarity、Minimum Similarity 及 Average Similarity 都比 K-means 高。而 HTS-DBSCAN 的 Minimum Similarity 為 0.64 是 Table 5.2 中群體 3 的站點相似度，而群體 3 是 HTS-DBSCAN 第一次分群時單純利用距離進行分群的結果(即利用 DBSCAN)，在沒有考慮借用量的狀況下，所以相似度才較其他群體低。但以整體而言，HTS-DBSCAN 群體資料的相似度仍較 K-means 高。

第二個部份是以 HTS-DBSCAN 第二次分群的結果(即利用 TS-DBSCAN)與 K-means 進行對比。在 Table 5.3 中 HTS-DBSCAN 的 Maximum Similarity of TS-DBSCAN、Minimum Similarity of TS-DBSCAN 及 Average Similarity of TS-DBSCAN 都較 K-means 的 Maximum Similarity、Minimum Similarity 及 Average Similarity 優勝，而且 HTS-DBSCAN 第二次分群的整體相似度亦較 HTS-DBSCAN 完整分群的 Maximum Similarity、Minimum Similarity 及 Average Similarity 高。HTS-DBSCAN 的 Minimum Similarity of TS-DBSCAN 只有 0.69 (群體 10)的相似度相對較低的原因是該群體所包含的資料較多，較容易影響群體計算的相似度。然而整體相似度仍高於其他方法，由此可見，本論文所提出的 HTS-DBSCAN 是有效於考慮時間序列與距離進行分群。

Table 5.3 HTS-DBSCAN 與 K-means 分群結果綜合數據

	HTS-DBSCAN	K-means
Maximum Cluster Size	32	25
Minimum Cluster Size	3	4
Average Cluster Size	11.59	14.18
Number of Noise	44	
Maximum Similarity	0.89	0.86
Minimum Similarity	0.64	0.62
Average Similarity	0.78	0.73
Maximum Similarity of TS-DBSCAN	0.89	
Minimum Similarity of TS-DBSCAN	0.69	
Average Similarity of TS-DBSCAN	0.82	

5.2 區域 OD 矩陣預測

在本章節中，我們會針對未來 OD 矩陣預測進行實驗，實驗當中我們會以多角度的方式進行比較或測試，以表示本論文所提出的深度學習模型是有效於針對 OD 矩陣進行預測。首先，在 5.2.1 中會針對本論文所提出的方法與其他深度學習模型進行比較。接著，在 5.2.2 中我們會以不同的時間單位進行預測，目的是希望表示本論文能針對不同的時間單位仍能有效進行預測。另外，在 5.2.3 中我們針對單一群體及所有群體進行學習，通過測試了解何種方式更能有效進行預測。在 5.2.4 中，我們會針對打同的因素進行預測，比較何種因素對於預測最有幫助。在 5.2.5 中我們分析當以不同的歷史資料時長輸入至模型中，模型的效果是否有不同的變化。在 5.2.6 則是針對預測的時長進行改變，藉著以上各種的實驗表示本論文所提出的方法有效於預測 OD 矩陣預測。

本章節我們最主要會以三種時間單位進行上述的五種實驗，分別是(1)小時、(2)時段及(3)天。假設我們以 t 作為當前的時間點，並以前三個時間點 t 、 $t-1$ 、 $t-2$ 作為模型的輸入，預測下一個時間點 $t+1$ 為例。第一種時間單位，小時則是輸入前三個小時作為輸入，並預測下一個小時，如給定早上 4-6 點 OD 矩陣，並預測早上 7 點的 OD 矩陣。第二種時間單位，時段與小時同理給定前三個時段作為輸入，並預測下一個時段。第三種，天則是給定前天、昨天與今天的 OD 矩陣，預測明天的 OD 矩陣。

值得注意的是，我們上述所提到的時間單位時段是以三個小時作為一個時段，因此一天共有八個時段，一天的時段分佈如 Table 5.4，我們八個時段的分佈是跟據台灣人民的生活步調與習慣進行切割。首先在凌晨 02:00 到 04:00 時，大多數人正在睡覺休息，到 05:00 至 07:00 正是普通市民起床準備上班的時間，而 08:00 至 10:00 則是人們早上上班的時段，在 11:00 至 03:00 為民眾午餐的時間，到了下午的 14:00 至 16:00 人們都開始下午的上班工作，17:00 到 19:00 則是開始下班與晚餐的時段，20:00 至 22:00 為民眾晚餐與準備休息的時段，最後 23:00 至 01:00 則是晚上睡覺的時間。

Table 5.4 一天八個時段的时间分佈與描述

時段	小時	描述
1	02:00 ~ 04:00	凌晨睡覺
2	05:00 ~ 07:00	早上起床
3	08:00 ~ 10:00	早上上班
4	11:00 ~ 13:00	中午吃飯
5	14:00 ~ 16:00	下午上班
6	17:00 ~ 19:00	下班及晚餐
7	20:00 ~ 22:00	晚餐及休息
8	23:00 ~ 01:00	晚上睡覺

我們把一天切割成八個時段的目的是考慮到以小時為單位，並同時訓練所有群體的資料時所耗費的時間與資源，因此把一天的所有小時進行切割，跟據民眾的生活習慣，最後得出 Table 5.4 的八個時段，藉此以八個時段為時間單位同時訓練所有群體的資料，減少小時為單位帶來的資源耗費的問題。

另外，我們在實驗中主要利用 Rooted Mean Square Error(RMSE)與 Mean Absolute Error(MAE)評估方法的誤差，RMSE 與 MAE 是大多學者用作評測模型效果的方法，RMSE 與 MAE 公式如下：

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad 5.1$$

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad 5.2$$

其中 y 為預測的結果， $\hat{y}$ 為真實的答案。

5.2.1 其他模型比較

為了表示本論文所提出的深度學習方法有效於預測深度學習之模型，因此本論文與其他深度學習模型進行比較，以下 Table 5.5 為本論文所提出模型與其他方法之比較，我們利用八個時段作為時間單位，同時訓練所有的群體，並以前三個時間點作為輸入，預測下一個時間點為目標。在 Table 5.5 中 LSTM_CNN、LSTM_DNN 與

LSTM_CNN_DNN 為本論文所提出的方法，LSTM_CNN 為加入歷史因素與空間因素的模型，而 LSTM_DNN 為加入歷史因素與外在因素，LSTM_CNN_DNN 則本論文提出最完整的深度學習模型，即加入歷史因素、空間因素及外在因素。當中我們與傳統的深度學習 DNN 與 LSTM 進行比較，從 Table 5.5 可以清楚得知 LSTM_CNN_DNN 在 RMSE 與 MAE 方面都是較佳的，其次則是加入歷史因素與外在因素進行預測的 LSTM_DNN。以 MAE 角度 LSTM_CNN_DNN 與 LSTM_DNN 分別是 3.45 與 3.84，即代表平均每筆資料的誤差是 3 至 4 個人，明顯比其他方法而言都較為優秀。

Table 5.5 以八個時段作為時間單位進行比較

Dataset	Metric	DNN	LSTM	LSTM CNN	LSTM DNN	LSTM CNN DNN
2016 年 台北市	RMSE	11.63	17.80	18.61	8.22	7.70
YouBike	MAE	5.40	7.87	8.09	3.84	3.45

5.2.2 不同時間單位預測

為了表示本論文所提出的方法能針對不同的時間單位進行預測，因此我們以小時、時段與天進行預測，藉此表示面對不同的時間單位，本論文仍能有效進行預測。

5.2.2.1 小時

以 Table 5.6 為例，我們以小時為時間單位，針對單一群體進行學習，並給定前三個時間點，預測下一個時間點的 OD 矩陣。在小時的實驗項目中，我們以群體 6 作為單一群體進行學習，即模型只針對始點為群體 6 的資料進行學習，預測群體 6 到其他群體的人流量。以單一群體進行學習的原因是避免以小時學習所有群體所耗費的資源與時間。而我們針對群體 6 進行學習的原因有兩個，第一個原因是群體 6 為所有群體中站點數最多，本論文的站點數共有 241 個，而群體 6 則佔了 35 個，其次是台北市中正區的群體 10 (29 站點)與的台北市大安區群體 12 (26 站點)。第二個原因是群體 6 為台北熱門地區信義區，當中包含信義廣場、台北 101、台北市政府、松山文創園區

等熱門地標。因此我們選擇群體 6 作為學習及預測的群體。

Table 5.6 以小時為時間單位預測

Dataset	Data Size	Epoch	Batch	Metric	LSTM_CNN_DNN
2016 年 台北市 YouBike	8781	100	30	RMSE	9.87
				MAE	4.50

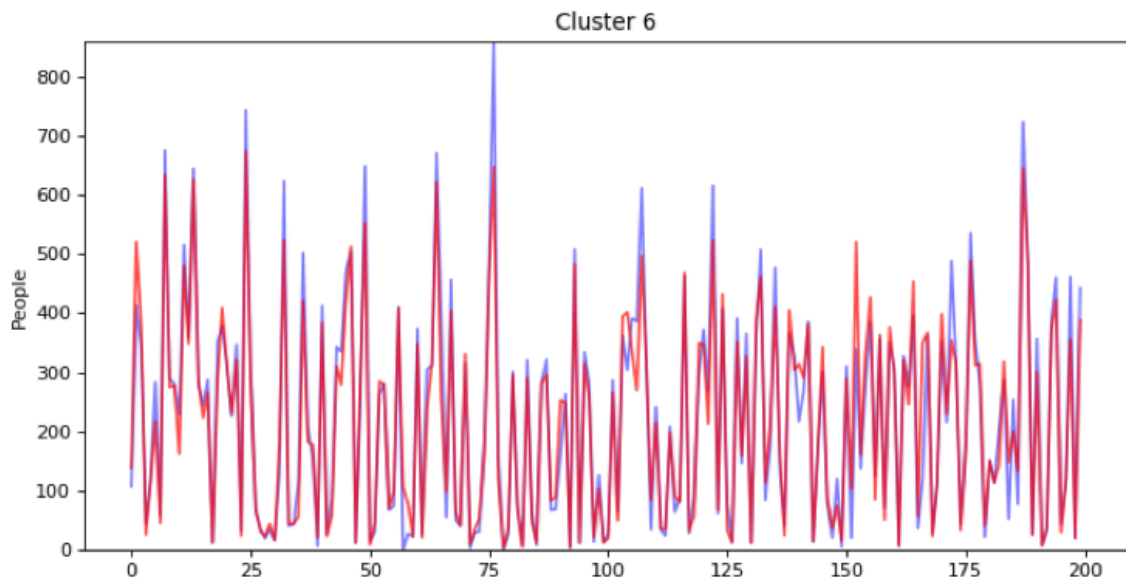


Figure 5.5 起點與終點為群體 6 的真實結果與預測結果比對折線圖

Table 5.6 中，我們以訓練的 Epoch 設定為 100，Batch 設定為 30，其資料大小為 8781 筆，而 RMSE 與 MAE 分別是 9.87 與 4.50，以 MAE 而言即每筆資料平均誤差 4 至 5 個人。我們同時也擷取群體 6 預測不同群體隨機 200 筆資料的折線圖進行比對。我們分別比對群體 6、群體 10 群體 12 與群體 2。首先在 Figure 5.5 中，我們取出群體 6 的原因是因為是針對自己的群體進行預測，也即是從群體 6 中使用 YouBike 到達群體 6 的人數。圖中藍色是代表真實結果，紅色則是預測的結果，從圖中我們也可以清楚了解預測的結果與真實的資料十分相近。

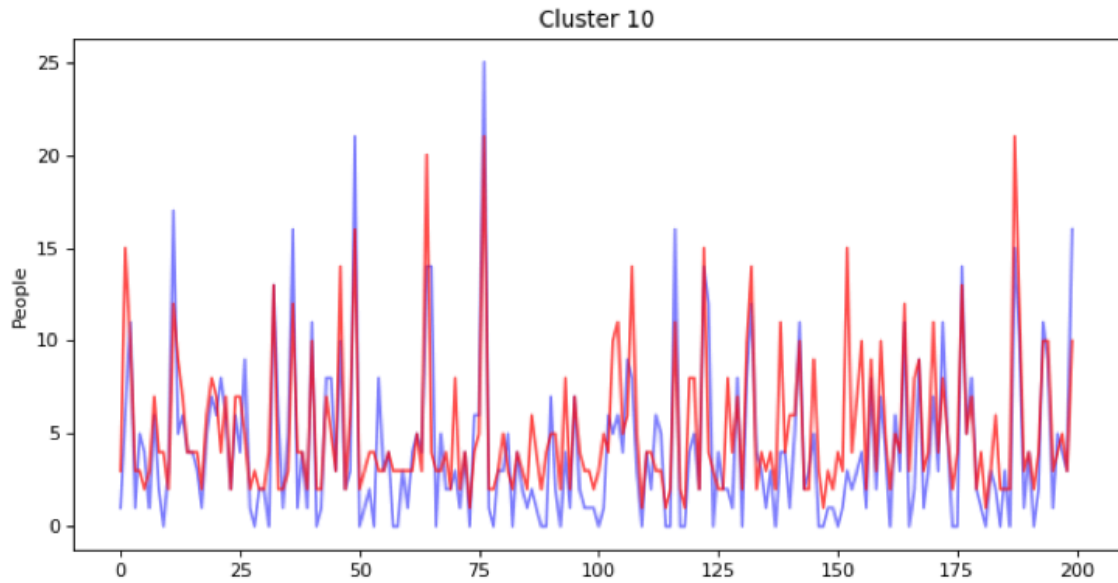


Figure 5.6 群體 6 始點到達群體 10 的真實結果與預測結果比對折線圖

Figure 5.6 是預測群體 6 出發到達群體 10 的真實與預測結果比對圖，利用群體 10 比對的原因是為所有群體中站點第二多的群體，群體 10 總數共有 29 個，而群體 10 大概位於台北市的中正區，從圖中我們可以了解藍色與紅色的線並不是十分的相近，但我們可以透過 Y 軸了解，當中每筆資料的誤差約 2 至 4 人，所以群體 6 對於到達群體 10 也是頗為準確。

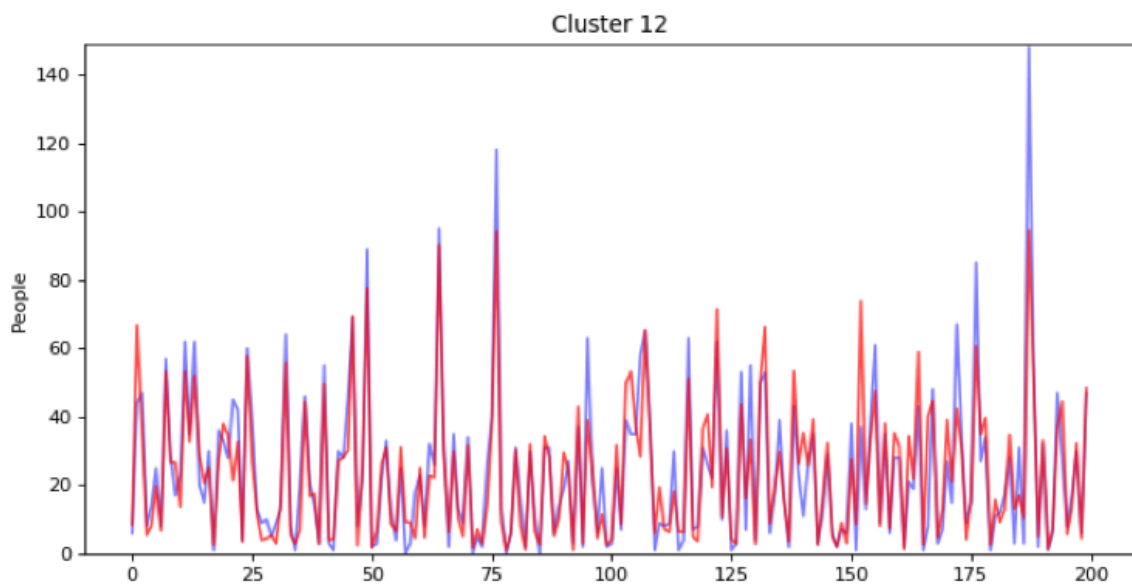


Figure 5.7 群體 6 始點到達群體 12 的真實結果與預測結果比對折線圖

Figure 5.7 為從群體 6 出發到達群體 12 的群體，而群體 12 為群體 6 距離最靠近

的群體，其站點數為 26 個，並位於大安區。從圖中可見預測的紅線與真實結果的藍線也是十分相近。Figure 5.8 則是群體 6 始點到達群體 2 的對比圖，群體 2 則與群體 12 相後，群體 2 是距離群體 6 最遠的，因此我們隨機取出的 200 筆資料每筆人流量都是 0 到 3 個人，而當中的誤差也是落在 0 至 3 個人。

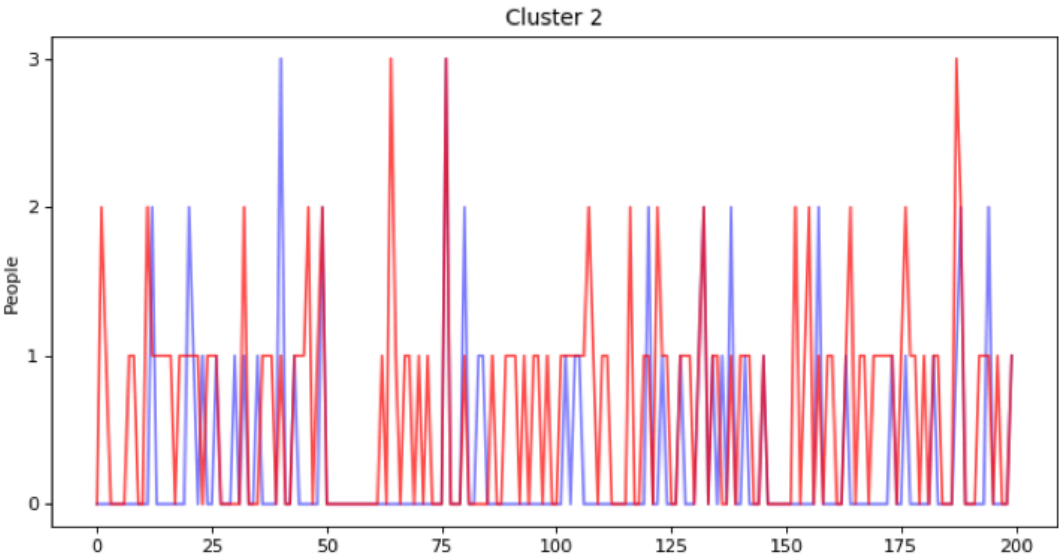


Figure 5.8 群體 6 始點到達群體 2 的真實結果與預測結果比對折線圖

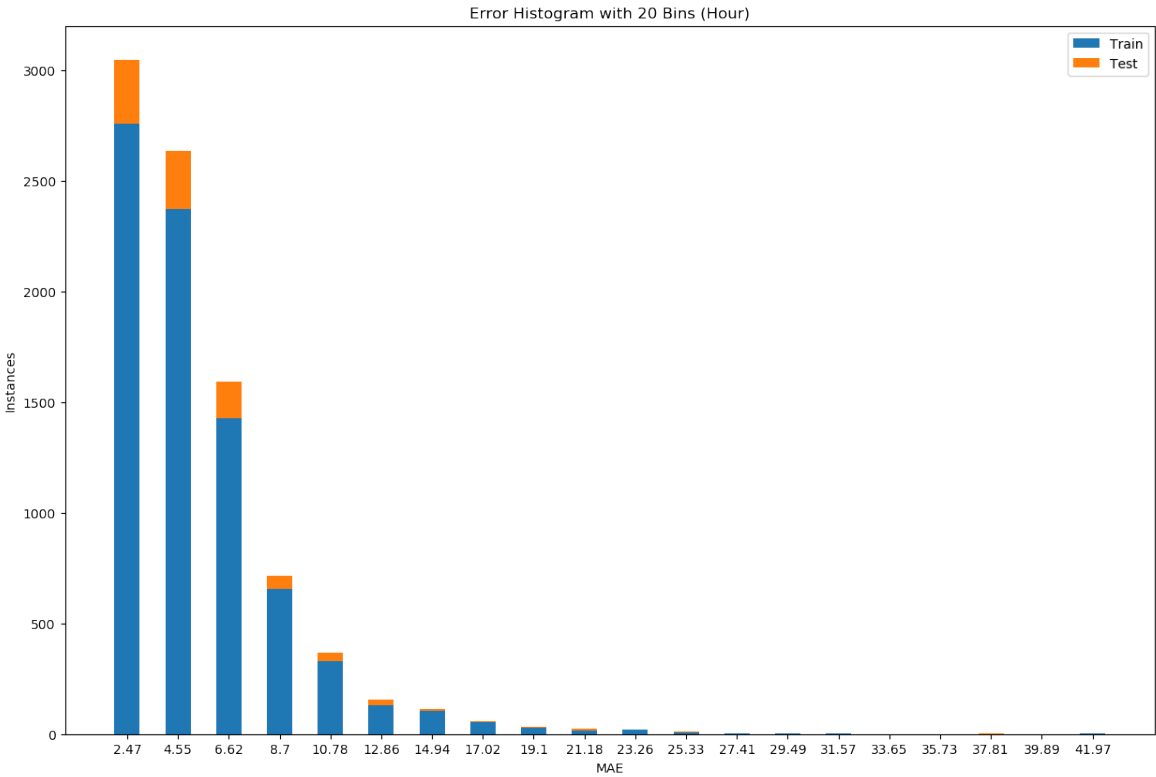


Figure 5.9 以小時作為單位的 Error Histogram

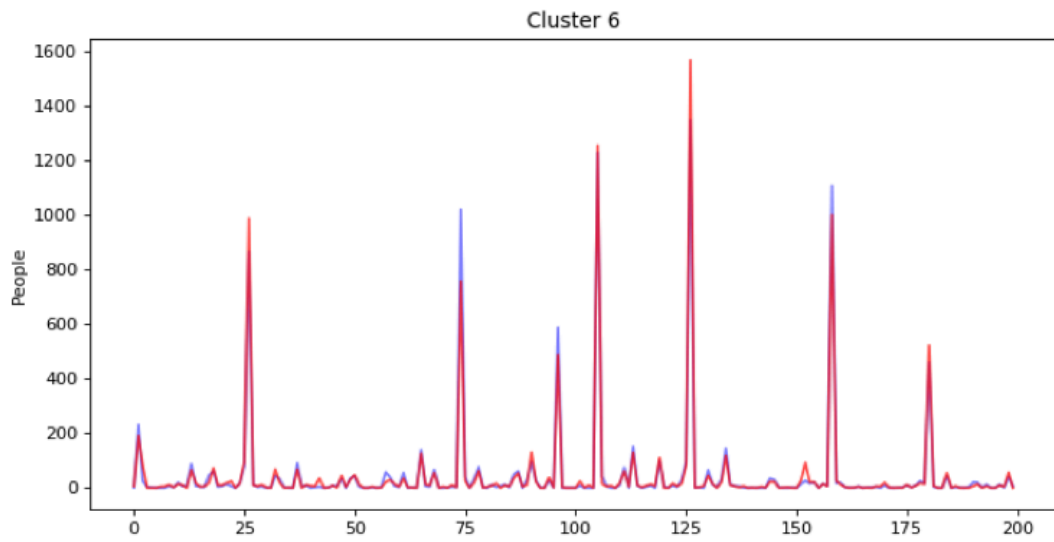
Figure 5.9 以小時作為單位的 Error Histogram，該圖表用作統計每一個 MAE 誤差之間分別有多少個案例，藍色是代表訓練的資料集，橘色是測試的資料集，從圖中我們可以看出大部份的訓練與測試的資料集都的 MAE 都是在 8.7 以下，即大部份的資料誤差都 8 至 9 個人或以下。由此可見，以本論文提出的方法在針對小時為單位進行預測是十分有效的。

5.2.2.2 時段

時段即是把一天的切割成 8 個時段，每個時段由三個小時組成，切割的方式跟據 Table 5.2。以 Table 5.7 中的 MSE 表示每筆資料誤差 3 至 4 個人。Figure 5.9 我們擷取幾個不同的群體進行比對，Figure 5.9(a)、(b)與(c)分別是所有群體中站點數最多的頭三個群體，首先 Figure 5.9(a)是所有群體預測到達群體 6 的資料，其站點數為 35 個，位於信義區。其次是 Figure 5.9(b)，所有群體預測到達群體 10 的資料，站點數為 29 個，位於信義區。站點數第三多的是群體 12，Figure 5.9(c)，共有 26 個站點，位於大安區。最後是 Figure 5.9(d)台北市的南港區，該區域是台北市的上班及企業區，使用 YouBike 的到達該區域的流量幅度相對其他群體較為明顯，從圖中也可以清楚明顯預測的結果與真實的結果是十分相近的。

Table 5.7 以時段為時間單位預測

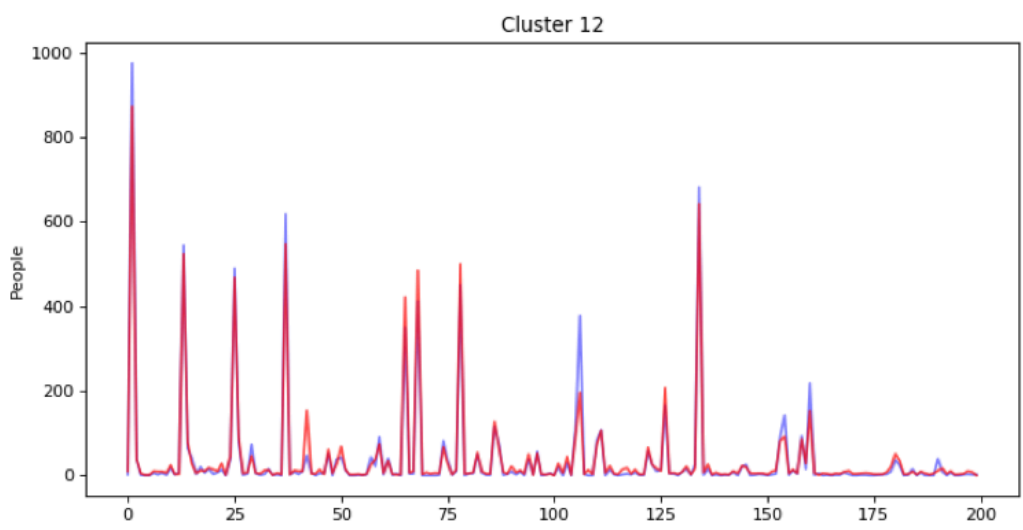
Dataset	Data Size	Epoch	Batch	Metric	LSTM_CNN_DNN
2016 年 台北市 YouBike	40978	100	30	RMSE	7.70
				MAE	3.45



(a)



(b)



(c)

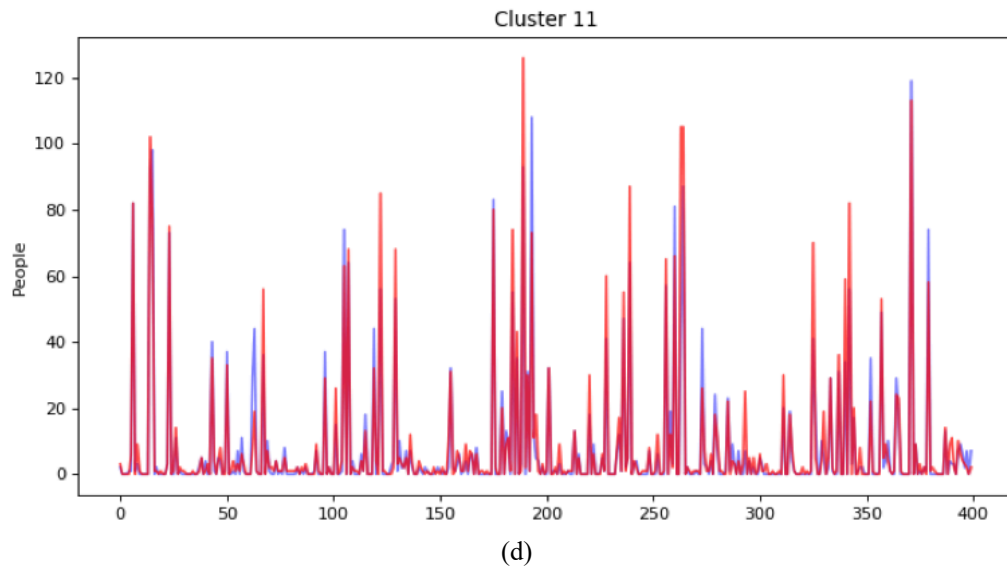


Figure 5.10 以時段為時間單位之真實結果與預測結果比對折線圖

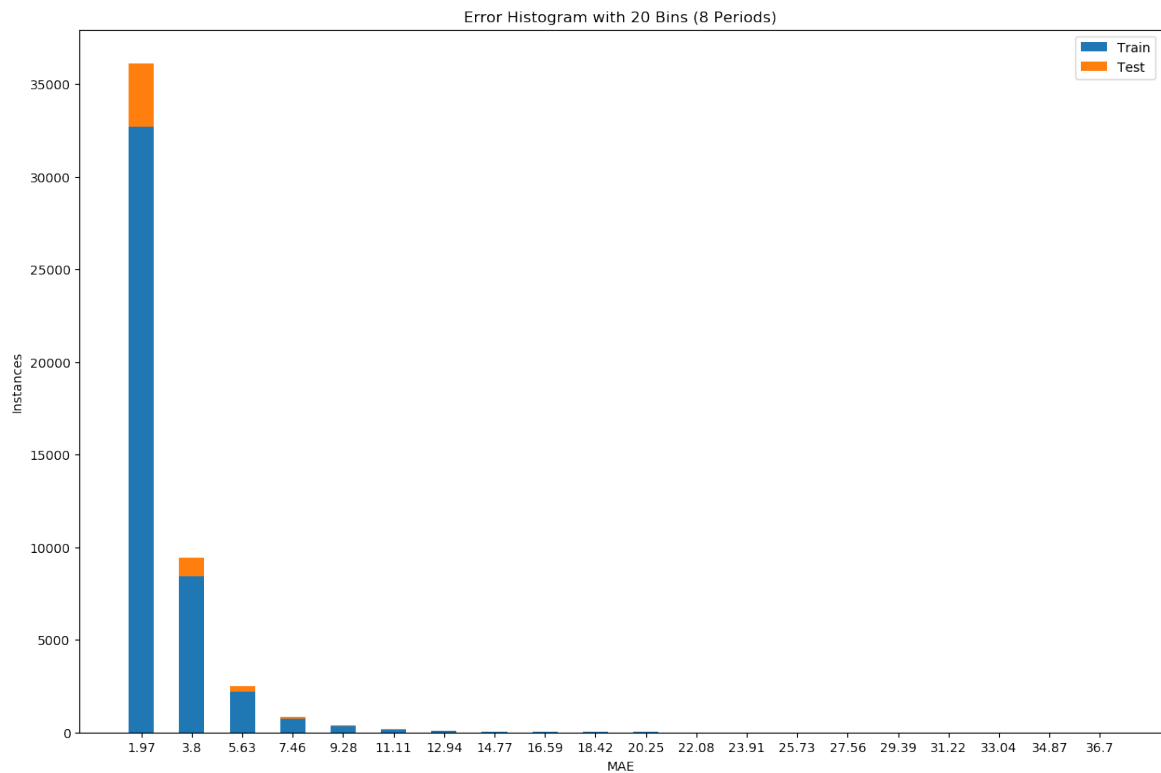


Figure 5.11 以時段作為單位的 Error Histogram

Figure 5.11 以時段作為單位的 Error Histogram，從圖中大部份的訓練與測試的資料集都的 MAE 都是在 3.8 以下，即大部份的資料誤差都 3 至 4 個人或以下。其餘的相對較多的是 3.8 至 5.63 的 MAE。在研究的過程中，我們發現利用模型同時訓練所群體時，模型有效於學習群體與群體間的流量變化，因此誤差相對小時單位而言較低。

5.2.2.3 天

Table 5.8 是以天為時間單位預測，同時學習所有群同的 OD 矩陣，Epoch 設定為 100，Batch 則為 30。從 Table 5.8 我們可以看到 RMSE 與 MAE 分別為 63.72 與 31.88，相對上述小時與時段兩個單位而言頗大。而我們從 Figure 5.10 了解當中的問題，我們一樣取出群體中站點數較多的頭三個群體，分別群體 6、群體 10 與群體 12。另外我們也取出最偏遠的群體 2，其站點數為 24 個，位於北投。

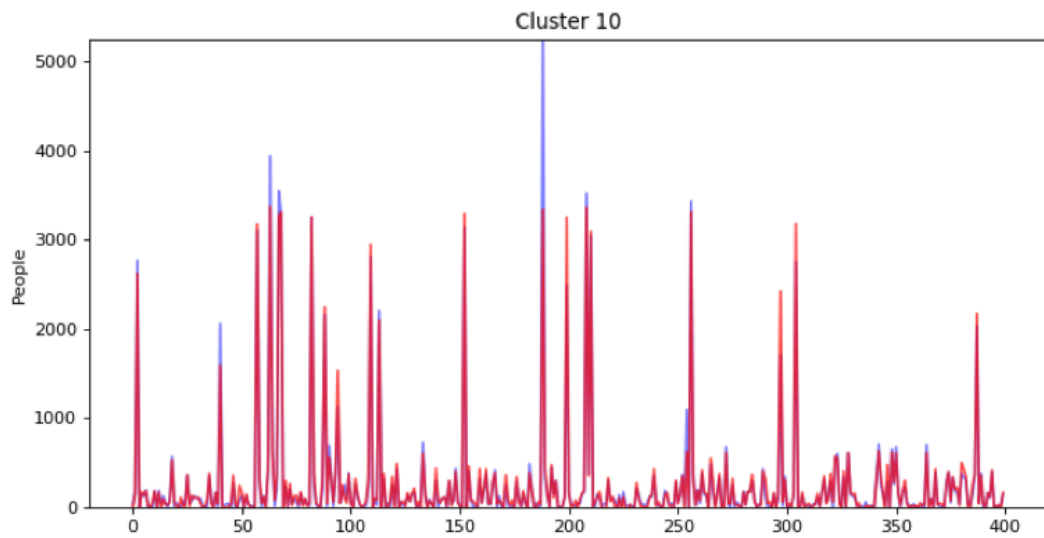
我們從 Figure 5.10 中我們可以了解每個群體基本每筆資料到達的人流數量高達千位數。以 Figure 5.10 (a)而言，其他群體到達群體 6 的一天人流數量更達到 7000 至 8000 人，對應 Table 5.8 的 MAE，平均每筆資料只落差 31.88 個人，而且預測的結果與真實的結果也是相對十分貼近。另外我們也分析，RMSE 與 MAE 誤差頗大的原因除了上述一天人流數量過多影響誤差的幅度外，以天為單位預測，難以精準考量每個小時的外在因素狀況(如天氣)或突發情況，導致 RMSE 與 MAE 誤差頗大的問題。

Table 5.8 以天為時間單位預測

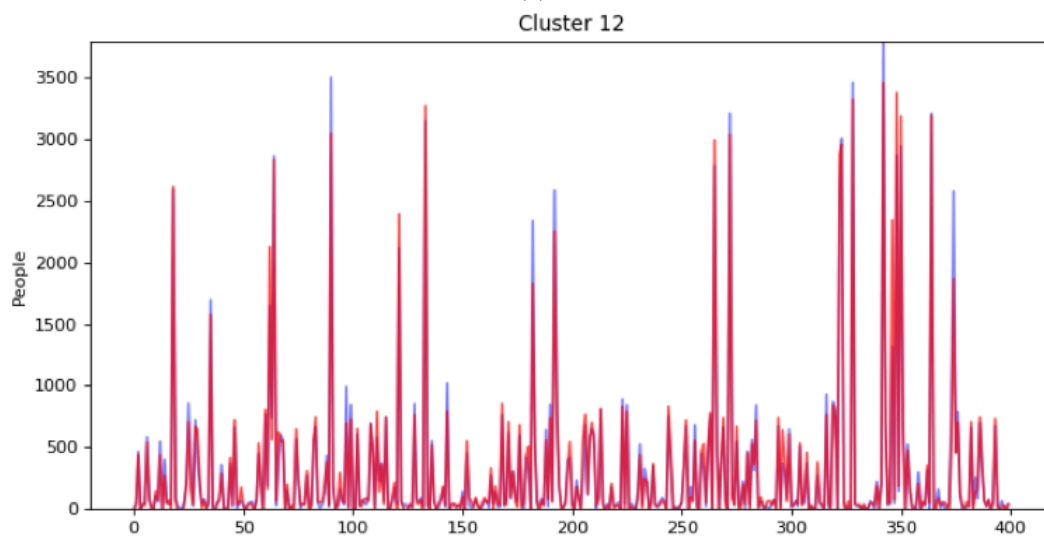
Dataset	Data Size	Epoch	Batch	Metric	LSTM_CNN_DNN
2016 年 台北市 YouBike	6171	100	30	RMSE	63.72
				MAE	31.88



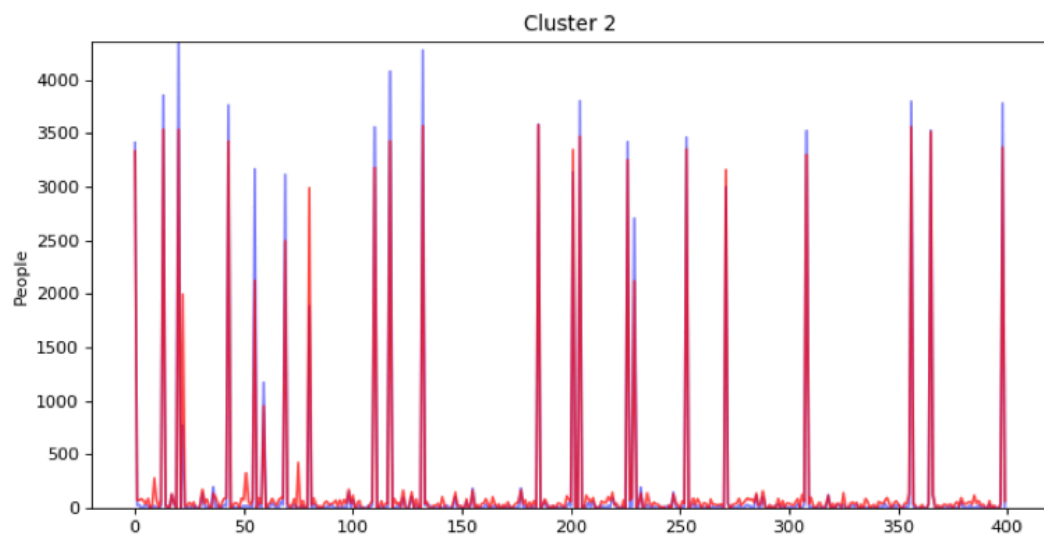
(a)



(b)



(c)



(d)

Figure 5.12 以天為時間單位之真實結果與預測結果比對折線圖

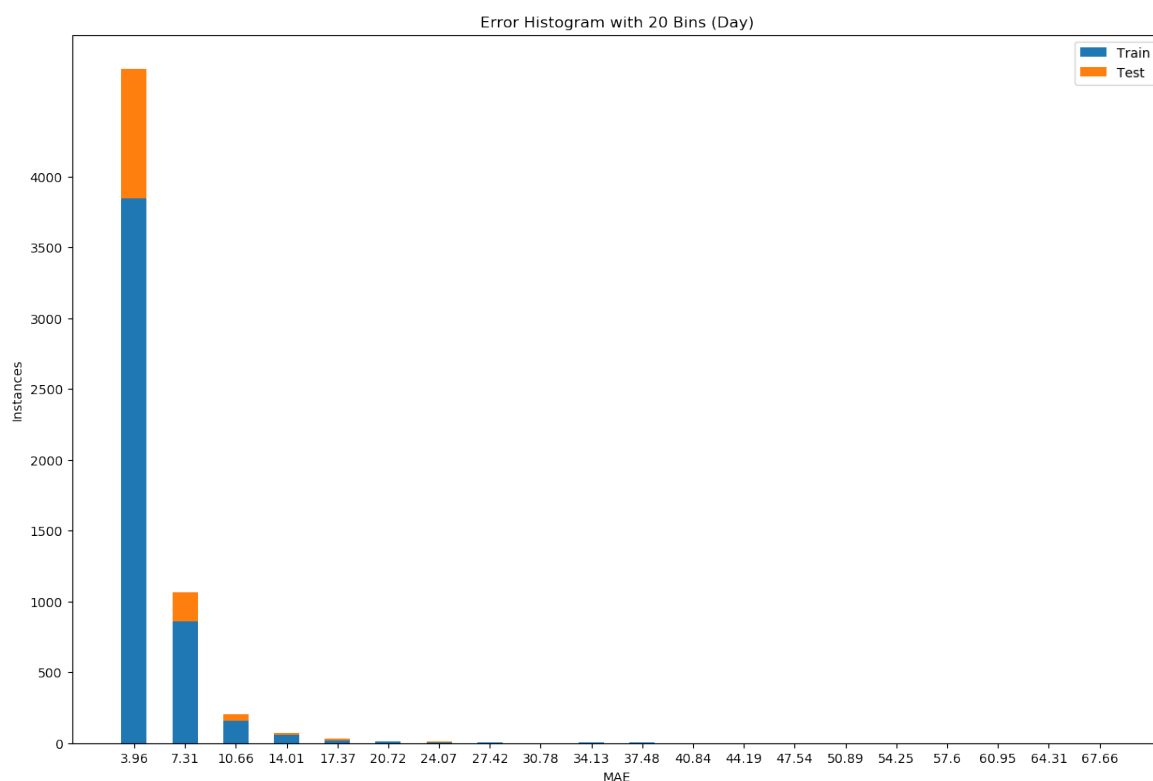


Figure 5.13 以天作為單位的 Error Histogram

Figure 5.13 以天作為單位的 Error Histogram，天與時段一樣是針對所有群體進行學習及預測，因此模型一樣學習不同群體之間的變化，從圖中我們可以清楚了解大量資料 MAE 分別在 3.96 以下，其次是在 3.96 至 7.31 之間，而少量的資料是落在 10.66 或以後。

5.2.3 單一群體及所有群體預測

為了表示本論文的始點針對單一群體及所有群體預測都能有優秀的效果，因此我們針對不同的群體進行單一群體的預測及所有群體進行預測。以下我們會在 5.2.3.1 把始點以不同的群體進行單一學習及預測的實驗。在 5.2.3.2 則輸入所有的群體作為始點，進行所有群體預測的實驗。並針對不同項目進行描述。

5.2.3.1 單一群體預測

本項目是我們只輸入單一的群體作為我們的始點，預測該單一群體到達其他群體的 YouBike OD 矩陣。以群體 6 為例，則代表輸入從群體 6 為起點的資料進行學習，

預測群體 6 到達其他群體的 YouBike 人流量。

在單一群體預測中，我們以小時作為單位，針對所有群體中站點數最多的群體進行預測，分別是群體 6 (35 個站點)、群體 10 (29 個站點)與群體 12 (26 個站點)，三個群體分別位於信義區、中正區及大安區。

從 Table 5.9 表示，在不同的群體下本論文所提出的方法鏈對單一群體預測都能有效進行預測，三個群體平均每筆資料分別誤差 2 至 5 個人，這證明本論文的方法以小時為單位針對單一群體預測是十分有效的。

Table 5.9 以小時為單位預測及學習單一群體的 OD 矩陣

Dataset	Data Size	Epoch	Batch	Cluster (Size)	Metric	LSTM_CNN_DNN
2016 年 台北市 YouBike	8781	100	30	6 (35)	RMSE	9.87
					MAE	4.50
				10 (29)	RMSE	5.37
					MAE	2.81
				12 (26)	RMSE	6.11
					MAE	3.31

5.2.3.2 所有群體預測

所有群體預測則是跟 5.2.3.1 相反，我們輸入的起點會是所有的群體，藉此表示所有群體預測在本論文所提出的方法是有效的。而我們針對所有群體預測是利用時段為單位，該結果與章節 5.2.2.2 以時段為單位預測相同，這證明所有群體預測是有效應用在本論文中。

Table 5.10 以時段為單位預測及學習所有群體的 OD 矩陣

Dataset	Data Size	Epoch	Batch	Metric	LSTM_CNN_DNN
2016 年 台北市 YouBike	40978	100	30	RMSE	7.70
				MAE	3.45

5.2.4 模型因素分析

模型因素的分析對於我預測有很大的幫助，我們可以從中了解個種因素對於預測是有幫助，而哪一種因素則是對模型有反效果，而在本章節中我們會針對不同的因素進行分析，我們會以小時與時段兩個不同的時間單位進行分析，藉此更可靠證明不同因素的重要性。

5.2.4.1 模型因素分析 - 小時

本章節中我們以小時為單位，證明不同的因素的重要性，以 Table 5.11 為例，首先 LSTM_CNN 則加入歷史與空間兩個因素，LSTM_DNN 為加入歷史與外在兩個因素，最後 LSTM_CNN_DNN 則是加入了歷史、空間與外在所有因素進行學習。

Table 5.11 是以小時為時間單位，預測單一群體 6 各種模型因素比對分析表，Table 5.11 中相對其他模型而言表現最差為 LSTM_CNN，而 LSTM_CNN 與 LSTM_CNN_DNN 效果則是相差一點而已，我們先從 Figure 5.11(a) LSTM_CNN 預測自己群體觀察，從圖中我們可以發現 LSTM_CNN 是有跟據真實結果進行學習，但與真實結果未必十分相似。相反 Figure 5.11 (b)的 LSTM_DNN 則與真實結果十分相似，我們可以從中推斷，外在因素相對空間因素更為重要，原因是外在因素所包含的天氣與時間等資訊其實是直接反映 OD 矩陣的真實狀況，而空間因素主要是表示距離與影響範圍，相對於外在因素影響 OD 矩陣流量的程度稍微較低。

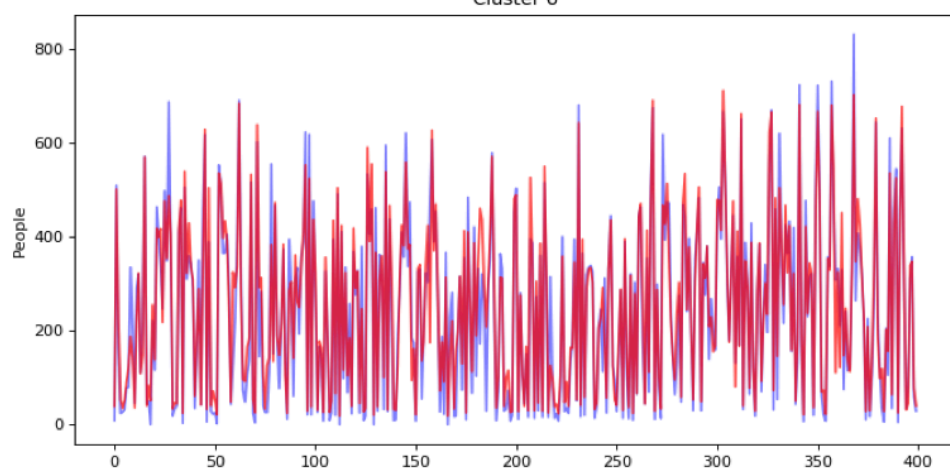
而我們從當包含所有完素的 Figure 5.11 (c) LSTM_CNN_DNN 中，他的結果相對於 LSTM_DNN 更加貼近真實的結果，特別在低流量時，LSTM_CNN_DNN 預測的結果更能準確預測。從 Figure 5.11 LSTM_CNN_DNN 的結果發現，外在因素是預測準確的最大因素，而空間因素則是輔助模型預測，讓模型更能貼近真實的資料狀況。

Table 5.11 各種模型因素比對分析 - 小時

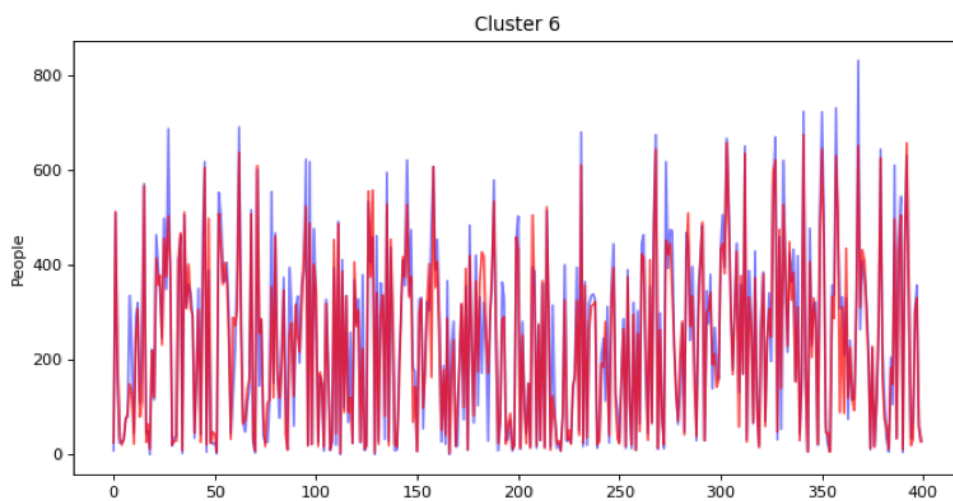
Dataset	Metric	LSTM_CNN	LSTM_DNN	LSTM_CNN_DNN
2016 年 台北市	RMSE	17.23	10.19	9.87
YouBike	MAE	7.09	4.64	4.50



(a) LSTM_CNN



(b) LSTM_DNN



(c) LSTM_CNN_DNN

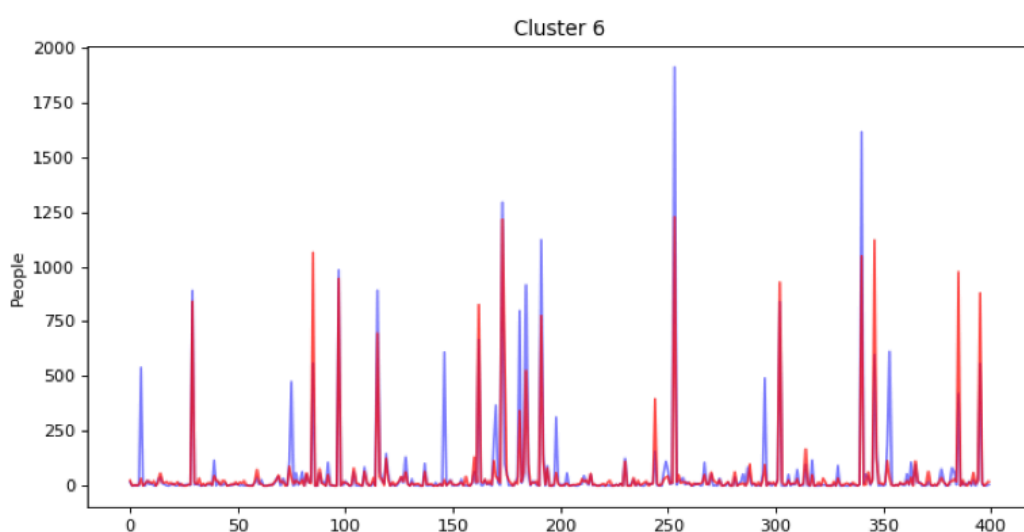
Figure 5.14 各種模型因素真實與預測結果比對圖 - 小時

5.2.4.2 模型因素分析 - 時段

Table 5.12 是以時段進行各種模型因素比對分析，其結果與 5.2.4.1 章節相似，LSTM_CNN 模型效果相對效差，LSTM_DNN 與 LSTM_CNN_DNN 效果則十分相近，從 Figure 5.12(a) LSTM_CNN 我們可以發現與 Figure 5.12(a)相同，部分的資料與真實的資料差異甚大，但加入外在因素的 Figure 5.12(b) LSTM_DNN 則預測更加準確，把 Figure 5.12(a) LSTM_CNN 無法預測的結果，也預測得更貼近真實結果。最後 Figure 5.12(b) LSTM_CNN_DNN 則對 LSTM_CNN 所預測的誤差進行學習，讓模型的效果更加準確。

Table 5.12 各種模型因素比對分析 - 時段

Dataset	Metric	LSTM_CNN	LSTM_DNN	LSTM_CNN_DNN
2016 年 台北市	RMSE	18.61	8.22	7.70
YouBike	MAE	8.09	3.84	3.45



(a) LSTM_CNN

從以上以小時與時段兩個時間單位進行模型因素的分析，我們可以發現對於預測最大幫助的是外在因素，外在因素所包含的天氣、時間等資訊，其實直接後映 OD 矩陣的人流狀況。相反空間因素如距離與影響範圍等資訊則像是輔助的角色，把預測的結果更加貼近真實的答案。因此我們可以證明本論文所提出的深度學習架構在預測區域 OD 矩陣是十分有效的。



(b) LSTM_DNN



(c) LSTM_CNN_DNN

Figure 5.15 各種模型因素真實與預測結果比對圖- 時段

5.2.5 輸入歷史時間點數量分析

輸入歷史時間點數量無疑會影響預測的結果，因此我們針對不同的輸入歷史時間點時長進行分析。

Table 5.13 輸入歷史時間點數量分析是以小時為時間單位進行預測，該項目最主要針對給定前 3、5 與 7 個歷史時間點進行分析，3 個時間點則代表給定前 3 個小時的歷史資料，而 5 則給定前 5 個小時的歷史資料，如此類推。我們可以從 Table 5.13 中發現隨著給定的更長遠的歷史資料，誤差也隨之下降，誤差然而下降的幅度也跟著下降，因此我們可以從中了解給定更長遠的歷史資料是有效於提升預測的效果，但給

定過於長遠的歷史資料未必能與預測校果成正比的上升，或有可能造成反效果，因此我們必需跟據自身的資料進行衡量，從而決定給定多預時長的歷史資料。

Table 5.13 輸入歷史時間點數量分析

Dataset	Data Size	Epoch	Batch	Num of prev. time step	Metric	LSTM_CNN_DNN
2016 年 台北市 YouBike	8781	100	30	3	RMSE	9.87
					MAE	4.50
				5	RMSE	9.14
					MAE	4.27
				7	RMSE	9.06
					MAE	4.18

VI. Conclusion

在本章節中，我們會分成兩個部分，在 6.1 中會描述本論文的優點與貢獻。在 6.2 中則討論在研究中所得出的結論。

6.1 優點與貢獻

本論文的優點與貢獻最主要分為三個，首先本論文開發了一套利用歷史交通資料預測未來區域 OD 矩陣框架，該框架減低 OD 矩陣的使用資源及有效預測不同的群體。我們提出新的分群方法進行站點的分群，減少 OD 矩陣起終點過多的問題。然後再以多角度考慮利用深度學習進行預測，其效果在章節 5 中證明有著優秀的表現。

另外，我們提出新的分群方法 HTS-DBSCAN，HTS-DBSCAN 能夠解決 DBSCAN 半徑太短造成雜訊點過多，以及半徑太長導致站點無法分割的問題。除此之外，HTS-DBSCAN 能夠考慮時間序列進行分群，面對交通資料能夠有效針對時間與空間兩個因素進行分群。

再者，我們提出了新的深度學習框架以預測區域 OD 矩陣，該深度學習框架有效考慮歷史、空間與外在因素，預測未來區域 OD 矩陣。而且我們提出了空間因素圖片，用作表示群體距離與群體中站點的影響範圍，其效果另模型預測更加貼近真實的結果，有助提升模型的效果。

6.2 結論

在本論文的研究中，我們得三個重要的結論：

首先，我們發現針對單一群體預測並非一定優於所有群體預測，所有群體預測可互相學習群體間的特性。大多數人認為模型針對單一群體學習，能減輕模型學習的負擔，但在 5.2.3 中我們可以發現模型同時預測所有群體時，模型會學習每個群體的相關性，利用不同群體學習群體間重要的變化，從而提升模型的校果。

接著，是台北市捷運站點的位置間接影響群體的人流量，我們發現很多 YouBike

站點都在捷運附近，而台北市捷運附近的 YouBike 站點與其他站點相比流量都會大幅增加，因此我們在未來可以加入台北市捷運等相關因素，減低捷運附近的 YouBike 站點的誤差問題。

最後是空間因素無法獨立進行預測，需依賴外在因素才能提升效果。在 5.2.4 中我們發現模型只單獨加入空間因素進行預測效果未如理想。當加入外在因素後，空間因素則發揮他的效用，讓預測的結果更加貼近真實的結果。

References

- [1] Lingbo Liu, Zhilin Qiu, Guanbin Li, Qing Wang, Wanli Ouyang, Liang Lin, "Contextualized Spatial–Temporal Network for Taxi Origin-Destination Demand Prediction," *IEEE Transactions on Intelligent Transportation Systems*, pp. 3875-3887, 2019.
- [2] Shudong Liu, Liaoyuan Zhang, Xu Chen, "Travel Destination Prediction Based on Origin-Destination Data," *Complex, Intelligent and Software Intensive Systems*, pp. 315-325, 2020.
- [3] Y. Englezou, Stelios Timotheou, Christos G. Panayiotou, "Bayesian estimation of the Origin-Destination matrix using traffic flow dynamics.," *International Conference on Intelligent Transportation Systems*, pp. 2545-2550, 2019.
- [4] Adrien Ickowicz, Ross Sparks:, "Estimation of an origin/destination matrix: application to a ferry transport data.," *Public Transport - Planning and Operations*, pp. 235-258, 2015.
- [5] Juha-Matti Kuusinen, Janne Sorsa, Marja-Liisa Siikonen, "The Elevator Trip Origin-Destination Matrix Estimation Problem.," *Transportation Science*, pp. 559-576, 2015.
- [6] Yuandong Wang, Hongzhi Yin, Hongxu Chen, Tianyu Wo, Jie Xu, Kai Zheng, "Origin-Destination Matrix Prediction via Graph Convolution: a New Perspective of Passenger Demand Modeling," *KDD*, pp. 1227-1235, 2019.
- [7] Huaxiu Yao, Fei Wu, Jintao Ke, Xianfeng Tang, Yitian Jia, Siyu Lu, Pinghua Gong, Jieping Ye, Zhenhui Li, "Deep Multi-View Spatial-Temporal Network for Taxi Demand Prediction," *AAAI*, pp. 2588-2595, 2018.
- [8] Anahid Nabavi Larijani, Ana-Maria Olteanu-Raimond, Julien Perret, Mathieu Brédif, Cezary Ziemlicki, "Investigating the mobile phone data to estimate the origin destination flow and analysis; case study: Paris region," *Transportation Research Procedia*, pp. 64-78 , 2015.
- [9] Junbo Zhang, Yu Zheng, Dekang Qi, "Deep Spatio-Temporal Residual Networks for Citywide Crowd Flows Prediction.," *AAAI*, pp. 1655-1661, 2017.
- [10] Martin Ester, Hans-Peter Kriegel, Jiirg Sander, Xiaowei Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *KDD*, 1996.
- [11] "ST-DBSCAN: An algorithm for clustering spatial – temporal data," *Data &*

Knowledge Engineering, pp. 208-221 , 2007.

- [12] Leland McInnes, John Healy, and Steve Astels, “hdbscan: Hierarchical density based clustering,” *The Journal of Open Source Software*, 2017.
- [13] Karen Simonyan, Andrew Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *ICLR*, 2015.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, “Deep Residual Learning for Image Recognition,” *CVPR*, pp. 770-778 , 2016.
- [15] Gao Huang, Zhuang Liu, Laurens van der Maaten, Kilian Q. Weinberger, “Densely Connected Convolutional Networks,” *CVPR*, 2016.
- [16] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, Jian Sun, “ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices,” *CVPR*, 2017.
- [17] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, Yoshua Bengio, “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling,” *Neural and Evolutionary Computing*, 2014.
- [18] Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-kin Wong, Wang-chun Woo, “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting,” *CVPR*, 2015.
- [19] Guokun Lai, Wei-Cheng Chang, Yiming Yang, Hanxiao Liu, “Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks,” *SIGIR*, 2018.
- [20] Qiang Liu, Shu Wu, Liang Wang, Tieniu Tan, “Predicting the Next Location: A Recurrent Model with Spatial and Temporal Contexts,” *AAAI*, 2016.
- [21] Shanshan Feng, Gao Cong, Bo An, Yeow Meng Chee, “POI2Vec: Geographical Latent Representation for Predicting Future Visitors,” *AAAI*, 2017.
- [22] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, Jeffrey Dean, “Distributed Representations of Words and Phrases and their Compositionality,” *NIPS*, 2013.
- [23] Jiangtao Ren, Qiwei Xie, “Efficient OD Trip Matrix Prediction Based on Tensor Decomposition,” *MDM*, pp. 180-185, 2017.
- [24] Yaguang Li, Kun Fu, Zheng Wang, Cyrus Shahabi, Jieping Ye, Yan Liu, “Multi-task Representation Learning for Travel Time Estimation,” *KDD*, 2018.
- [25] “台灣政府資料開放平台,” [線上]. Available: <https://data.gov.tw>.
- [26] “<https://taipei.youbike.com.tw/>,” [線上]. Available: YouBike 公共自行車.
- [27] “Weather Underground,” [線上]. Available: <https://www.wunderground.com>.
- [28] Benjamin Moseley, Joshua R. Wang, “Approximation Bounds for Hierarchical

- Clustering: Average Linkage, Bisecting K-means, and Local Search,” *NIPS*, 2017.
- [29] Y. Wu, S. Yang, and X. Yan, “Ontology-based subgraph querying.,” *ICDE*, p. 697 – 708, 2013.